Benemérita Universidad Autónoma de Puebla

Facultad de Ciencias de la Computación



T E S I S

"Predicción de congestión en redes de computadoras utilizando técnicas de Machine Learning"

Presenta:
Ricardo Valera Martínez

Para obtener el grado de:
Licenciatura en Ingeniería en Ciencias de la Computación

Director: **Mauro Alberto López Muñoz**
Codirector: **Guillermo Colorado Jiménez**

Puebla, Pue., Febrero 2026

Índice general

1. Introducción **10**
- 1.1. Planteamiento del problema . . . 10
- 1.2. Objetivos . . . 12
  - 1.2.1. Objetivo General . . . 12
  - 1.2.2. Objetivos Específicos . . . 12
- 1.3. Justificación . . . 12
- 1.4. Alcances y Limitaciones . . . 14

2. Marco Teórico **16**
- 2.1. Fundamentos de redes de computadoras . . . 16
  - 2.1.1. Capacidad del enlace: ancho de banda, throughput y utilización . . . 17
  - 2.1.2. Modelo OSI y TCP/IP . . . 18
  - 2.1.3. Concepto de congestión . . . 19
  - 2.1.4. Métricas de rendimiento . . . 22
- 2.2. Técnicas de *Machine Learning* aplicables . . . 25
  - 2.2.1. Aprendizaje supervisado: regresión vs. clasificación . . . 25
  - 2.2.2. Modelos de series temporales (ARIMA) . . . 26
  - 2.2.3. Modelos de aprendizaje automático clásico (Random Forest, SVR) . . . 26
  - 2.2.4. Modelos de Aprendizaje Profundo (LSTM, GRU) . . . 27
  - 2.2.5. Métricas de evaluación . . . 28

3. Estado del Arte **30**
- 3.1. Revisión de trabajos sobre predicción de tráfico con métodos estadísticos . . . 30
- 3.2. Predicción de tráfico y congestión con *Machine Learning* clásico . . . 31
- 3.3. Predicción con *Deep Learning* . . . 31
- 3.4. Herramientas de monitoreo y *datasets* utilizados . . . 31
- 3.5. Síntesis del estado del arte y brecha a cubrir . . . 34

4. Metodología **39**
- 4.1. Tipo de investigación . . . 39
- 4.2. Definición operativa de congestión . . . 41
- 4.3. Descripción de las métricas a utilizar . . . 42
- 4.4. Fuente de datos . . . 45
- 4.5. Preprocesamiento de datos . . . 47
- 4.6. Modelos a implementar y justificación de su elección . . . 51

4.7. Métricas de evaluación del modelo 54
4.8. Herramientas y entorno 57

5. Desarrollo e Implementación **60**
5.1. Obtención y exploración de los conjuntos de datos 60
5.2. Preprocesamiento 66
5.3. Construcción del modelo ARIMA 71
5.4. Construcción del modelo *Random Forest* 72
5.5. Construcción del modelo LSTM 73

6. Resultados y Discusión **75**
6.1. Resultados de ARIMA 76
6.2. Resultados de *Random Forest* 86
6.3. Resultados de LSTM 97
6.4. Comparativa de modelos 109
6.5. Discusión 115

7. Conclusiones **121**
7.1. Contribuciones 122
7.2. Limitaciones del estudio 123
7.3. Trabajo futuro 127

Referencias bibliográficas **129**

A. Código fuente de ejemplos críticos **132**
A.1. Decodificador de trazas libpcap 132
A.2. Script de recolección en RouterOS 134
A.3. Pipeline de preprocesamiento 135
A.4. Generador de datos sintéticos 137
A.5. Entrenamiento y evaluación de modelos 140

Índice de figuras

2.1. Representación de una red entre dos computadoras. 16
2.2. Comparativa entre las capas del modelo OSI y la arquitectura TCP/IP. . . . 19
2.3. Ubicación del problema de predicción de congestión dentro de los paradigmas de *Machine Learning*. 26

3.1. Taxonomía de las técnicas de monitoreo de red empleadas en la literatura revisada. 32
3.2. Línea de tiempo de la evolución metodológica en predicción de tráfico y congestión de red. 36

4.1. *Pipeline* de transformación de métricas crudas en variables de entrada y variable objetivo. 45
4.2. Construcción de secuencias de entrada mediante ventana deslizante para los modelos LSTM/GRU. 50
4.3. División cronológica del *dataset* primario en conjuntos de entrenamiento, validación y prueba. 51
4.4. Diseño experimental general: entrenamiento sobre el *dataset* primario y validación cruzada contra MAWI. 54
4.5. Matriz de confusión para la tarea de clasificación de congestión. 55

5.1. Patrón de tráfico promedio y máximo por hora del día, *dataset* primario. . . 62
5.2. Distribución de `rx_bps` (escala logarítmica en frecuencia), *dataset* primario. . 62
5.3. Distribución de duración e intensidad de los 126 eventos de congestión sintéticos. 63
5.4. Evento sintético inyectado: la utilización determina el RTT y la pérdida de paquetes según la Teoría de Colas. 64
5.5. Distribución del factor de utilización, *dataset* piloto MAWI. 66
5.6. Matriz de distribución de utilización normalizada: contraste entre el espectro del entrenamiento (arriba izq.), la contención del tráfico real (arriba centro/der.) y la masa anómala inyectada (abajo). 70

6.1. ARIMA(1,1,1): pronóstico progresivo ($h = 5$) frente al valor real, partición de prueba del piloto MAWI. 78
6.2. ARIMA(3,0,3): predicción frente al valor real, prueba de regresión (*dataset* primario, datos reales). 79
6.3. Comparación de métricas de clasificación, validación frente a prueba, ARIMA sobre el *dataset* primario. 81

6.4. ARIMA(3,0,3): predicción frente al valor real y umbral $U_{th} = 0{,}80$, prueba de clasificación (*dataset* primario, datos sintéticos aislados). 82
6.5. Generalización: ARIMA(3,0,3) entrenado con el *dataset* primario y evaluado sobre MAWI. 83
6.6. RMSE de regresión de ARIMA en los contextos de evaluación. 84
6.7. *Random Forest*: predicción frente al valor real, partición de prueba del piloto MAWI (escala normalizada). 86
6.8. Importancia relativa de variables en *Random Forest*, *dataset* piloto MAWI. . 88
6.9. *Random Forest*: predicción frente al valor real, validación de regresión (*dataset* primario, datos reales). 89
6.10. *Random Forest*: predicción frente al valor real, prueba de regresión (*dataset* primario, datos reales). 89
6.11. RMSE de regresión de *Random Forest* en los contextos de evaluación (escala normalizada). 90
6.12. Comparación de métricas de clasificación, validación frente a prueba, *Random Forest* sobre el *dataset* primario. 91
6.13. *Random Forest*: predicción frente al valor real y umbral $U_{th} = 0{,}80$, validación de clasificación (*dataset* primario, datos sintéticos aislados). 92
6.14. *Random Forest*: predicción frente al valor real y umbral $U_{th} = 0{,}80$, prueba de clasificación (*dataset* primario, datos sintéticos aislados). 93
6.15. Importancia relativa de variables en *Random Forest*, *dataset* primario. 94
6.16. Importancia relativa de variables en *Random Forest*: piloto frente al *dataset* primario. 94
6.17. Curva de aprendizaje de la red LSTM (pérdida MSE por época), *dataset* piloto MAWI. 97
6.18. Curva de aprendizaje de la red LSTM, *dataset* primario. La separación entre curvas refleja la composición de las particiones: entrenamiento con eventos sintéticos (azul) y validación con tráfico real ocioso (naranja). 98
6.19. Curva de aprendizaje en ejes independientes. Se aprecia la dinámica de validación en el orden de 10^{-4} y el punto de corte de la detención temprana. . . 99
6.20. Curva de aprendizaje de la red LSTM en escala logarítmica, *dataset* primario. 100
6.21. LSTM: predicción frente al valor real, partición de prueba del piloto MAWI (escala normalizada). 101
6.22. LSTM: predicción frente al valor real, validación de regresión (*dataset* primario, datos reales). 102
6.23. LSTM: predicción frente al valor real, prueba de regresión (*dataset* primario, datos reales). 103
6.24. RMSE de regresión de LSTM en los contextos de evaluación (escala logarítmica). 104
6.25. Comparación de métricas de clasificación, validación frente a prueba, LSTM sobre el *dataset* primario. 105
6.26. LSTM: predicción frente al valor real y umbral $U_{th} = 0{,}80$, validación de clasificación (*dataset* primario, datos sintéticos aislados). 106
6.27. LSTM: predicción frente al valor real y umbral $U_{th} = 0{,}80$, prueba de clasificación (*dataset* primario, datos sintéticos aislados). 106
6.28. *Recall* de clasificación (prueba) de los tres modelos sobre el *dataset* primario. 107

6.29. Métricas de clasificación de los tres modelos en el *dataset* primario (partición de prueba). 112
6.30. RMSE de regresión (eje X) frente a *recall* de clasificación (eje Y), *dataset* primario, partición de prueba. 113
6.31. Comparación multidimensional de los tres modelos (*dataset* primario, partición de prueba). 114
6.32. Esquema conceptual de un sistema de alertamiento híbrido. No fue implementado ni evaluado en este trabajo. 119

Índice de cuadros

2.1. Resumen de métricas de rendimiento de red utilizadas en la investigación. . . 24
2.2. Resumen comparativo de los modelos de predicción considerados en esta tesis. 29

3.1. Comparación de herramientas de monitoreo empleadas en la literatura revisada. 32
3.2. Comparación de datasets empleados en la literatura revisada frente a los datasets adoptados en esta tesis. 33
3.3. Síntesis comparativa de los trabajos revisados en el Estado del Arte. 35
3.4. Posicionamiento metodológico de esta tesis frente a la literatura revisada. . . 37

4.1. Clasificación del tipo de investigación de esta tesis. 41
4.2. Variables de entrada consideradas para los modelos predictivos. 43
4.3. Disponibilidad y método de obtención de métricas según la fuente de datos. . 44
4.4. Contraste del proceso de resampleo entre las dos fuentes de datos empleadas. 49
4.5. Resumen de las etapas de preprocesamiento de datos. 51
4.6. Modelos considerados durante la planeación de esta tesis y justificación de su exclusión del alcance experimental. 53
4.7. Rol de cada métrica de regresión en el análisis comparativo de esta tesis. . . 55
4.8. Síntesis del esquema de evaluación aplicado a los modelos de esta tesis. . . . 57
4.9. Herramientas empleadas para la recolección del *dataset* primario. 58
4.10. Bibliotecas de Python empleadas para la implementación de cada etapa metodológica. 58

5.1. Resumen de interrupciones de conectividad durante la recolección del *dataset* primario. 61
5.2. Estadísticas descriptivas del *dataset* primario (21 días reales, 25 422 observaciones). 61
5.3. Composición final del *dataset* primario tras la incorporación de datos sintéticos. 64
5.4. Resultados de la extracción de telemetría sobre las trazas MAWI procesadas. 65
5.5. Estadísticas descriptivas del *dataset* piloto MAWI (1 800 observaciones, 1 minuto). 66
5.6. Composición de la división cronológica del *dataset* piloto MAWI tras el preprocesamiento. 67
5.7. Sesiones continuas detectadas en el conjunto de entrenamiento del *dataset* primario. 69
5.8. Rango de utilización normalizada por partición, *dataset* primario. Los valores atípicos certifican la ausencia de fuga de información metodológica. 70

5.9. Configuración del modelo ARIMA. 72
5.10. Configuración del modelo *Random Forest*. 73
5.11. Configuración del modelo LSTM. 74

6.1. Identificación y diagnóstico del ajuste de ARIMA en cada conjunto de datos. 76
6.2. Órdenes ARIMA con menor AIC, *dataset* piloto MAWI. 76
6.3. Métricas de regresión de ARIMA(1,1,1), *dataset* piloto MAWI. 77
6.4. Clasificación y alertamiento (TAE) de ARIMA(1,1,1), fase piloto. 78
6.5. Métricas de regresión de ARIMA(3,0,3), *dataset* primario (escala original de utilización). 79
6.6. Métricas de clasificación de ARIMA(3,0,3), $U_{th} = 0{,}80$, datos sintéticos aislados. 80
6.7. Generalización de ARIMA(3,0,3) sobre MAWI (granularidad de 1 minuto). . 82
6.8. Síntesis de los resultados de ARIMA. N/A: no aplicable con el umbral oficial (Sección 5.1). 84
6.9. Métricas de regresión de *Random Forest* (escala normalizada), *dataset* piloto MAWI. 86
6.10. Clasificación y alertamiento (TAE) de *Random Forest*, fase piloto. 87
6.11. Métricas de regresión de *Random Forest*, *dataset* primario (escala normalizada). 88
6.12. Métricas de clasificación de *Random Forest*, $U_{th} = 0{,}80$, datos sintéticos aislados. 91
6.13. Importancia relativa de variables, *Random Forest*, *dataset* primario. 93
6.14. Comparación de *Random Forest* entre la fase piloto y el *dataset* primario. Las métricas de clasificación usan umbrales distintos y no son directamente comparables. 95
6.15. Síntesis de los resultados de *Random Forest*. N/A: no aplicable con el umbral oficial (Sección 5.1). Los RMSE están en escala normalizada. 95
6.16. Métricas de regresión de LSTM (escala normalizada), *dataset* piloto MAWI. 100
6.17. Clasificación y alertamiento (TAE) de LSTM, fase piloto. 101
6.18. Métricas de regresión de LSTM, *dataset* primario (escala normalizada). . . . 102
6.19. Métricas de clasificación de LSTM, $U_{th} = 0{,}80$, datos sintéticos aislados. . . . 104
6.20. Comparación de LSTM entre la fase piloto y el *dataset* primario. Los RMSE corresponden a series distintas y las métricas de clasificación usan umbrales distintos, por lo que no son directamente comparables. N/A: no aplicable con el umbral oficial (Sección 5.1). 107
6.21. Síntesis de los resultados de LSTM. N/A: no aplicable con el umbral oficial (Sección 5.1). Los RMSE están en escala normalizada. 108
6.22. Condiciones de evaluación de cada modelo en el *dataset* primario. 110
6.23. Tabla maestra de resultados de los tres modelos. Los RMSE de *Random Forest* y LSTM en el piloto están reescalados a la escala original. N/A: no aplicable con el umbral oficial (Sección 5.1). 110
6.24. Desempeño de los tres modelos en la fase piloto (partición de prueba, MAWI). Clasificación con $U_{test} = 0{,}25$. 111
6.25. Clasificación con $U_{th} = 0{,}80$ en la partición sintética de prueba. VP, FP y FN provienen de las matrices de confusión de las Secciones 6.1, 6.2 y 6.3. 112
6.26. Relación de los hallazgos con el marco teórico, la literatura y la metodología. 116

6.27. Factores plausibles de la disociación entre regresión y clasificación. Ninguno se verificó experimentalmente. 117
6.28. Cierre de las brechas del Estado del Arte a la luz de los resultados del Capítulo 6. 119

7.1. Síntesis de limitaciones operativas, logísticas y empíricas del estudio, su impacto en las afirmaciones de la tesis y su directriz metodológica de resolución. 124
7.2. Delimitación rigurosa del alcance científico de las conclusiones dictaminadas en esta investigación. 126

DEDICATORIA

A mi padre, ***Ricardo Valera Hernández****, por ser el pilar constante en cada etapa de mi formación, por su paciencia, su apoyo incondicional y por nunca dejar de creer en mí incluso cuando yo dudaba, por su ejemplo de responsabilidad y esfuerzo, por enseñarme que el trabajo honesto y la perseverancia son el camino para alcanzar cualquier meta.*

A mi madre, ***María Margarita Irma Martínez Pérez****, por ser la guía y el apoyo incondicional que hizo posible este trayecto. Gracias por su confianza infinita, por sus palabras que siempre supieron darme fuerza y por estar presente en cada paso de mi formación profesional. Este trabajo es testimonio de su esfuerzo y dedicación constante en mi vida.*

A mi novia, ***Vanessa F. Linarte Castañeda****, por su amor, su comprensión y su compañía en los momentos de cansancio y dificultad, por motivarme a seguir adelante cuando parecía imposible y por ser parte fundamental de este logro.*

A mi hermano menor, ***Emilio Valera Martínez****, por su complicidad y apoyo en los momentos de distracción y de enfoque. Gracias por estar ahí, por los consejos y por ser un compañero constante en este camino.*

Al ***Sr. Héctor H. Linarte*** *y la* ***Sra. Cristina E. Castañeda****, por recibirme en su familia como a un hijo más, por sus palabras de aliento y por el apoyo incondicional brindado durante este proceso. Su confianza y cariño fueron fundamentales para mantener el ánimo en la recta final.*

A mi director de tesis, el ***Mtro. Mauro Alberto López Muñoz****, y a mi codirector, el* ***Mtro. Guillermo Colorado Jiménez****, por su invaluable guía, su paciencia y por compartir su conocimiento para llevar este proyecto a buen puerto.*

Capítulo 1

Introducción

"El futuro de las redes de computadoras no solo depende de transmitir datos más rápido, sino de hacerlo de manera inteligente."

La evolución en las redes de computadoras ha permitido el desarrollo de una gran cantidad de servicios y aplicaciones que forman parte de la vida cotidiana, desde la navegación web y las plataformas de comunicación, hasta los sistemas distribuidos, la computación en la nube y el intercambio masivo de información en tiempo real. Debido a este crecimiento constante en la cantidad de dispositivos conectados y el volumen de tráfico generado, las redes modernas enfrentan cada vez mayores desafíos relacionados con el rendimiento, la disponibilidad y la calidad del servicio.

1.1. Planteamiento del problema

En la era digital actual, la conectividad se ha convertido en un servicio esencial, tan fundamental como la electricidad o el agua corriente. Desde las videollamadas que facilitan la comunicación remota, hasta las plataformas de *streaming* que ofrecen entretenimiento bajo demanda, así como la rápida expansión del Internet de las Cosas (IoT) en hogares y ciudades, gran parte de la vida cotidiana depende de un flujo constante e ininterrumpido de datos. Según el informe *Cisco Annual Internet Report (2018–2023)*, el número de usuarios de Internet a nivel mundial pasó de 3900 millones en 2018 a 5300 millones en 2023, mientras que los dispositivos conectados a redes IP aumentaron de 18 400 millones a 29 300 millones en el mismo período [1].

Las redes de computadoras son el tejido que sostiene la infraestructura tecnológica moderna. Sin embargo, no son un recurso infinito: cada enrutador, cada conmutador y cada enlace de fibra óptica tiene una capacidad limitada. Cuando la demanda de transmisión de datos supera dicha capacidad, se produce un fenómeno conocido como congestión de red. En términos sencillos (cuya definición formal se abordará en secciones posteriores), la congestión es el equivalente digital a un embotellamiento en una autopista: demasiados paquetes de datos intentando circular al mismo tiempo por un canal con aforo máximo. Las consecuencias son inmediatas y perceptibles para el usuario final: congelamiento de videollamadas, lentitud

en la carga de páginas web, interrupción en la descarga de archivos o, en el peor de los casos, la pérdida total de la conexión. En entornos críticos como hospitales, redes financieras o infraestructuras industriales, estos fallos se traducen en pérdidas económicas millonarias e incluso en riesgos para la seguridad humana.

Tradicionalmente, la gestión de redes ha mantenido un enfoque reactivo: los administradores monitorizan los equipos y, cuando detectan que un enlace está saturado, toman medidas correctivas como redirigir el tráfico, limitar ciertas aplicaciones o aumentar la capacidad del enlace. Este modo de actuar, aunque ha resultado funcional durante décadas, presenta una limitación fundamental: actúa después de que el problema ha comenzado a degradar la calidad del servicio. En escenarios de latencia crítica, como la operación de un coche autónomo que necesita tomar decisiones en milisegundos o un procedimiento quirúrgico a distancia mediante robótica, reaccionar a posteriori no es una opción viable; la anticipación resulta indispensable.

De lo anterior se desprende el problema central que da origen a esta investigación: los mecanismos actuales de gestión de redes de computadoras carecen, en su mayoría, de la capacidad de anticipar la congestión antes de que esta se manifieste, limitándose a detectarla y corregirla una vez iniciada la degradación del servicio. Esta naturaleza reactiva impide a los administradores de red y a los sistemas automatizados de gestión tomar decisiones oportunas —como redirigir tráfico, priorizar aplicaciones críticas o escalar recursos— antes de que el impacto sea perceptible. El problema de investigación se formula, entonces, de la siguiente manera: **¿es posible, a partir del análisis de datos históricos de operación de una red, construir un modelo capaz de predecir con suficiente antelación que un enlace alcanzará un estado de congestión, de tal forma que dicha predicción resulte útil para la toma de decisiones preventivas?**

En este contexto, las técnicas de Machine Learning (ML) se presentan como una vía prometedora. El ML, como rama de la inteligencia artificial, permite a los sistemas informáticos identificar patrones a partir de datos históricos y realizar predicciones sin requerir una programación explícita para cada escenario, lo que lo convierte en una herramienta natural para abordar la necesidad de anticipación descrita.

El presente trabajo de tesis se sitúa en la intersección entre las redes de computadoras y el aprendizaje automático. El objetivo central es desarrollar un sistema basado en técnicas de Machine Learning capaz de predecir la congestión en un enlace de red mediante el análisis de métricas de tráfico obtenidas de forma híbrida (activa y pasiva). En lugar de mantener un enfoque reactivo frente a niveles críticos de utilización, se busca construir un modelo que permita emitir alertas tempranas sobre la saturación inminente de un enlace, facilitando la toma de medidas preventivas antes de que la calidad del servicio se vea comprometida.

1.2. Objetivos

A continuación se detallan los objetivos que guían la presente investigación, los cuales establecen las directrices fundamentales para el desarrollo e implementación del sistema propuesto.

1.2.1. Objetivo General

Desarrollar un sistema predictivo basado en Machine Learning capaz de anticipar la congestión en enlaces de red a partir del análisis de datos operativos, evaluando su desempeño en escenarios de tráfico de distinta escala y naturaleza, con el fin de aportar evidencia sobre la viabilidad de este enfoque para optimizar la gestión de recursos y la calidad del servicio.

1.2.2. Objetivos Específicos

- Recopilar y organizar un conjunto de datos (dataset) operativos de una red de computadoras, abarcando parámetros clave de telemetría como ancho de banda, rendimiento, disponibilidad y métricas vinculadas a la transmisión de datos.
- Diseñar un procedimiento de preprocesamiento para limpiar, normalizar y estructurar los datos obtenidos, garantizando su idoneidad para el entrenamiento algorítmico.
- Implementar, entrenar y comparar distintos modelos de Machine Learning (tales como enfoques de series temporales, árboles de decisión o redes neuronales) orientados al análisis de patrones de tráfico.
- Construir un prototipo funcional que integre el modelo de mejor rendimiento para generar predicciones y alertas tempranas sobre posibles escenarios de congestión.
- Evaluar el desempeño y la precisión del sistema desarrollado mediante métricas de validación estándar propias de los modelos predictivos.
- Determinar y documentar la aplicabilidad del modelo propuesto en entornos reales para respaldar la toma de decisiones preventivas en la administración de redes.

1.3. Justificación

El crecimiento exponencial del tráfico en las redes de computadoras ha superado las estrategias tradicionales de gestión basadas en la monitorización reactiva. En la actualidad, la demanda de ancho de banda generada por servicios de videoconferencia, plataformas de *streaming*, aplicaciones en la nube y dispositivos del Internet de las Cosas (IoT) somete a las infraestructuras a picos de utilización impredecibles. En este escenario, los administradores se ven forzados a intervenir únicamente después de que los niveles de congestión ya han degradado la calidad del servicio, lo que deriva en una experiencia deficiente para los usuarios y, en ámbitos críticos, en pérdidas económicas o riesgos de seguridad. Por consiguiente, resulta indispensable desarrollar herramientas que permitan anticipar la saturación de los enlaces,

migrando de una operación reactiva a una gestión predictiva que tome decisiones antes de que la congestión se materialice.

La aplicación de técnicas de *Machine Learning* en el ámbito de las redes abre una oportunidad real para este cambio de paradigma [2]. A diferencia de los métodos estadísticos convencionales, los algoritmos de aprendizaje automático tienen la capacidad de descubrir patrones complejos y no lineales a partir de grandes volúmenes de datos operativos, como series temporales de utilización, latencia o pérdida de paquetes. Modelos como las redes neuronales recurrentes, los bosques aleatorios o las arquitecturas de aprendizaje profundo han demostrado un desempeño notable en la predicción de tráfico en otros contextos, pero su aplicación sistemática para anticipar la congestión en enlaces individuales constituye un área de investigación con vacíos aún por cubrir, particularmente en lo referente a la comparación de múltiples familias de modelos en condiciones realistas de operación.

En el ámbito práctico, un sistema que prediga con suficiente antelación que un enlace alcanzará su límite de capacidad puede transformar la administración de redes. Por ejemplo, un administrador podría redirigir flujos de tráfico, activar mecanismos de balanceo de carga, reducir la prioridad de aplicaciones no críticas o incluso aprovisionar recursos adicionales de forma automática antes de que el usuario perciba degradación alguna. Esta capacidad de anticipación resulta crucial en entornos donde la continuidad del servicio es prioritaria, tales como centros hospitalarios que dependen de la telemedicina, entidades financieras con transacciones en tiempo real o plantas industriales con procesos automatizados. De esta manera, la presente investigación no solo busca generar conocimiento teórico, sino también ofrecer una herramienta con un impacto directo en la fiabilidad y eficiencia de las infraestructuras digitales.

Desde el punto de vista metodológico, la propuesta de construir un modelo predictivo basado en datos de operación reales y evaluarlo mediante métricas de error rigurosas —como el Error Cuadrático Medio (MSE), el Error Absoluto Medio (MAE) y la comparación de tasas de predicción en distintos horizontes temporales— aporta un procedimiento replicable para la comunidad académica y profesional. La selección de tres paradigmas de modelado distintos (series temporales estadísticas, aprendizaje automático clásico y aprendizaje profundo) permite, además, trazar un mapa claro de las fortalezas y limitaciones de cada enfoque, ofreciendo criterios objetivos para futuras implementaciones. En un campo donde la mayoría de los estudios se concentran en la predicción general del volumen de tráfico, esta tesis se distingue por centrarse en la detección temprana de un evento específico y operacionalmente crítico: la congestión.

Finalmente, la investigación se justifica por su alineación con las tendencias actuales de la industria hacia las redes autónomas y la inteligencia artificial aplicada a las operaciones de TI (AIOps). Diversas empresas del sector tecnológico y organismos de estandarización impulsan actualmente la incorporación de modelos predictivos en los sistemas de gestión de red, reconociendo que la supervisión tradicional resulta insuficiente para soportar las exigentes demandas de latencia y disponibilidad de las aplicaciones modernas. En este sentido, el trabajo propuesto contribuye a cerrar la brecha entre las capacidades teóricas del aprendizaje automático y su aplicación práctica en un problema real de ingeniería de redes, generando evidencia empírica sobre su viabilidad, precisión y pertinencia.

1.4. Alcances y Limitaciones

El presente trabajo de tesis se circunscribe al desarrollo de un prototipo funcional de predicción de congestión, implementado enteramente en código y orientado a validar la viabilidad utilizando un conjunto de datos de operación de red. El alcance del proyecto se limita a la construcción, entrenamiento y evaluación de estos modelos mediante métricas cuantitativas, sin comprometerse con el desarrollo de una interfaz gráfica de usuario, una aplicación empaquetada o un sistema de producción completo. Por lo tanto, los resultados deben interpretarse como una prueba de concepto que demuestra la capacidad predictiva de los algoritmos seleccionados, y no como una solución lista para ser desplegada en un entorno operacional real.

En términos de datos, el prototipo se desarrolla y evalúa sobre dos fuentes complementarias: una red privada de escala reducida sobre la cual se realizó recolección propia de datos de operación durante un periodo de 30 días, y un archivo académico de tráfico de Internet de gran escala empleado como conjunto de validación externa. Si bien esta combinación permite evaluar el desempeño de los modelos bajo condiciones de tráfico de naturaleza y escala distintas —ampliando el alcance de las conclusiones respecto a un estudio basado en una única fuente—, no debe interpretarse como una validación exhaustiva sobre la totalidad de topologías, tecnologías de acceso o patrones de tráfico posibles en una red de computadoras. La generalización de los resultados a escenarios sustancialmente distintos a los aquí evaluados (por ejemplo, redes móviles, entornos industriales o infraestructuras de centros de datos) queda fuera del alcance de este trabajo y se plantea como una línea de investigación futura.

Desde el punto de vista computacional, se ha establecido un límite práctico en la complejidad de los algoritmos y en los recursos de hardware empleados. El prototipo se ejecuta en un entorno de estación de trabajo convencional, sin recurrir a infraestructura de cómputo distribuido o aceleración especializada (GPU de alto rendimiento). En consecuencia, los modelos profundos se configuran con un número acotado de capas y neuronas, y no se explora el ajuste exhaustivo de hiperparámetros mediante búsqueda en malla de gran tamaño. Este enfoque permite obtener resultados en tiempos razonables, pero sacrifica potencialmente la obtención del modelo óptimo absoluto, lo cual es una limitación deliberada acorde con el carácter prototípico del desarrollo.

Otra restricción relevante es que el sistema no opera en tiempo real sobre una red viva, sino que se entrena y evalúa sobre trazas históricas. La predicción se realiza fuera de línea, asumiendo que los datos han sido previamente recolectados y preprocesados. Por tanto, no se han implementado mecanismos de ingestión continua de datos, detección de deriva de concepto (*concept drift*) ni reentrenamiento automático del modelo, aspectos que resultarían necesarios para una eventual transición a un sistema de alerta temprana en producción. Esta limitación es congruente con la elección de priorizar la comparación de modelos bajo condiciones controladas [3].

Finalmente, es imperativo distinguir entre la variable objetivo del sistema y sus variables predictoras. Para fines de etiquetado y evaluación, el estado de congestión se define y cuantifica exclusivamente a partir del factor de utilización del enlace (variable objetivo). Sin embargo, el prototipo predictivo sí incorpora métricas de red complementarias —como estimaciones de rendimiento y disponibilidad obtenidas de forma pasiva— como variables de entrada (*features*) para el entrenamiento de los algoritmos. Esta delimitación conceptual per-

mite definir un umbral matemático claro para la congestión, reconociendo simultáneamente que el modelo requiere de una perspectiva multivariable para capturar los patrones complejos que preceden a la saturación del enlace.

Capítulo 2

Marco Teórico

2.1. Fundamentos de redes de computadoras

Considérese un escenario con dos computadoras aisladas, cada una con información valiosa almacenada en su interior. Para que esos datos puedan ser aprovechados por el otro equipo, un usuario debe copiarlos en un dispositivo de almacenamiento externo, trasladarlos físicamente e introducirlos en la segunda máquina. Este procedimiento, aunque funcional, es lento, propenso a errores humanos y completamente inviable cuando la cantidad de información crece o cuando los dispositivos se encuentran a cientos o miles de kilómetros de distancia. Precisamente para resolver esta limitación nacieron las redes de computadoras: sistemas que conectan dos o más dispositivos de cómputo mediante algún medio de transmisión —cables de cobre, fibra óptica o el espectro electromagnético— con el propósito de intercambiar datos de manera rápida, confiable y eficiente [4].

En esencia, una red de computadoras es un conjunto de nodos autónomos interconectados que cooperan para compartir recursos e información. El término *nodo* designa a cualquier dispositivo capaz de originar, recibir o retransmitir datos: computadoras personales, servidores, teléfonos móviles, sensores, impresoras e incluso electrodomésticos inteligentes. Estos nodos se comunican a través de enlaces, que son los caminos físicos o inalámbricos por los que viaja la información. A su vez, los enlaces se conectan entre sí por medio de dispositivos de interconexión como conmutadores y enrutadores, cuya misión es encaminar los datos desde su origen hasta su destino, eligiendo la ruta más adecuada en cada momento y gestionando las colas de espera que se forman cuando varios flujos de tráfico coinciden en un mismo punto. El comportamiento de estas colas de espera bajo condiciones de alta demanda es, como se detallará en la sección 2.1.3, el origen mismo del fenómeno de congestión.

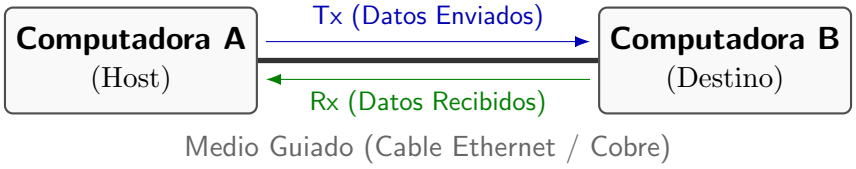


Figura 2.1: Representación de una red entre dos computadoras.

El funcionamiento de una red se organiza de manera jerárquica para gestionar la complejidad inherente a la comunicación digital. Esta organización se plasma en los llamados modelos de referencia, que dividen el problema de transmitir información en varias capas superpuestas, cada una con responsabilidades claramente acotadas: desde los aspectos puramente físicos de la transmisión (voltajes, pulsos de luz, ondas de radio) hasta la interacción con las aplicaciones que el usuario ve en pantalla. Esta arquitectura en capas permite que un fabricante mejore un componente sin necesidad de rediseñar toda la red, y que tecnologías muy dispares puedan interoperar siempre que respeten los protocolos definidos en cada nivel. El modelo de referencia más utilizado para describir esta organización, el modelo OSI, se desarrolla con mayor profundidad en la sección 2.1.2.

Más allá de cómo se organizan las capas, existe una decisión de diseño fundamental que determina cómo viajan realmente los datos a través de una red: el paradigma de conmutación. En las redes telefónicas tradicionales se empleaba la **conmutación de circuitos**: cuando dos personas iniciaban una llamada, la red reservaba un camino físico dedicado y exclusivo entre ambos extremos durante toda la conversación. Este enfoque garantiza una calidad de servicio constante y predecible, pero resulta profundamente ineficiente desde el punto de vista de la utilización de recursos: si los interlocutores hacen una pausa de diez segundos, el circuito reservado permanece completamente ocioso y ningún otro usuario puede aprovecharlo, incluso si la red entera está saturada de solicitudes en ese instante.

Las redes de computadoras modernas, incluida Internet, optaron por un modelo radicalmente distinto: la **conmutación de paquetes**. En este paradigma, los datos se fragmentan en pequeñas unidades llamadas paquetes o datagramas, cada una encabezada por información de control (dirección de origen, destino, número de secuencia, entre otros campos). Cada paquete viaja de nodo en nodo de manera independiente y sin reservar recursos exclusivos, compartiendo dinámicamente los enlaces con paquetes pertenecientes a otras comunicaciones completamente distintas. Cuando un remitente hace una pausa en su transmisión, otros flujos aprovechan inmediatamente el enlace liberado, lo que multiplica la eficiencia global del sistema comparado con la conmutación de circuitos [5].

Esta compartición estadística de los recursos es, precisamente, la razón por la que Internet puede ofrecer conectividad a miles de millones de dispositivos con una infraestructura física finita. Sin embargo, tiene una contrapartida inevitable: como ningún flujo individual tiene garantizado un ancho de banda exclusivo, si demasiados remitentes transmiten de manera simultánea a través del mismo enlace, la demanda agregada puede superar la capacidad disponible. Es precisamente en este punto —cuando la suma de los flujos que compiten por un recurso compartido excede lo que ese recurso puede entregar— donde se origina el fenómeno que esta investigación busca predecir: la congestión de red, que se define y caracteriza formalmente en las siguientes secciones.

2.1.1. Capacidad del enlace: ancho de banda, throughput y utilización

Para poder cuantificar cuándo un enlace se encuentra cerca de su límite, es necesario introducir tres métricas básicas que se emplearán a lo largo del presente trabajo. Cada enlace en una red posee una capacidad máxima teórica de transmisión, denominada **ancho de banda** (*bandwidth*), que se mide en bits por segundo (bps). Un enlace de fibra óptica típico

puede ofrecer 1 Gbps, mientras que una conexión inalámbrica doméstica quizás solo alcance 50 Mbps. Esta cifra representa el límite superior teórico del enlace, no la cantidad de datos que efectivamente circula por él en un momento dado.

El tráfico real que atraviesa el enlace en un instante determinado se conoce como ***throughput*** o rendimiento efectivo, y su valor siempre es inferior o igual al ancho de banda nominal, ya que factores como la sobrecarga de los encabezados de protocolo, las retransmisiones y el propio comportamiento del tráfico impiden alcanzar el máximo teórico de manera sostenida. La razón entre el *throughput* observado y el ancho de banda disponible se denomina **factor de utilización**, y constituye una de las métricas más reveladoras sobre el estado de salud de un enlace:

$$U = \frac{\text{Throughput observado}}{\text{Ancho de banda nominal}} \tag{2.1}$$

Mientras el factor de utilización se mantenga en niveles moderados, la red opera con holgura: existe margen suficiente para absorber ráfagas de tráfico sin que se produzcan retrasos perceptibles. Conforme U se aproxima a 1 (es decir, al 100 % de la capacidad), comienzan a manifestarse los síntomas característicos de la congestión, los cuales se explican en detalle a continuación.

2.1.2. Modelo OSI y TCP/IP

El Modelo de Interconexión de Sistemas Abiertos (OSI, *Open Systems Interconnection*), estandarizado por la Organización Internacional de Normalización (ISO) en 1984, constituye un marco conceptual que divide el proceso de comunicación en redes en siete capas funcionales independientes [4]. Cada nivel abstrae un conjunto específico de responsabilidades, desde la transmisión física de bits hasta la interacción con las aplicaciones del usuario. Esta estratificación facilita la interoperabilidad entre equipos de distintos fabricantes y permite analizar los fenómenos de red, como la congestión, dentro de una estructura bien definida. En particular, la gestión de la congestión incide directamente en las capas de transporte y red, donde se regulan el control de flujo, el enrutamiento y la asignación de recursos; aspectos que serán modelados en este trabajo a partir de métricas de utilización.

Aunque el modelo OSI es el estándar de referencia teórica, en la práctica, las redes modernas operan bajo la arquitectura TCP/IP (*Transmission Control Protocol/Internet Protocol*). Este modelo condensa las siete capas de OSI en cuatro niveles prácticos: Acceso a la Red, Internet, Transporte y Aplicación. Para efectos de esta investigación, la telemetría recolectada en los enlaces —como el *throughput* y el factor de utilización— se sustenta en los protocolos del modelo TCP/IP (tales como IP en la capa de red y TCP/UDP en la capa de transporte), si bien el análisis subyacente del fenómeno de congestión sigue los principios teóricos establecidos por OSI.

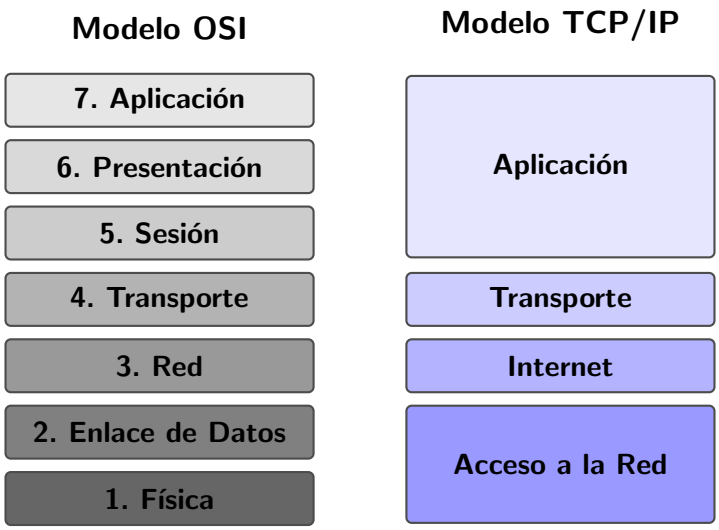


Figura 2.2: Comparativa entre las capas del modelo OSI y la arquitectura TCP/IP.

2.1.3. Concepto de congestión

Con las nociones de ancho de banda, *throughput* y utilización ya establecidas, es posible caracterizar con precisión qué ocurre dentro de una red cuando la demanda de tráfico se aproxima o supera la capacidad disponible de un enlace. A este fenómeno se le denomina **congestión de red**, y constituye el objeto central de estudio de esta investigación.

Qué ocurre dentro de un enrutador

Para entender el origen físico de la congestión es necesario examinar qué sucede dentro de un enrutador o conmutador cuando procesa tráfico. Cada dispositivo de interconexión dispone de pequeñas memorias temporales llamadas ***buffers***, donde se almacenan los paquetes que arriban mientras esperan ser procesados y reenviados por el enlace de salida correspondiente. Si los paquetes llegan a una tasa inferior a la capacidad del enlace de salida, el *buffer* apenas se ocupa y los paquetes son despachados casi de inmediato, con un tiempo de espera mínimo.

El problema surge cuando, durante un intervalo sostenido, la tasa de llegada de paquetes supera la tasa a la que el enlace de salida puede transmitirlos. En este escenario, la cola de espera dentro del *buffer* comienza a crecer progresivamente. Este fenómeno es análogo a lo que ocurre en una caseta de cobro de autopista: mientras los vehículos lleguen a un ritmo menor al que la caseta puede atender, la fila permanece corta y el retraso es mínimo; pero si llegan más vehículos de los que la infraestructura puede procesar, la fila se alarga sin control y el tiempo de espera se incrementa de forma no lineal. Es importante subrayar este último comportamiento: el retraso generado por la congestión **no crece de manera proporcional** al exceso de tráfico, sino que tiende a dispararse abruptamente una vez que se cruza cierto umbral de utilización, un fenómeno exhaustivamente documentado en la teoría de colas aplicada a redes de datos [6].

Latencia y sus componentes

El tiempo total que un paquete tarda en viajar desde que ingresa a la red hasta que llega a su destino final se conoce como **latencia** o retardo extremo a extremo, y se compone de cuatro elementos fundamentales:

- **Tiempo de propagación**: el tiempo que tarda la señal física en recorrer el medio de transmisión (cable, fibra, espectro electromagnético), determinado por la distancia y la velocidad de propagación del medio.
- **Tiempo de transmisión**: el tiempo que tarda el nodo emisor en colocar todos los bits del paquete sobre el enlace físico, el cual depende del tamaño del paquete y del ancho de banda disponible.
- **Tiempo de procesamiento**: el tiempo que cada nodo intermedio emplea en examinar el encabezado del paquete y determinar la ruta de reenvío.
- **Tiempo de encolado**: el tiempo que el paquete permanece en espera dentro del *buffer* de un dispositivo antes de ser transmitido.

En condiciones normales de operación, el tiempo de encolado es prácticamente insignificante frente a los otros tres componentes. Sin embargo, bajo condiciones de congestión, este componente suele convertirse en el factor dominante de la latencia total, siendo el responsable principal del retraso que experimenta el usuario.

Jitter: la variabilidad del retraso

Dado que la llegada de paquetes a un enrutador es irregular e impredecible por naturaleza, el tamaño de la cola de espera fluctúa constantemente a lo largo del tiempo. Como consecuencia, la latencia experimentada por paquetes consecutivos de un mismo flujo no es constante, sino que varía de un paquete a otro. Esta variabilidad en el retardo se denomina ***jitter***, y resulta particularmente perjudicial para aplicaciones sensibles al tiempo que requieren una entrega de datos a ritmo constante (como videollamadas, juegos en línea o transmisiones de audio y video en tiempo real), donde un *jitter* elevado se traduce en cortes, distorsiones o congelamiento de la imagen, incluso cuando la latencia promedio parece aceptable.

Pérdida de paquetes y la respuesta reactiva de TCP

Cuando la congestión se agrava lo suficiente, el *buffer* de un enrutador puede llenarse por completo. Llegado a este límite, el dispositivo no tiene más opción que descartar los paquetes entrantes que no caben en la memoria disponible, evento conocido como **pérdida de paquetes**. Una tasa de pérdida ocasional puede ser mitigada o pasar desapercibida, pero al volverse frecuente, degrada severamente la calidad de la comunicación.

Es en este escenario donde interviene el comportamiento de los protocolos de transporte, particularmente TCP. Este protocolo interpreta la pérdida de paquetes como una señal indirecta de congestión en la red, y en respuesta reduce drásticamente su tasa de envío mediante mecanismos de control de congestión [7], para posteriormente incrementarla de manera gradual mientras no detecte nuevas pérdidas. Este mecanismo, formalizado en estándares como

el RFC 2914 [8], ha sido fundamental para preservar la estabilidad de Internet durante décadas. No obstante, es crucial notar su naturaleza intrínsecamente **reactiva**: el mecanismo se activa únicamente *después* de que la congestión ha provocado pérdidas reales, momento en el cual la calidad del servicio se ha degradado y el problema ya es perceptible. Esta limitación reactiva es la motivación central que impulsa el enfoque predictivo propuesto en esta tesis.

El cuello de botella

En cualquier ruta que conecte dos nodos de una red a través de múltiples enlaces intermedios, existe un concepto estructural determinante: el **cuello de botella** (*bottleneck*). Se trata del enlace con la menor capacidad disponible a lo largo de toda la ruta, el cual establece la velocidad máxima teórica a la que pueden fluir los datos de extremo a extremo. De manera análoga al tramo más estrecho de una tubería que limita el caudal total de agua, el cuello de botella es, por definición, el punto donde la congestión se manifiesta primero y con mayor intensidad. Por consiguiente, estos enlaces son los candidatos naturales para la aplicación de técnicas de predicción y prevención.

Definición operativa de congestión

A partir de los elementos anteriores (utilización, *buffers*, latencia, *jitter*, pérdida de paquetes y cuellos de botella), es posible formular una definición precisa y *medible* de congestión, la cual se adoptará de manera consistente a lo largo del desarrollo metodológico y experimental del presente trabajo.

Se establece que un enlace se encuentra en estado de **congestión** cuando su factor de utilización U (Ecuación 2.1) supera un umbral crítico U_{th} durante un intervalo de tiempo sostenido, punto a partir del cual el comportamiento del sistema deja de ser lineal y comienzan a manifestarse los síntomas de degradación: crecimiento acelerado del tiempo de encolado, incremento del *jitter* y pérdida de paquetes. Formalmente:

$$\text{Congestión} \iff U(t) \geq U_{th} \quad \text{durante un intervalo } [t, t + \Delta t] \tag{2.2}$$

La determinación del umbral crítico U_{th} y la ventana de observación Δt no se establecerá de manera arbitraria. Para garantizar el rigor empírico, el valor de U_{th} se derivará en el Capítulo 4 a partir de un análisis estadístico sobre el conjunto de datos operativos, identificando el punto de inflexión en la distribución (por ejemplo, mediante el percentil 85 o 90) donde la métrica de utilización comienza a correlacionarse con un incremento exponencial en las fluctuaciones de la red. De igual forma, el intervalo Δt estará dictado por la granularidad de muestreo soportada por las herramientas de instrumentación pasiva empleadas durante la recolección de las trazas de 30 días.

Esta definición operativa presenta dos ventajas fundamentales. En primer lugar, se apoya en una métrica directamente observable y de fácil recolección pasiva (el factor de utilización), a diferencia del *jitter* o la pérdida de paquetes, que requieren instrumentación invasiva. En segundo lugar, al establecer un umbral cuantitativo, permite convertir la detección de congestión en un problema formal de aprendizaje automático. De este modo, el problema puede abordarse como una tarea de **regresión** (predecir el valor futuro de $U(t)$) o como una tarea de **clasificación binaria** (predecir si $U(t)$ superará o no el umbral U_{th} en un horizonte temporal

determinado); ambos enfoques se explorarán y compararán en los capítulos metodológico y experimental.

Es importante reconocer que esta definición constituye una simplificación deliberada de un fenómeno inherentemente multifactorial. Como se estableció en la sección de alcances y limitaciones, variables como el *jitter* o la tasa de pérdida no son modeladas directamente por el prototipo, aunque su comprensión teórica, expuesta en esta sección, resulta indispensable para justificar por qué el factor de utilización constituye un indicador robusto y suficientemente informativo para anticipar la saturación de un enlace.

2.1.4. Métricas de rendimiento

En la sección anterior se explicó qué son la utilización, la latencia, el *jitter* y la pérdida de paquetes, y por qué se manifiestan como consecuencia de la congestión. Corresponde ahora precisar cómo se cuantifica formalmente cada una de estas métricas, qué unidades emplean y mediante qué mecanismos se obtienen en la práctica, ya que estas definiciones constituirán el conjunto de variables de entrada (*features*) que alimentarán los algoritmos de Machine Learning propuestos en esta tesis.

Utilización del enlace

Como se estableció en la Ecuación 2.1, la utilización U es la razón entre el *throughput* observado y el ancho de banda nominal del enlace. En términos de instrumentación, esta métrica se obtiene típicamente mediante el protocolo SNMP (*Simple Network Management Protocol*), consultando periódicamente los contadores de octetos entrantes y salientes de una interfaz de red (`ifInOctets` e `ifOutOctets` de la MIB-II) [4]. Dado que estos contadores son acumulativos, el *throughput* en un intervalo $[t_1, t_2]$ se calcula como la diferencia entre lecturas consecutivas dividida entre el tiempo transcurrido:

$$\text{Throughput}(t_1, t_2) = \frac{8 \times (\text{Octetos}(t_2) - \text{Octetos}(t_1))}{t_2 - t_1} \tag{2.3}$$

donde el factor 8 convierte de bytes a bits, ya que el ancho de banda se expresa convencionalmente en bits por segundo (bps). La granularidad del muestreo (por ejemplo, cada 30 segundos, 1 minuto o 5 minutos) determina la resolución temporal con la que posteriormente se podrá detectar y predecir la congestión, y constituye una decisión metodológica que se abordará formalmente en el Capítulo 4.

Latencia (RTT)

La latencia de ida y vuelta, o *Round-Trip Time* (RTT), es el tiempo que transcurre entre el envío de un paquete y la recepción de su respuesta. Se mide de forma activa mediante herramientas de diagnóstico (que emplean paquetes ICMP *Echo Request/Reply*) o mediante el análisis de las marcas de tiempo (*timestamps*) del protocolo TCP durante el establecimiento de una conexión (*three-way handshake*) [5]. Formalmente:

$$\text{RTT} = t_{\text{recepción}} - t_{\text{envío}} \tag{2.4}$$

En la práctica no se trabaja con una sola medición de RTT, sino con una serie de mediciones sucesivas, ya que es precisamente la variabilidad entre estas mediciones consecutivas la que da origen al *jitter*.

Jitter

A diferencia de la explicación conceptual ofrecida en la sección anterior, aquí se presenta la definición formal adoptada por el estándar RFC 3550, que especifica el protocolo RTP (*Real-time Transport Protocol*) utilizado ampliamente en aplicaciones de voz y video sobre IP [9]. Dicho estándar define el *jitter* como la variación estadística en el tiempo de llegada entre paquetes consecutivos, y propone un estimador de primer orden que se actualiza de forma incremental con cada nuevo paquete recibido:

$$J(i) = J(i-1) + \frac{|D(i-1,i)| - J(i-1)}{16} \tag{2.5}$$

donde $D(i-1,i)$ representa la diferencia en los tiempos de tránsito relativos entre dos paquetes consecutivos $i-1$ e i, calculada como:

$$D(i-1,i) = (R_i - R_{i-1}) - (S_i - S_{i-1}) \tag{2.6}$$

siendo R_i el instante de llegada del paquete i en el receptor y S_i la marca de tiempo con la que dicho paquete fue enviado por el emisor. El factor $1/16$ actúa como una ganancia de suavizado que proporciona un estimador con un buen equilibrio entre estabilidad estadística y capacidad de reacción ante cambios súbitos, evitando que picos aislados distorsionen la medición general [9].

Pérdida de paquetes

La tasa de pérdida de paquetes (*packet loss rate*) se expresa como el porcentaje de paquetes enviados que nunca llegan a su destino, respecto del total de paquetes transmitidos en un intervalo de observación:

$$P_{\text{loss}} = \frac{\text{Paquetes enviados} - \text{Paquetes recibidos}}{\text{Paquetes enviados}} \times 100\,\% \tag{2.7}$$

Esta métrica puede obtenerse de dos maneras. La primera es de forma **activa**, inyectando tráfico de prueba (por ejemplo, mediante `iperf` o ráfagas ICMP) y contabilizando las respuestas. La segunda es de forma **pasiva**, consultando los contadores de descarte que los propios dispositivos exponen a través de SNMP (como `ifInDiscards` o `ifOutDiscards`), sin generar tráfico adicional que pudiera contribuir a la propia congestión [4]. Dado el planteamiento de recolección híbrida de esta tesis, la aproximación pasiva resulta prioritaria para la pérdida de paquetes, mientras que los métodos activos se reservan para mediciones controladas de latencia.

Throughput vs. Goodput

Conviene distinguir con precisión entre dos términos que suelen confundirse. El ***throughput***, definido en la Ecuación 2.3, contabiliza la totalidad de los bits que atraviesan el enlace

en un intervalo de tiempo, incluyendo los encabezados de los distintos protocolos (Ethernet, IP, TCP/UDP) y cualquier paquete retransmitido debido a pérdidas previas. El ***goodput***, en cambio, contabiliza únicamente los bits de datos útiles a nivel de aplicación que fueron entregados exitosamente, excluyendo dicha sobrecarga (*overhead*) de protocolo:

$$\text{Goodput} = \frac{\text{Bits útiles entregados a la capa de aplicación}}{\text{Tiempo total de transmisión}} \tag{2.8}$$

Bajo condiciones de congestión severa, es posible observar un *throughput* aparentemente estable mientras el *goodput* cae drásticamente, ya que una proporción creciente del tráfico corresponde a retransmisiones de paquetes perdidos. Esta distinción es relevante porque un modelo que observe únicamente el *throughput* podría no detectar que la red ya opera de manera ineficiente.

Disponibilidad

Finalmente, la **disponibilidad** (*availability*) de un enlace se define como la proporción de tiempo durante la cual dicho recurso se encuentra operativo y accesible, respecto del tiempo total de observación:

$$\text{Disponibilidad} = \frac{\text{Tiempo operativo}}{\text{Tiempo operativo} + \text{Tiempo de inactividad}} \times 100\,\% \tag{2.9}$$

Si bien la disponibilidad se asocia tradicionalmente con fallos de hardware o interrupciones del servicio, una congestión suficientemente severa y sostenida puede degradar la calidad de la red hasta un punto en que resulta indistinguible de una interrupción total para el usuario final. Por esta razón, y dado que fue contemplada en los objetivos específicos, se incorpora como una métrica complementaria de evaluación.

Síntesis de métricas

La Tabla 2.1 sintetiza las métricas descritas, sus unidades y su método de obtención, sentando las bases analíticas del conjunto de datos que se procesará en el Capítulo 4.

Métrica	**Unidad**	**Ecuación**	**Método de obtención**
Utilización	Adimensional (%)	2.1	SNMP (pasivo)
Throughput	bps	2.3	SNMP (pasivo)
Latencia (RTT)	ms	2.4	ICMP / TCP timestamps (activo)
Jitter	ms	2.5	RTCP / estimación RFC 3550
Pérdida de paquetes	%	2.7	SNMP (pasivo) / *probes* (activo)
Goodput	bps	2.8	Derivado de *throughput*
Disponibilidad	%	2.9	Monitoreo continuo (híbrido)

Cuadro 2.1: Resumen de métricas de rendimiento de red utilizadas en la investigación.

2.2. Técnicas de *Machine Learning* aplicables

Una vez establecidos los fundamentos de redes de computadoras y caracterizado formalmente el fenómeno de la congestión, corresponde presentar el conjunto de herramientas metodológicas que esta tesis emplea para abordar el problema de predicción planteado: las técnicas de *Machine Learning* (ML). Esta sección no pretende ser un tratado exhaustivo de aprendizaje automático, sino una revisión dirigida y suficiente de los conceptos, paradigmas y modelos específicos pertinentes para la tarea de predecir series temporales de utilización de red, los cuales serán implementados y evaluados en los capítulos posteriores.

2.2.1. Aprendizaje supervisado: regresión vs. clasificación

El aprendizaje automático se organiza tradicionalmente en tres grandes paradigmas: el aprendizaje supervisado (donde el modelo aprende a partir de ejemplos previamente etiquetados), el aprendizaje no supervisado (donde se buscan patrones ocultos en datos sin etiquetar) y el aprendizaje por refuerzo (donde un agente aprende mediante la interacción con un entorno). El problema de predicción de congestión planteado en esta tesis se enmarca de manera natural dentro del **aprendizaje supervisado**, ya que se dispone de datos históricos de operación de la red (las variables de entrada o *features*) junto con el valor real observado que se desea predecir (la utilización futura del enlace).

Dentro del aprendizaje supervisado, y de acuerdo con la naturaleza de la variable a predecir, es posible distinguir dos tareas fundamentales:

- **Regresión**: la variable objetivo es continua. En el contexto de esta investigación, corresponde a predecir directamente el valor futuro del factor de utilización $U(t+h)$ para un horizonte de predicción h, expresado como un número real.
- **Clasificación**: la variable objetivo es discreta o categórica. En este caso, corresponde a predecir si el enlace se encontrará o no en estado de congestión en el horizonte h, de acuerdo con la definición operativa formulada en la Ecuación 2.2 (es decir, si $U(t+h) \geq U_{th}$), reduciendo el problema a una clasificación binaria (congestión / no congestión).

Ambos enfoques son complementarios: un modelo de regresión ofrece información más rica (el valor exacto esperado de utilización), mientras que uno de clasificación proporciona una respuesta directamente accionable para un sistema de alertas tempranas. Por esta razón, se explorará la formulación del problema bajo ambos enfoques según el modelo empleado.

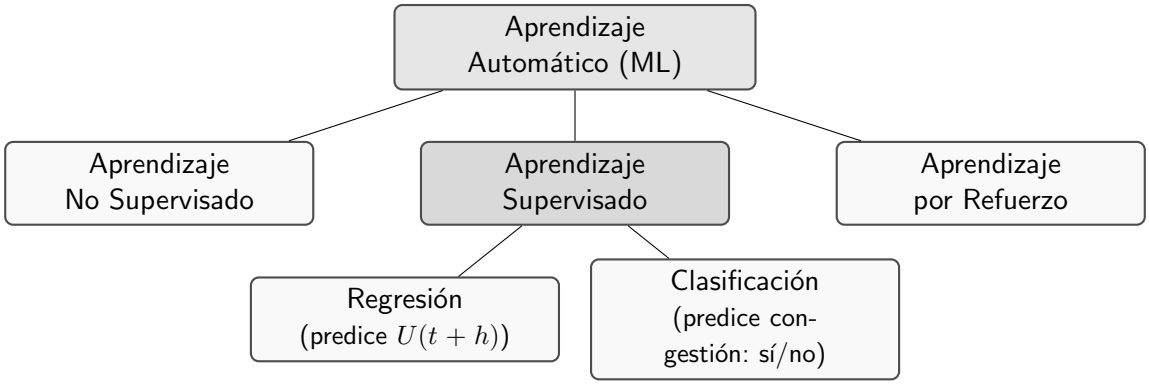


Figura 2.3: Ubicación del problema de predicción de congestión dentro de los paradigmas de *Machine Learning*.

2.2.2. Modelos de series temporales (ARIMA)

El modelo **ARIMA** (*AutoRegressive Integrated Moving Average*), desarrollado por Box y Jenkins, constituye uno de los enfoques estadísticos más consolidados para el análisis y predicción de series temporales [10]. Su nombre resume sus tres componentes:

- **AR (AutoRegresivo)**: el valor de la serie se modela como una combinación lineal de sus propios valores pasados, con un orden p que indica cuántos rezagos (*lags*) se consideran.
- **I (Integrado)**: aplica un proceso de diferenciación de orden d a la serie original para eliminar tendencias y volverla estacionaria.
- **MA (Media Móvil)**: el valor de la serie se modela en función de los errores de predicción pasados, con un orden q.

Un modelo se denota comúnmente como ARIMA(p, d, q), y se expresa formalmente como:

$$\left(1 - \sum_{i=1}^{p} \phi_i L^i\right) (1 - L)^d X_t = \left(1 + \sum_{i=1}^{q} \theta_i L^i\right) \varepsilon_t \tag{2.10}$$

donde X_t es la serie temporal de interés (la utilización en el instante t), L es el operador de rezago, ϕ_i y θ_i son los coeficientes, y ε_t representa el ruido blanco. Este modelo resulta fundamental como **línea base** (*baseline*) metodológica: al ser un método clásico y computacionalmente económico, establece un punto de referencia mínimo de desempeño contra el cual comparar los algoritmos más sofisticados.

2.2.3. Modelos de aprendizaje automático clásico (Random Forest, SVR)

A diferencia de ARIMA, que asume relaciones lineales explícitas entre los valores pasados y futuros, los modelos de aprendizaje automático clásico son capaces de capturar relaciones complejas y no lineales entre múltiples variables de entrada simultáneamente, lo cual

resulta valioso cuando se dispone de varias métricas de red (utilización, latencia, pérdida de paquetes).

Random Forest es un método de aprendizaje por conjunto (*ensemble*) que construye una multitud de árboles de decisión durante el entrenamiento, cada uno sobre una muestra aleatoria de los datos y considerando un subconjunto aleatorio de variables [11]. Para tareas de regresión, la predicción final se obtiene promediando las predicciones individuales:

$$\hat{y} = \frac{1}{T} \sum_{i=1}^{T} h_i(x) \tag{2.11}$$

donde T es el número total de árboles y $h_i(x)$ es la predicción del árbol i-ésimo. Esta estrategia reduce significativamente el riesgo de sobreajuste (*overfitting*) y ofrece una ventaja analítica clave: permite calcular la **importancia relativa** de las variables (*feature importance*), identificando empíricamente qué métricas predicen mejor la congestión.

Por su parte, **SVR** (*Support Vector Regression*) busca encontrar una función que se ajuste a los datos permitiendo un margen de tolerancia, penalizando únicamente las predicciones cuyo error exceda dicho margen. Su capacidad para emplear funciones de *kernel* le permite modelar relaciones no lineales complejas, constituyendo una alternativa robusta adicional.

Es imperativo señalar que estos algoritmos tabulares carecen de una representación interna del tiempo; tratan cada observación como un evento estático. Para aplicarlos a series temporales, es necesario transformar los datos mediante la técnica de **ventanas deslizantes** (*sliding windows*). Bajo este enfoque, la entrada al modelo se transforma en un vector que concatena un histórico de tamaño k: $[X_t, X_{t-1}, \ldots, X_{t-k}]$. Esto convierte la dependencia temporal en una dependencia espacial tabular que los algoritmos pueden procesar.

2.2.4. Modelos de Aprendizaje Profundo (LSTM, GRU)

Las redes neuronales recurrentes (RNN) fueron diseñadas específicamente para procesar datos secuenciales, manteniendo una suerte de memoria interna que se actualiza en cada paso temporal. Sin embargo, las RNN convencionales sufren de un problema conocido como el desvanecimiento del gradiente (*vanishing gradient*), que les impide aprender de manera efectiva dependencias que abarcan largos periodos de tiempo; una limitación especialmente relevante para el tráfico de red, donde los patrones de congestión pueden depender de comportamientos observados varios minutos u horas atrás.

Las **LSTM** (*Long Short-Term Memory*) fueron propuestas precisamente para resolver esta limitación, mediante la introducción de una estructura interna de compuertas (*gates*) que regulan de manera explícita qué información se conserva, se actualiza o se descarta a lo largo del tiempo [12]. Cada unidad LSTM mantiene un estado de celda c_t que actúa como memoria de largo plazo, y tres compuertas (de olvido, de entrada y de salida) que controlan el flujo de información:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \quad \text{(compuerta de olvido)} \tag{2.12}$$

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad \text{(compuerta de entrada)} \tag{2.13}$$

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \quad \text{(compuerta de salida)} \tag{2.14}$$

donde σ es la función de activación sigmoide, x_t es la entrada en el instante t, h_{t-1} es el estado oculto anterior, y W y b son los pesos y sesgos aprendidos. Esta arquitectura permite a la red aprender selectivamente qué eventos pasados (por ejemplo, una ráfaga de sincronización de datos ocurrida varios minutos atrás) siguen siendo relevantes en el presente.

Las **GRU** (*Gated Recurrent Units*) constituyen una variante simplificada de las LSTM, diseñada para reducir la complejidad computacional sin sacrificar capacidad predictiva [13]. Las GRU combinan las compuertas de olvido y entrada, prescindiendo del estado de celda independiente. Esto resulta en un modelo con menos parámetros entrenables y más eficiente de entrenar sobre conjuntos de datos de tamaño moderado.

2.2.5. Métricas de evaluación

Para comparar el desempeño de los modelos (ARIMA, Random Forest, SVR, LSTM y GRU) es necesario definir métricas cuantitativas de error. Sea y_i el valor real observado, $\hat{y}_i$ el valor predicho por el modelo, y n el número total de observaciones.

RMSE (*Root Mean Squared Error*, raíz del error cuadrático medio):

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2} \tag{2.15}$$

Al elevar al cuadrado cada error antes de promediar, el RMSE penaliza de manera desproporcionada los errores grandes, lo cual resulta conveniente para este trabajo: un modelo que falle drásticamente al predecir un evento de congestión severa es más problemático que uno con errores pequeños y uniformes.

MAE (*Mean Absolute Error*, error absoluto medio):

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i| \tag{2.16}$$

A diferencia del RMSE, el MAE pondera todos los errores de manera lineal e igualitaria, resultando menos sensible a valores atípicos (*outliers*) y ofreciendo una interpretación directa en las mismas unidades que la variable original.

MAPE (*Mean Absolute Percentage Error*, error porcentual absoluto medio):

$$\text{MAPE} = \frac{100\,\%}{n}\sum_{i=1}^{n}\left|\frac{y_i - \hat{y}_i}{y_i}\right| \tag{2.17}$$

El MAPE expresa el error como un porcentaje relativo, facilitando su interpretación independientemente de la escala y permitiendo comparar el desempeño entre distintos conjuntos de datos.

Para la tarea de **clasificación binaria** (congestión vs. operación normal), dado que la congestión es un evento anómalo y el *dataset* está desbalanceado, se utilizarán las siguientes métricas basadas en la matriz de confusión (Verdaderos Positivos VP, Verdaderos Negativos VN, Falsos Positivos FP y Falsos Negativos FN):

$$\text{Precisión} = \frac{VP}{VP + FP} \tag{2.18}$$

$$\text{Recall} = \frac{VP}{VP + FN} \tag{2.19}$$

$$\text{F1-Score} = 2 \times \frac{\text{Precisión} \times \text{Recall}}{\text{Precisión} + \text{Recall}} \tag{2.20}$$

La Tabla 2.2 sintetiza los cinco modelos presentados, sirviendo como punto de partida para la justificación metodológica que se desarrollará en el Capítulo 4.

Modelo	**Paradigma**	**Relaciones**	**Complejidad**
ARIMA	Estadístico clásico	Lineales	Baja
Random Forest	ML clásico (*ensemble*)	No lineales	Media
SVR	ML clásico (*kernel*)	No lineales	Media
LSTM	*Deep Learning* (RNN)	No lineales, largo plazo	Alta
GRU	*Deep Learning* (RNN)	No lineales, largo plazo	Media-Alta

Cuadro 2.2: Resumen comparativo de los modelos de predicción considerados en esta tesis.

Capítulo 3

Estado del Arte

Este capítulo sitúa la tesis dentro de la literatura sobre predicción de tráfico y congestión en redes de computadoras, con el fin de justificar con evidencia empírica las decisiones metodológicas del Capítulo 4.

La revisión sigue un orden cronológico y metodológico: métodos estadísticos clásicos (Sección 3.1), *Machine Learning* clásico (Sección 3.2), *Deep Learning* (Sección 3.3), herramientas de monitoreo y conjuntos de datos empleados para validar estos enfoques (Sección 3.4) y, finalmente, la brecha que esta tesis busca cubrir (Sección 3.5).

De cada trabajo se analizan cuatro aspectos: 1) el enfoque metodológico, 2) el origen de los datos, 3) las métricas y los resultados reportados, y 4) las limitaciones, ya sean reconocidas por los autores o identificables mediante una lectura crítica. Estas últimas no buscan desacreditar a otros investigadores, sino señalar los problemas que la literatura aún no resuelve y que motivan esta tesis.

Cabe advertir que la mayoría de los trabajos no predice la congestión como un evento discreto y accionable (Sección 2.1.3, Ecuación 2.2), sino el volumen de tráfico como serie temporal continua. Esta distinción es central para la contribución de este trabajo y se retoma en la Sección 3.5.

3.1. Revisión de trabajos sobre predicción de tráfico con métodos estadísticos

Los métodos estadísticos clásicos, en particular la familia ARIMA (Sección 2.2.2), dominaron durante años la predicción de tráfico por su fundamento sólido y su simplicidad. Moayedi y Masnadi-Shirazi [14] los emplearon para detectar anomalías a partir de las desviaciones entre el tráfico predicho y el observado, lo que expone su limitación estructural: dependen de que el tráfico se parezca a su historia reciente, algo que suele romperse en las congestiones severas (Sección 2.1.3). Tian [15] lo mitigó con un modelo híbrido que combina ARIMA con un filtro de Volterra para la componente no lineal, y Kochetkova et al. [16] mostraron que SARIMA y Holt-Winters siguen vigentes como líneas base económicas al explotar la estacionalidad, también esperable en el *dataset* primario (Sección 4.4). En conjunto, estos modelos rinden bien con tráfico regular, pero fallan ante comportamientos no lineales o súbitos, lo que motiva la revisión de *Machine Learning* clásico (Sección 3.2).

3.2. Predicción de tráfico y congestión con *Machine Learning* clásico

Para superar las limitaciones lineales, el *Machine Learning* clásico (Sección 2.2.3) modela relaciones no lineales entre múltiples variables de red. Sneha et al. [17] predijeron la congestión a nivel de enrutador, de forma compatible con la Ecuación 2.2, y encontraron que *Random Forest* y KNN obtenían el mejor desempeño, lo que respalda incluir *Random Forest* en esta tesis. Qian et al. [18] mostraron, en redes en chip, que SVR modela bien la saturación en sistemas de colas, fundamento de la Sección 2.1.3. Labonne et al. [19], el trabajo más cercano al planteamiento del problema (Sección 1.1), predijeron la utilización de enlaces y aportaron una estrategia de generación de tráfico saturado aplicable al *dataset* primario si este presenta pocos episodios de congestión. Sin embargo, ninguno explota la naturaleza secuencial del tráfico, lo que motiva los modelos recurrentes de la Sección 3.3.

3.3. Predicción con *Deep Learning*

Las arquitecturas de *Deep Learning* (Sección 2.2.4), en particular LSTM y GRU, capturan dependencias temporales de largo alcance con menos ingeniería manual de características. Oliveira et al. [20] observaron que arquitecturas simples (MLP y RNN) superaron a un autoencoder más profundo, por lo que mayor complejidad no garantiza mejor desempeño con pocos datos. Saha et al. [21] atienden justamente ese escenario, común en redes pequeñas como la del *dataset* primario (Sección 4.4), mediante aprendizaje por transferencia y aumento de datos, opciones a explorar como trabajo futuro. Santos et al. [22] compararon directamente LSTM y GRU y hallaron que el mejor modelo varía según el tráfico analizado, por lo que esta tesis evalúa ambos sobre los dos conjuntos de datos sin suponer de antemano cuál es superior. Esto respalda el diseño comparativo de ARIMA, *Random Forest* y LSTM/GRU, y junto con las secciones anteriores se sintetiza en la Sección 3.5.

3.4. Herramientas de monitoreo y *datasets* utilizados

Además de los modelos predictivos revisados en las secciones anteriores, resulta igualmente relevante examinar cómo la comunidad académica ha obtenido los datos que alimentan dichos modelos, ya que las decisiones de instrumentación —qué herramienta emplear, con qué granularidad y sobre qué tipo de red— condicionan directamente la calidad y el alcance de cualquier sistema de predicción de congestión. Esta sección organiza dicha revisión en dos partes: primero, las herramientas y protocolos de monitoreo empleados típicamente en la literatura; y segundo, una síntesis de los conjuntos de datos concretos utilizados por los trabajos revisados en las Secciones 3.1 a 3.3, incluyendo el propio *MAWI Working Group Traffic Archive* adoptado como *dataset* de validación en esta tesis (Sección 4.4).

Taxonomía de las técnicas de monitoreo

Como se adelantó en la Sección 2.1.4, las técnicas de monitoreo de red se dividen fundamentalmente en dos familias: el monitoreo **pasivo**, que observa el tráfico existente sin generar tráfico adicional, y el monitoreo **activo**, que inyecta tráfico de prueba deliberadamente para medir el comportamiento de la red. La Figura 3.1 sintetiza esta taxonomía junto con las herramientas y protocolos más representativos de cada familia, varios de los cuales fueron adoptados directamente en la metodología de recolección de datos de esta tesis (Sección 4.4).

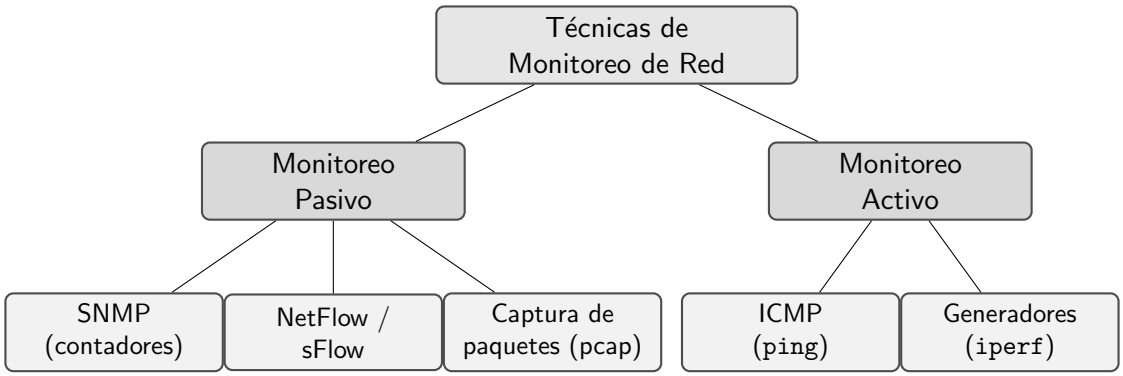


Figura 3.1: Taxonomía de las técnicas de monitoreo de red empleadas en la literatura revisada.

La Tabla 3.1 amplía esta taxonomía, contrastando cada herramienta en términos de la sobrecarga (*overhead*) que introduce en la red, la granularidad temporal que permite obtener, y el tipo de información que provee.

Herramienta	**Tipo**	***Overhead***	**Información obtenida**
SNMP	Pasivo	Muy bajo	Contadores agregados de utilización, errores y descartes (Sección 2.1.4).
NetFlow / sFlow	Pasivo	Medio*	Metadatos por flujo: IPs, puertos, bytes, paquetes. *El *overhead* real depende críticamente de la tasa de muestreo (*sampling rate*) configurada.
Captura (pcap)	Pasivo	Medio-Alto	Registro completo de paquetes; requiere procesamiento posterior (ej. `tshark`).
ICMP (`ping`)	Activo	Bajo	Latencia (RTT) y pérdida de paquetes hacia un destino específico.
Generadores (`iperf`)	Activo	Alto	*Throughput* máximo alcanzable; puede inducir congestión artificialmente.

Cuadro 3.1: Comparación de herramientas de monitoreo empleadas en la literatura revisada.

Como se estableció en la Sección 4.4, esta tesis adopta un enfoque predominantemente pasivo (SNMP para utilización, complementado con mediciones activas ligeras mediante ICMP para latencia y pérdida de paquetes), evitando deliberadamente el uso de generadores de tráfico como `iperf` sobre la red privada bajo estudio, dado que estos podrían introducir una carga adicional que alterara artificialmente las condiciones reales de operación que se busca modelar.

Síntesis de datasets empleados en la literatura

La Tabla 3.2 sintetiza los conjuntos de datos empleados por los trabajos revisados en las Secciones 3.1 a 3.3 de este capítulo, permitiendo identificar patrones respecto al tipo de red, la duración de recolección y la naturaleza —real o simulada— de cada dataset.

Trabajo	**Dataset**	**Naturaleza**	**Escala**	**Duración**
Moayedi & Masnadi-Shirazi [14]	Tráfico propio	Real	No especificada	No especificada
Kochetkova et al. [16]	Red móvil 5G	Real	Operador móvil	Corto plazo
Sneha et al. [17]	Simulación	Simulado	Enrutador individual	No aplica
Labonne et al. [19]	Red simulada (VirtualBox)	Simulado	Red de laboratorio	No especificada
Santos et al. [22]	Telecom Italia (Milán)	Real	Metropolitana	2 meses
Saha et al. [21]	Juniper Networks + ISPs pequeños	Real	ISP (variable)	Variable
Esta tesis (primario)	**Red del Cliente**	**Real**	**Red pequeña/PYME**	**(numero final) días**
Esta tesis (validación)	**MAWI Archive** [23]	**Real**	**Tránsito 1-10 Gbps**	**Diaria (continua)**

Cuadro 3.2: Comparación de datasets empleados en la literatura revisada frente a los datasets adoptados en esta tesis.

Dos observaciones se desprenden de esta comparación. En primer lugar, la duración adoptada para el dataset primario de esta tesis es del mismo orden de magnitud que la empleada por Santos et al. [22], uno de los trabajos más metodológicamente cercanos revisados en este capítulo, lo cual respalda la suficiencia de dicho periodo para capturar patrones cíclicos diarios y semanales de tráfico. En segundo lugar, se observa que una proporción considerable de los trabajos revisados —particularmente aquellos enfocados en ML clásico— recurren a datos simulados en lugar de tráfico real, una decisión metodológica que facilita el control experimental pero que limita la validez de los hallazgos frente a condiciones de operación reales. Esta observación refuerza el valor diferencial del enfoque híbrido adoptado en esta tesis, que combina datos reales de dos naturalezas y escalas distintas en lugar de recurrir a tráfico simulado o a una única fuente.

3.5. Síntesis del estado del arte y brecha a cubrir

Las secciones anteriores de este capítulo revisaron nueve trabajos distintos, organizados cronológica y metodológicamente en tres grandes familias: métodos estadísticos clásicos (Sección 3.1), *Machine Learning* clásico (Sección 3.2) y *Deep Learning* (Sección 3.3), complementados con una revisión de las herramientas de instrumentación y los *datasets* empleados por dichos trabajos (Sección 3.4). El propósito de esta sección final es triple: primero, consolidar en un solo lugar los hallazgos dispersos a lo largo del capítulo; segundo, analizar de manera sistemática el *qué*, *por qué*, *cómo*, *para qué* y *cuándo* de la literatura revisada, con el fin de identificar patrones que no resultan evidentes al leer cada trabajo de manera aislada; y tercero —y más importante—, delimitar con precisión la brecha específica que esta tesis busca cubrir, así como el aporte concreto que este trabajo ofrece de vuelta a la comunidad académica.

Tabla maestra de trabajos revisados

La Tabla 3.3 reúne los nueve trabajos citados en este capítulo, contrastando su enfoque metodológico, la naturaleza de los datos empleados, su aporte principal y —el elemento más relevante para los fines de esta síntesis— la limitación específica que, desde la perspectiva de esta tesis, dicho trabajo deja abierta.

Trabajo	**Año**	**Enfoque**	**Aporte principal**	**Limitación relevante**
Moayedi & Masnadi-Shirazi [14]	2008	ARIMA	Detección de anomalías vía residuos de predicción	No captura no linealidades; congestión inferida indirectamente.
Tian [15]	2021	ARIMA + Volterra	Mejora de ARIMA con componente no lineal	Sigue centrado en volumen de tráfico, no en congestión como evento.
Kochetkova et al. [16]	2023	SARIMA / Holt-Winters	Explota estacionalidad en redes 5G	Sin comparación contra ML/DL en el mismo estudio.
Sneha et al. [17]	2020	*Random Forest*, KNN, GB, DT	Predicción de congestión a nivel de enrutador	Datos simulados; sin validación en tráfico real.
Qian et al. [18]	2016	SVR	Modelado de latencia basado en colas	Dominio distinto (NoC, no redes IP tradicionales).
Labonne et al. [19]	2021	ARIMA, MLP, LSTM	*Framework* completo de generación y predicción de utilización	Red simulada; sin validación externa.
Oliveira et al. [20]	2016	MLP, RNN, SAE	Comparación de arquitecturas de red neuronal	Mayor complejidad (SAE) no garantiza mejor desempeño.
Saha et al. [21]	2024	LSTM + *Transfer Learning*	Aborda *datasets* pequeños en ISPs	No compara contra modelos estadísticos ni ML clásico.
Santos et al. [22]	2022	LSTM vs. GRU	Comparación directa LSTM/GRU en tráfico real (2 meses)	Un solo paradigma (DL); sin ARIMA ni ML clásico.

Cuadro 3.3: Síntesis comparativa de los trabajos revisados en el Estado del Arte.

Evolución cronológica y metodológica

La Figura 3.2 presenta una línea de tiempo que sitúa los trabajos revisados —junto con los hitos teóricos fundamentales presentados en el Marco Teórico de esta tesis (Capítulo 2)— dentro de la evolución histórica del campo, permitiendo visualizar de manera inmediata la transición gradual de los métodos estadísticos hacia el *Machine Learning* clásico y, finalmente, hacia el *Deep Learning*.

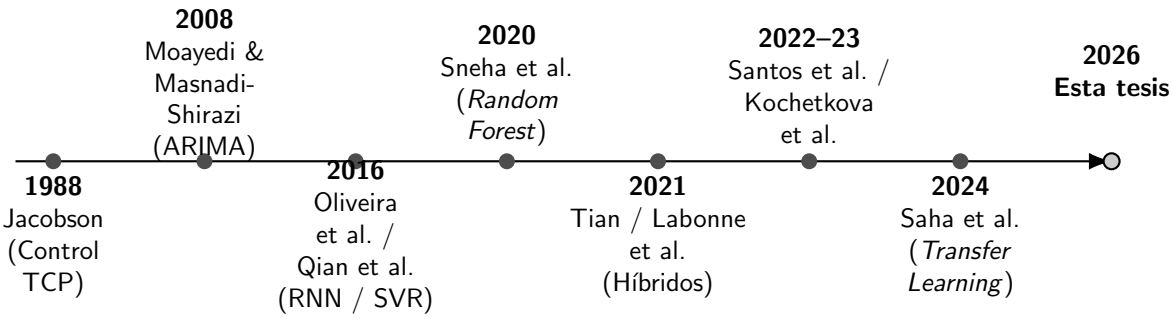


Figura 3.2: Línea de tiempo de la evolución metodológica en predicción de tráfico y congestión de red.

Esta visualización revela un patrón claro: durante casi dos décadas (1988–2008) predominaron los métodos estadísticos; a partir de 2016 comienza una transición gradual y simultánea hacia el ML clásico y las primeras arquitecturas de *Deep Learning*; y a partir de 2020 los tres paradigmas coexisten activamente en la literatura, sin que ninguno haya desplazado por completo a los anteriores. Esta coexistencia —más que una sustitución progresiva— es, como se argumentará más adelante, uno de los elementos que justifica el diseño comparativo de esta tesis.

Síntesis de la literatura

La literatura revisada se ha concentrado en predecir el **volumen de tráfico** como serie temporal continua (Moayedi & Masnadi-Shirazi [14]; Tian [15]; Kochetkova et al. [16]; Oliveira et al. [20]; Santos et al. [22]), y trata la congestión como una consecuencia implícita de ese volumen. Solo Sneha et al. [17] y, de forma indirecta, Qian et al. [18] la abordan como una variable discreta y accionable, cercana a la clasificación binaria de la Ecuación 2.2. Esta orientación responde a que el objetivo predominante es optimizar la gestión de recursos (asignación de ancho de banda, planeación de capacidad, QoS) y no prevenir eventos concretos de degradación, lo que explica el uso generalizado de métricas de error continuo (RMSE, MAE, MAPE) en lugar de precisión, *recall* o *F1-score*, más informativas para el alertamiento temprano que plantea esta tesis. Además, cada trabajo evalúa un solo paradigma de modelado, o variantes dentro de uno, y ninguno compara estadísticos, ML clásico y *Deep Learning* sobre los mismos datos; los de ML clásico suelen apoyarse en datos simulados (Tabla 3.1), y los que usan datos reales rara vez validan en más de un escenario. Aunque casi todos aspiran a una gestión proactiva (Sección 1.1), pocos definen un umbral operativo que permita actuar, con la excepción de Sneha et al. [17]. La investigación reciente (Figura 3.2) apunta hacia la aplicabilidad con datos limitados (Saha et al. [21]) y hacia comparaciones más rigurosas entre arquitecturas, que muestran que ningún modelo es superior en todo contexto (Santos et al. [22]).

A partir de lo anterior es posible formular con precisión la brecha que esta tesis busca cubrir, articulada en tres componentes:

1. **Brecha conceptual**: la literatura revisada predice predominantemente el *volumen* de tráfico como serie continua, mientras que esta tesis predice la **congestión** como evento

discreto y accionable, mediante la definición operativa formalizada en la Ecuación 2.2 (Sección 2.1.3), permitiendo abordar el problema tanto desde la regresión como desde la clasificación (Sección 2.2.1).

2. **Brecha metodológica**: ningún trabajo revisado compara de manera simultánea y bajo condiciones experimentales controladas representantes de los tres paradigmas de modelado —estadístico (ARIMA), ML clásico (*Random Forest*) y *Deep Learning* (LSTM/GRU)—, tal como se plantea en el diseño metodológico de esta tesis (Capítulo 4).

3. **Brecha de validación**: la mayoría de los trabajos revisados se apoyan en un único *dataset*, ya sea real o simulado, sin contrastar sus hallazgos frente a escenarios de red de naturaleza o escala distinta. Esta tesis, mediante el enfoque híbrido descrito en la Sección 4.4 —combinando la red privada de escala reducida bajo estudio con el archivo de tráfico de gran escala MAWI—, aborda directamente esta limitación, en línea con la recomendación implícita de trabajos como el de Saha et al. [21] respecto a la necesidad de validar modelos más allá de un único escenario de datos.

La Tabla 3.4 sintetiza visualmente este posicionamiento, contrastando las características metodológicas predominantes en la literatura revisada frente al diseño adoptado en esta investigación.

Dimensión	**Literatura revisada (predominante)**	**Esta tesis**
Variable objetivo	Volumen de tráfico (continuo)	Congestión (discreta, vía U_{th})
Paradigmas comparados	Uno solo, o variantes de uno solo	Tres (estadístico, ML clásico, DL) simultáneamente.
Naturaleza de los datos	Real o simulado, una sola fuente	Real, dos fuentes complementarias (híbrido).
Escala de validación	Una sola escala de red	Dos escalas contrastantes (privada reducida y tránsito de gran escala).
Umbral accionable	Raramente explícito	Definido formalmente (U_{th}, Ecuación 2.2).

Cuadro 3.4: Posicionamiento metodológico de esta tesis frente a la literatura revisada.

Aporte de esta tesis a la literatura existente

En correspondencia con las tres brechas identificadas, esta tesis ofrece tres aportes concretos a la comunidad académica. Primero, aporta evidencia empírica comparativa entre los

tres paradigmas de modelado revisados en este capítulo, evaluados bajo condiciones experimentales controladas y sobre los mismos conjuntos de datos —un tipo de comparación ausente en la literatura revisada, donde cada paradigma suele evaluarse de manera aislada—. Segundo, aporta un caso de estudio de recolección de datos reales en una red heterogénea de pequeña escala (residencial/PYME con videovigilancia, servicios en la nube y tráfico de ISP coexistiendo en el mismo enlace), un escenario significativamente menos representado en la literatura que las redes troncales de gran escala o los entornos simulados de laboratorio. Tercero, aporta una definición operativa explícita y formalizada de congestión (Ecuación 2.2), que traduce directamente la salida de los modelos predictivos en un criterio accionable para la toma de decisiones, respondiendo a la limitación de ümbral accionable raramente explícitoïdentificada en la Tabla 3.4.

Recíprocamente, esta tesis también reconoce su deuda con la literatura revisada: la definición del umbral operativo U_{th} se apoya en la evidencia de estacionalidad documentada por Kochetkova et al. [16]; el diseño de la comparación LSTM/GRU replica la metodología validada por Santos et al. [22] sobre un periodo de recolección equivalente; y la estrategia de mitigación ante *datasets* de tamaño reducido, en caso de ser necesaria, se apoyaría directamente en el precedente de *transfer learning* establecido por Saha et al. [21]. De esta manera, el capítulo de Metodología que sigue a continuación (Capítulo 4) no parte de cero, sino que se construye deliberadamente sobre las fortalezas identificadas y en respuesta directa a las limitaciones documentadas a lo largo de esta revisión del estado del arte.

Capítulo 4

Metodología

Los capítulos anteriores construyeron el fundamento conceptual de esta tesis: se definió formalmente qué se entiende por congestión de red (Capítulo 2, Ecuación 2.2), y se identificaron, mediante la revisión sistemática de la literatura, tres brechas específicas que este trabajo busca cubrir (Capítulo 3, Sección 3.5). Corresponde ahora al presente capítulo dar el paso de la revisión a la ejecución: traducir esas definiciones y las brechas identificadas en un plan de trabajo preciso, replicable y justificado en cada una de sus decisiones.

A diferencia de los capítulos precedentes, en los que el objeto de estudio era la producción académica de terceros, este capítulo describe exclusivamente decisiones metodológicas propias de esta investigación: qué tipo de investigación se está realizando, cómo se define y mide operativamente cada variable, de dónde provienen los datos, cómo se procesan, qué modelos se implementan y con qué criterios se evalúan. Cada una de estas decisiones no es arbitraria, sino que responde directamente a lo establecido en los capítulos anteriores —la definición operativa de congestión, el *dataset* híbrido descrito en la Sección 4.4, y los tres paradigmas de modelado comparados en la Sección 2.2—, por lo que el lector encontrará a lo largo de este capítulo referencias constantes hacia atrás, cerrando el círculo argumentativo abierto desde el planteamiento del problema.

Es importante señalar que las decisiones metodológicas aquí documentadas no necesariamente se tomaron en un único momento ni de manera lineal; muchas de ellas —como la elección del *dataset* híbrido o la definición del umbral de congestión— fueron resultado de un proceso iterativo de análisis y ajuste. Sin embargo, este capítulo las presenta de manera ordenada y sistemática, tal como corresponde a un documento metodológico formal, comenzando por la caracterización general del tipo de investigación que enmarca a esta tesis.

4.1. Tipo de investigación

La presente investigación se clasifica, de acuerdo con los criterios propuestos por Hernández-Sampieri [24], en cuatro dimensiones complementarias: su finalidad, su enfoque, su diseño y su alcance. Esta caracterización no es un mero formalismo, sino que determina directamente las decisiones metodológicas que se detallan en el resto de este capítulo, desde la forma en que se recolectan los datos hasta los criterios empleados para evaluar los modelos desarrollados.

Según su finalidad: investigación aplicada

Esta tesis constituye una investigación de tipo **aplicada** (también denominada tecnológica), en tanto que su propósito no es generar conocimiento teórico nuevo sobre las técnicas de *Machine Learning* empleadas —ARIMA, *Random Forest* y LSTM, cuyos fundamentos matemáticos ya se encuentran plenamente establecidos y fueron revisados en la Sección 2.2—, sino aplicar dicho conocimiento existente a la resolución de un problema práctico y delimitado: la predicción de congestión en un escenario real de red, tal como se planteó en la Sección 1.1. El criterio de éxito de esta investigación, por tanto, no reside en la novedad matemática de los modelos empleados, sino en la evidencia empírica que se obtenga sobre su viabilidad y desempeño comparativo al resolver el problema planteado.

Según su enfoque: investigación cuantitativa

El enfoque adoptado es **cuantitativo**, ya que la totalidad de las variables involucradas —utilización del enlace, latencia, pérdida de paquetes (Sección 2.1.4)— se expresan como magnitudes numéricas susceptibles de medición directa. El análisis de los resultados se apoya estrictamente en las métricas estadísticas de error (RMSE, MAE, MAPE) y en las métricas de clasificación (Precisión, Recall, *F1-Score*) definidas y fundamentadas teóricamente en el Capítulo 2. No se contempla, en ningún punto del desarrollo de esta tesis, la recolección o el análisis de datos de naturaleza cualitativa.

Según su diseño: mixto secuencial (observacional y experimental)

Desde la perspectiva de su diseño, esta investigación adopta un enfoque **mixto secuencial**, compuesto por dos fases distintas que reflejan la dualidad del problema abordado:

En su primera fase (Recolección de datos), el diseño es predominantemente **no experimental y longitudinal**. Se observa y registra el comportamiento de la red a lo largo de un periodo extendido de 30 días para capturar la evolución y estacionalidad del tráfico. Si bien se emplea instrumentación híbrida —incluyendo sondeos activos ligeros (ICMP) como se documentará en la Sección 4.4—, no se manipulan artificialmente las cargas de trabajo de los usuarios ni se induce deliberadamente la congestión en la infraestructura de la red privada bajo estudio.

En su segunda fase (Modelado predictivo), el diseño transita hacia un enfoque **experimental computacional**. El desarrollo, entrenamiento y evaluación de los modelos de *Machine Learning* (ARIMA, *Random Forest*, LSTM/GRU) requiere la manipulación sistemática y controlada de variables independientes —tales como hiperparámetros, arquitecturas de red y ventanas de rezago (*lags*)— para medir su efecto sobre las variables dependientes, representadas por las métricas de error predictivo. Esta fase se ejecuta en un entorno de trabajo aislado, garantizando que la experimentación algorítmica no impacte la red de producción.

Según su alcance: comparativo y explicativo

Finalmente, esta investigación posee un alcance **comparativo**, en tanto que su objetivo central no se limita a describir el comportamiento del tráfico de red, sino a contrastar

sistemáticamente el desempeño de tres paradigmas de modelado predictivo —estadístico, *Machine Learning* clásico y *Deep Learning*— evaluados bajo las mismas condiciones experimentales, tal como se estableció en la brecha metodológica identificada en la Sección 3.5. Este alcance comparativo se complementa con un componente **explicativo**, en la medida en que no basta con determinar *cuál* modelo ofrece el mejor desempeño, sino que se busca argumentar *por qué* dicho modelo se desempeña mejor —por ejemplo, si su ventaja se debe a la naturaleza no lineal del tráfico, a la presencia de dependencias temporales de largo plazo, o a la escala del conjunto de datos empleado—, apoyándose para ello en los fundamentos teóricos desarrollados en el Capítulo 2 y en la evidencia comparativa documentada en el Capítulo 3.

La Tabla 4.1 sintetiza esta clasificación de manera esquemática.

Dimensión	**Clasificación**	**Justificación breve**
Finalidad	Aplicada / tecnológica	Aplica técnicas de *Machine Learning* existentes a un problema práctico de redes.
Enfoque	Cuantitativa	Variables numéricas; análisis estadístico de error.
Diseño	No experimental, longitudinal	Observación pasiva de la red durante 30 días, sin manipulación.
Alcance	Comparativo-explicativo	Contrasta tres paradigmas de modelado y explica sus diferencias de desempeño.

Cuadro 4.1: Clasificación del tipo de investigación de esta tesis.

4.2. Definición operativa de congestión

De acuerdo con la Ecuación 2.2 establecida en el Marco Teórico, la operacionalización de la variable objetivo exige definir valores numéricos precisos para el umbral crítico de utilización (U_{th}) y la ventana temporal sostenida (Δt). Esta sección establece los parámetros base que gobernarán el etiquetado (*labeling*) del conjunto de datos antes de su ingesta en los modelos de *Machine Learning*.

Justificación del umbral U_{th}

Se adopta como valor inicial $U_{th} = 0{,}80$ (80 % de utilización), por dos razones convergentes. En primer lugar, desde la perspectiva teórica desarrollada en la Sección 2.1.3, la teoría de colas aplicada a redes de datos establece que el tiempo de espera promedio en un sistema de colas crece de manera proporcional a $\frac{1}{1-\rho}$, donde ρ representa la utilización del recurso [6]. Esta expresión matemática diverge conforme $\rho \to 1$, y su crecimiento deja de ser marginal precisamente en la vecindad de $\rho = 0{,}8$, punto a partir del cual pequeños incrementos adicionales en la utilización producen incrementos desproporcionadamente grandes en el tiempo de encolado. En segundo lugar, desde la perspectiva práctica de la industria, un umbral de utilización entre el 70 % y el 80 % constituye una referencia ampliamente adoptada en la

operación de redes empresariales para la activación de mecanismos de gestión de colas y alertamiento temprano; un criterio que se refleja, por ejemplo, en los valores por defecto de mecanismos de descarte preventivo como WRED (*Weighted Random Early Detection*) en equipos comerciales, los cuales suelen configurarse con umbrales iniciales de descarte alrededor del 80 % de ocupación del *buffer*.

Este valor de $U_{th} = 0{,}80$ funge como parámetro de control inicial (línea base) derivado de la literatura. Sin embargo, para garantizar que el modelo predictivo se ajuste a la realidad topológica de la red bajo estudio, este umbral será sometido a validación empírica durante el Análisis Exploratorio de Datos. El valor definitivo para el etiquetado del algoritmo de clasificación se fijará matemáticamente identificando el percentil exacto donde la distribución de utilización histórica comience a correlacionarse con fluctuaciones anómalas en la red.

Justificación de la ventana temporal Δt

En congruencia con la granularidad de muestreo establecida para la recolección de datos (Sección 4.4), se define $\Delta t = 5$ minutos como el intervalo mínimo sostenido durante el cual la condición $U(t) \geq U_{th}$ debe cumplirse para que un periodo se etiquete como congestión. Esta ventana, equivalente a cinco muestras consecutivas bajo una granularidad de muestreo de un minuto, responde a un balance deliberado entre dos riesgos opuestos: una ventana demasiado corta (por ejemplo, una única muestra) resulta susceptible a ráfagas instantáneas y transitorias de tráfico que no representan una congestión sostenida ni operacionalmente relevante —generando así falsos positivos que degradarían la utilidad práctica del sistema de predicción—; mientras que una ventana demasiado larga podría pasar por alto episodios de saturación breves pero significativos para aplicaciones sensibles a la latencia, como las cámaras de videovigilancia o las videollamadas mencionadas en la Sección 4.4. Al igual que con U_{th}, este valor podrá ser ajustado tras el análisis exploratorio del *dataset* primario si la evidencia empírica así lo sugiere.

Formulación instanciada para la etapa de preprocesamiento

Tomando como base los parámetros iniciales, la lógica de etiquetado binario que se programará en el *pipeline* de preprocesamiento de datos se define como:

$$\text{Congestión} \iff U(t) \geq 0{,}80 \quad \text{durante al menos 5 minutos consecutivos} \tag{4.1}$$

4.3. Descripción de las métricas a utilizar

Habiendo definido en la Sección 2.1.4 el fundamento teórico de las métricas de rendimiento de red, y en la Sección 4.2 el criterio operativo de congestión, corresponde ahora precisar qué subconjunto específico de estas métricas se empleará como variables de entrada (*features*) de los modelos predictivos, cuál constituye la variable objetivo y qué variables adicionales derivadas se incorporarán para enriquecer la capacidad predictiva del sistema.

Variable objetivo

La variable objetivo (*target*) de esta tesis es, en todos los casos, el factor de utilización futuro del enlace, $U(t+h)$, donde h representa el horizonte de predicción. Dicha variable se emplea de dos formas distintas según el paradigma de modelado, conforme a lo establecido en la Sección 2.2.1:

- Como variable **continua** (tarea de regresión), para los modelos ARIMA, *Random Forest*, SVR y LSTM en su formulación de regresión.
- Como variable **binaria derivada** (tarea de clasificación), aplicando la Ecuación 4.1 sobre $U(t+h)$, para la formulación de clasificación de los mismos modelos.

Variables de entrada (*features*)

La Tabla 4.2 enumera las variables de entrada consideradas para el entrenamiento de los modelos, clasificadas en tres grupos: métricas directas (obtenidas mediante los mecanismos de instrumentación descritos en la Sección 4.4), variables temporales derivadas (que capturan patrones cíclicos sin necesidad de instrumentación adicional) y variables de rezago o ventana (que explicitan la dependencia temporal de la serie, necesaria para modelos que no procesan secuencias de manera nativa, como *Random Forest*).

Grupo	**Variable**	**Tipo**	**Unidad / Rango**	**Fuente**
Métricas directas	Utilización $U(t)$	Continua	Adimensional [0,1]	SNMP
	Latencia (RTT)	Continua	ms	ICMP
	Pérdida de paquetes	Continua	%	SNMP / ICMP
Temporales derivadas	Hora del día	Categórica cíclica	0–23 (seno/coseno)	*Timestamp*
	Día de la semana	Categórica cíclica	0–6 (seno/coseno)	*Timestamp*
	Indicador fin de semana	Binaria	{0, 1}	*Timestamp*
	Indicador horario laboral	Binaria	{0, 1}	*Timestamp*
Rezago / ventana	$U(t-1), U(t-2), \ldots, U(t-k)$	Continua	Adimensional [0,1]	Derivada de $U(t)$
	Media móvil de U (15 min)	Continua	Adimensional [0,1]	Derivada de $U(t)$

Cuadro 4.2: Variables de entrada consideradas para los modelos predictivos.

Conviene justificar dos decisiones reflejadas en la Tabla 4.2. Primero, las variables temporales cíclicas (hora del día, día de la semana) se codifican mediante transformaciones seno y coseno en lugar de emplearse directamente como enteros numéricos. Esto se realiza con el

fin de preservar su naturaleza cíclica, evitando, por ejemplo, que el modelo interprete que la hora 23 y la hora 0 se encuentran algebraicamente lejanas entre sí, cuando en la realidad temporal son consecutivas:

$$x_{\text{seno}} = \sin\left(\frac{2\pi \cdot x}{x_{\text{máx}}}\right), \qquad x_{\text{coseno}} = \cos\left(\frac{2\pi \cdot x}{x_{\text{máx}}}\right) \tag{4.2}$$

Segundo, las variables de rezago ($U(t-1)$ hasta $U(t-k)$) resultan indispensables para los modelos que no poseen una noción arquitectónica intrínseca de secuencia temporal (*Random Forest* y SVR), ya que estos algoritmos tratan cada observación como independiente y requieren que la dependencia temporal se proporcione de manera explícita en el vector de entrada. En contraste, la arquitectura LSTM procesa la secuencia de observaciones de manera nativa, por lo que estas variables de rezago, si bien pueden incluirse, no resultan estrictamente necesarias para dichos modelos.

Disponibilidad de métricas por fuente de datos

Dado que esta tesis emplea dos fuentes de datos de naturaleza distinta (Sección 4.4), no todas las métricas de la Tabla 4.2 se encuentran disponibles de la misma forma en ambos casos. La Tabla 4.3 contrasta la disponibilidad y el método de obtención de cada métrica en el *dataset* primario frente al *dataset* de validación (MAWI).

Métrica	***Dataset* primario**	***Dataset* de validación (MAWI)**
Utilización	Directa, vía SNMP en tiempo real	Derivada de trazas `.pcap` agregando bytes por ventana de tiempo
Latencia (RTT)	Directa, vía sondas ICMP activas	No disponible directamente; se aproxima mediante el tiempo entre paquetes de un flujo TCP
Pérdida de paquetes	Directa, vía contadores SNMP de descarte	Estimada indirectamente mediante retransmisiones TCP detectadas en la traza
Variables temporales	Directas, *timestamp* del muestreo	Directas, *timestamp* de captura de cada traza diaria

Cuadro 4.3: Disponibilidad y método de obtención de métricas según la fuente de datos.

Esta asimetría no invalida el enfoque híbrido adoptado, pero sí implica que, para el *dataset* de validación, algunas variables de entrada requieren un paso adicional de derivación durante el preprocesamiento, el cual se detalla en la Sección 4.5. Es fundamental advertir que esta diferencia en los métodos de recolección —particularmente entre la latencia limpia obtenida mediante sondeos ICMP en la red privada y la latencia ruidosa inferida a través de flujos TCP

en el archivo MAWI— introduce un desafío estadístico conocido como cambio de dominio (*domain shift*).

Si los algoritmos de *Machine Learning* asimilan directamente las magnitudes crudas de estas métricas, el desempeño predictivo colapsará durante la fase de validación debido al desajuste de escalas y distribuciones. Para mitigar esta divergencia estructural, se aplicará un proceso de escalado y normalización independiente (por ejemplo, mediante *StandardScaler* o *MinMaxScaler*) a cada conjunto de datos de manera aislada. Esta transformación, formalizada en la etapa de preprocesamiento (Sección 4.5), garantiza que los modelos aprendan la varianza relativa y los patrones de degradación de las métricas, independientemente de la magnitud absoluta dictada por las herramientas de monitoreo subyacentes.

Pipeline de construcción de variables

La Figura 4.1 ilustra esquemáticamente el flujo mediante el cual las métricas crudas, obtenidas directamente de la instrumentación descrita en la Sección 4.4, se transforman en el vector final de variables de entrada empleado por cada modelo.

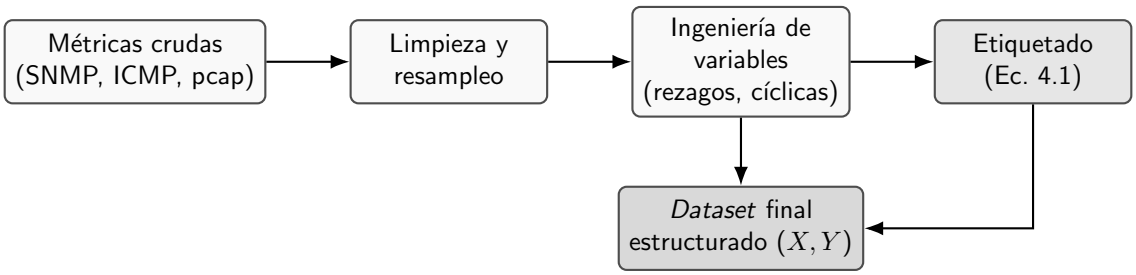


Figura 4.1: *Pipeline* de transformación de métricas crudas en variables de entrada y variable objetivo.

Este *pipeline* se aplica de manera independiente sobre cada una de las dos fuentes de datos, garantizando que el vector final de variables tenga una estructura equivalente entre ambas, condición necesaria para poder evaluar un mismo modelo entrenado sobre el *dataset* primario contra el conjunto de validación MAWI, tal como se planteó en el diseño comparativo de esta tesis (Sección 3.5). El detalle específico de cada etapa de este flujo —en particular, la limpieza de datos faltantes y el resampleo de las trazas MAWI— se desarrolla con mayor profundidad en la Sección 4.5 (Preprocesamiento de datos).

4.4. Fuente de datos

Para el desarrollo de esta tesis se adoptó una estrategia híbrida de recolección de datos, combinando una fuente primaria de tráfico real capturado de forma pasiva en una red administrada directamente por el autor, con una fuente secundaria de validación externa proveniente de un archivo académico de tráfico de Internet ampliamente reconocido. Esta combinación busca equilibrar dos necesidades que, de manera aislada, ningún *dataset* por sí solo podría satisfacer plenamente: por un lado, la posibilidad de definir con precisión el proceso de recolección, las métricas capturadas y el contexto operativo del enlace bajo estudio;

y por otro, la necesidad de validar que los modelos desarrollados no se sobreajusten a las particularidades de un único escenario, sino que sean capaces de generalizar su capacidad predictiva a condiciones de tráfico distintas en naturaleza y escala.

Dataset primario: Red privada PYME

La fuente primaria de datos proviene de una red privada PYME sobre la cual el autor tiene acceso administrativo completo, en el contexto de un proyecto de despliegue de infraestructura. Dicha red opera bajo un enrutador MikroTik que concentra y gestiona simultáneamente múltiples servicios que compiten por el mismo enlace de salida a Internet. Es fundamental delimitar la topología física de este acceso: el tráfico hacia el exterior se enruta a través de una única interfaz WAN (como un enlace LTE), lo cual condiciona intrínsecamente el perfil base de latencia, *jitter* y variabilidad estocástica del canal. Sobre esta infraestructura coexisten un sistema de videovigilancia con transmisión continua (NVR), tráfico de uso corporativo/residencial cotidiano, y un servicio de almacenamiento en la nube (Nextcloud) accesible de forma remota mediante una VPN SSTP.

Esta arquitectura de servicios genera un escenario de estrés operativo altamente propicio para la investigación de congestión. En particular, la encapsulación de tráfico TCP (como el generado por Nextcloud) dentro de un túnel VPN que también opera sobre TCP (SSTP) introduce una susceptibilidad arquitectónica al fenómeno conocido como *TCP Meltdown*. Bajo esta condición, si el enlace físico experimenta una ligera pérdida de paquetes por saturación, tanto la capa de aplicación como la capa del túnel intentan retransmitir la información de manera simultánea y descoordinada. Esta redundancia de retransmisiones amplifica la congestión original, colapsando rápidamente la capacidad del enlace mediante un patrón no lineal. La captura de estas dinámicas complejas convierte a este conjunto de datos en un entorno de validación excepcional para evaluar si los modelos de *Machine Learning* logran anticipar el colapso con mayor precisión que los modelos estadísticos tradicionales.

Es importante señalar que, dado que los datos provienen de la infraestructura de un tercero, se obtuvo la autorización explícita del propietario de la red para realizar el monitoreo pasivo de metadatos de tráfico con fines exclusivamente académicos, en línea con las buenas prácticas de investigación en redes de computadoras que involucran datos de operación reales [25].

Dataset secundario: MAWI Working Group Traffic Archive

Como fuente de validación externa se empleará el **MAWI Working Group Traffic Archive**, un repositorio de trazas de tráfico mantenido de forma continua desde 2006 por el proyecto WIDE, que captura de manera diaria el tráfico del enlace de tránsito entre la red académica WIDE y su proveedor ascendente en Tokio, Japón [23]. A diferencia del *dataset* primario, que corresponde a una red de escala reducida con un puñado de servicios concurrentes, las trazas de MAWI capturan tráfico agregado de un enlace de tránsito de gran escala (entre 1 y 10 Gbps), compuesto por miles de flujos simultáneos provenientes de múltiples organizaciones y aplicaciones. Las direcciones IP contenidas en estas trazas se encuentran anonimizadas mediante un esquema que preserva la estructura de prefijos de red (*prefix-preserving anonymization*, empleando el algoritmo Crypto-PAn), lo cual elimina cualquier

posibilidad de identificar usuarios individuales sin comprometer la utilidad analítica de los datos para el estudio de patrones agregados de tráfico.

La incorporación de este segundo *dataset* responde a una limitación reconocida en la literatura sobre predicción de tráfico de red: los modelos entrenados y evaluados sobre una única fuente de datos tienden a mostrar un desempeño optimista que no necesariamente se sostiene cuando se exponen a condiciones de tráfico distintas a las observadas durante el entrenamiento [3]. Al validar los modelos desarrollados en esta tesis sobre las trazas de MAWI —de una naturaleza y escala sustancialmente distinta a la del *dataset* primario—, es posible obtener evidencia más robusta sobre la capacidad de generalización de los enfoques de *Machine Learning* propuestos, más allá de las particularidades de un único escenario de red.

Dado que las trazas de MAWI se distribuyen en formato de captura de paquetes (`.pcap`) y no como series temporales de utilización previamente agregadas, fue necesario un procesamiento adicional para derivar de ellas las mismas métricas empleadas en el *dataset* primario. Este procesamiento se describe con mayor detalle en la Sección 4.5 (Preprocesamiento de datos).

Consideraciones éticas y de privacidad

En ambos casos, la recolección y el procesamiento de los datos se rigen por un principio de minimización: se capturan y almacenan únicamente los metadatos estrictamente necesarios para el análisis de utilización, latencia y pérdida de paquetes, sin acceder en ningún momento al contenido de las comunicaciones transportadas. En el caso del *dataset* primario, se cuenta con la autorización expresa del propietario de la infraestructura; en el caso de MAWI, el uso de los datos se limita a fines de investigación, en estricto cumplimiento con los términos de uso establecidos por el propio archivo, los cuales restringen su empleo a propósitos académicos y prohíben cualquier acción que comprometa la privacidad de los usuarios cuyo tráfico fue originalmente capturado.

4.5. Preprocesamiento de datos

El *pipeline* esquematizado en la Figura 4.1 (Sección 4.3) identificó cuatro grandes etapas de transformación entre las métricas crudas y el vector final de variables de entrada. Esta sección detalla técnicamente cada una de dichas etapas, prestando especial atención a los puntos en los que el tratamiento del *dataset* primario y el *dataset* de validación (MAWI) difiere, dada la naturaleza distinta de ambas fuentes descrita en la Sección 4.4.

Limpieza de datos

La limpieza de datos aborda dos problemas recurrentes en la recolección de series temporales de red: valores faltantes y valores atípicos (*outliers*).

Los **valores faltantes** pueden originarse por interrupciones temporales del proceso de monitoreo, fallos de conectividad hacia el propio dispositivo de consulta SNMP, o reinicios del equipo de red. Para intervalos faltantes de corta duración (hasta 3 muestras consecutivas,

equivalente a 3 minutos bajo la granularidad establecida), se aplicará interpolación lineal entre el último valor observado y el siguiente valor disponible. Para intervalos de mayor duración, el criterio adoptado es descartar dicho segmento de la serie por completo en lugar de imputarlo, ya que una interpolación extendida introduciría patrones artificiales que no reflejan el comportamiento real de la red, contaminando potencialmente el proceso de entrenamiento de los modelos.

Los **valores atípicos** se identifican mediante el criterio del rango intercuartílico (IQR), marcando como atípico cualquier observación que se encuentre fuera del rango:

$$[Q_1 - 1{,}5 \cdot \text{IQR},\ Q_3 + 1{,}5 \cdot \text{IQR}] \tag{4.3}$$

donde Q_1 y Q_3 son el primer y tercer cuartil de la variable analizada, e $\text{IQR} = Q_3 - Q_1$. Es importante señalar que, a diferencia de otros contextos de análisis de datos donde los atípicos se eliminan sistemáticamente, en esta tesis dichos valores **no se descartan de manera automática**: un pico súbito y extremo de utilización podría constituir precisamente el tipo de evento de congestión que se busca predecir, y no un error de medición. Por ello, cada valor marcado como atípico se contrasta manualmente contra las otras métricas disponibles en el mismo instante (por ejemplo, si un pico de utilización coincide con un aumento simultáneo de latencia y pérdida de paquetes, se conserva como evento genuino; si ocurre de manera aislada y sin correlato en las demás métricas, se trata como un posible error de instrumentación y se interpola).

Resampleo y agregación temporal

Como se documentó en la Tabla 4.3 (Sección 4.3), el *dataset* primario se obtiene ya en forma de serie temporal muestreada directamente a la granularidad deseada (1 minuto), mientras que el *dataset* MAWI se distribuye como capturas de paquetes individuales (`.pcap`). Por tanto, este último requiere un proceso adicional de **agregación temporal**: mediante herramientas de análisis de tráfico (`tshark` / `tstat`), los paquetes capturados se agrupan en ventanas no traslapadas de 1 minuto, calculando para cada ventana el total de bytes transmitidos, a partir de lo cual se deriva la utilización $U(t)$ mediante la Ecuación 2.3, empleando como ancho de banda nominal la capacidad conocida del enlace de tránsito capturado por MAWI.

La Tabla 4.4 resume este contraste entre ambas fuentes.

Aspecto	***Dataset* primario**	***Dataset* de validación (MAWI)**
Formato original	Serie temporal (SNMP/ICMP)	Captura de paquetes (`.pcap`)
Granularidad original	1 minuto (ya muestreada)	Por paquete individual (microsegundos)
Proceso requerido	Ninguno (uso directo)	Agregación mediante `tshark` / `tstat`
Granularidad final	1 minuto	1 minuto (tras agregación)

Cuadro 4.4: Contraste del proceso de resampleo entre las dos fuentes de datos empleadas.

Normalización de variables

Dado que los modelos de *Deep Learning* (LSTM) son sensibles a la escala de las variables de entrada —un gradiente calculado sobre variables de magnitudes muy dispares puede converger de manera inestable durante el entrenamiento—, todas las variables continuas de la Tabla 4.2 se normalizan mediante escalamiento Min-Max al rango $[0, 1]$:

$$x_{\text{norm}} = \frac{x - x_{\text{mín}}}{x_{\text{máx}} - x_{\text{mín}}} \tag{4.4}$$

Se opta por escalamiento Min-Max en lugar de estandarización Z-score porque la variable de mayor interés, la utilización $U(t)$, posee un rango natural acotado $[0, 1]$ (Sección 2.1.3). Un punto metodológico crítico: para el *dataset* primario, los valores $x_{\text{mín}}$ y $x_{\text{máx}}$ se calculan **estrictamente** a partir del conjunto de entrenamiento, aplicándose posteriormente sobre los conjuntos de validación y prueba para evitar la fuga de información (*data leakage*).

Sin embargo, como se advirtió en la Sección 4.3, el *dataset* de validación externa (MAWI) presenta un cambio de dominio (*domain shift*) significativo, particularmente en las distribuciones de latencia inferida. Por consiguiente, MAWI es sometido a un proceso de escalado independiente, ajustando sus propios parámetros de normalización para homologar las varianzas relativas antes de someterlo a los modelos preentrenados, garantizando así una evaluación justa de la capacidad de generalización algorítmica.

Ventaneo y construcción de rezagos (*Lags*)

Para que los algoritmos de aprendizaje automático puedan predecir a partir de series temporales, es necesario transformar la serie lineal en conjuntos de variables estructuradas mediante una **ventana deslizante** (*sliding window*). Este proceso aplica para todos los modelos, pero difiere en la topología de los datos generados:

Para los modelos clásicos (*Random Forest*), que carecen de memoria interna (Sección 2.2.3), la ventana aplana el historial temporal en un vector tabular de características (2D), donde cada observación pasada se convierte en una variable independiente. Por el contrario, para la arquitectura LSTM (Sección 2.2.4), el ventaneo preserva la estructura tridimensional

(3D) de secuencias temporales que la red recurrente consume de manera nativa. Dada una longitud de ventana w, cada bloque de entrada se construye como:

$$X_t = [U(t-w), U(t-w+1), \ldots, U(t-1)] \longrightarrow y_t = U(t+h) \tag{4.5}$$

donde w es el número de observaciones pasadas consideradas (análogo, aunque no idéntico en propósito, al orden p del modelo ARIMA presentado en la Sección 2.2.2) y h es el horizonte de predicción. La Figura 4.2 ilustra este proceso de manera esquemática.

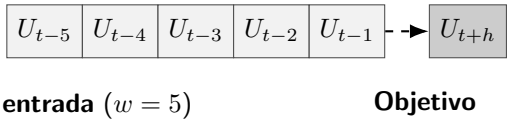


Figura 4.2: Construcción de secuencias de entrada mediante ventana deslizante para los modelos LSTM/GRU.

El valor específico de w (longitud de la ventana) y h (horizonte de predicción) se determinará experimentalmente en el Capítulo 5, evaluando el desempeño del modelo bajo distintas configuraciones, si bien como punto de partida se contemplan ventanas equivalentes a periodos de 30 a 60 minutos de historia (es decir, $w = 30$ a 60 bajo la granularidad de 1 minuto establecida), y horizontes de predicción de $h = 5$ a 15 minutos, congruentes con el propósito de alertamiento temprano planteado en el Capítulo 1.

División de datos: entrenamiento, validación y prueba

Un error metodológico común al trabajar con series temporales —y que esta tesis evita deliberadamente— es dividir los datos en conjuntos de entrenamiento y prueba mediante muestreo aleatorio, tal como se acostumbra en problemas de aprendizaje supervisado convencionales. Este enfoque resulta inadecuado en el presente contexto porque violaría el orden temporal de la serie: permitiría, por ejemplo, que el modelo se entrene con datos de un instante posterior y sea evaluado sobre un instante anterior, lo cual no solo carece de sentido práctico —un sistema de predicción real jamás tendría acceso a datos del futuro—, sino que produciría estimaciones de desempeño artificialmente optimistas por fuga de información temporal.

Por esta razón, la división de los datos se realiza de manera estrictamente **cronológica**, tal como se ilustra en la Figura 4.3: el segmento más antiguo de la serie se destina a entrenamiento, el segmento intermedio a validación (empleado para el ajuste de hiperparámetros) y el segmento más reciente a prueba (empleado exclusivamente para la evaluación final reportada en el Capítulo 6).

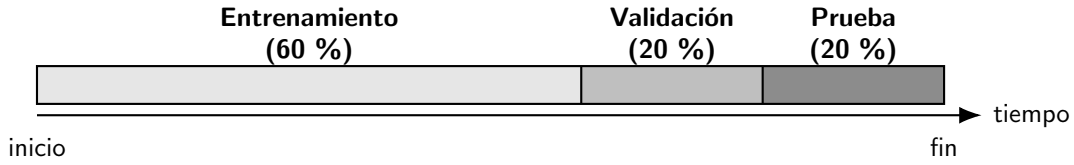


Figura 4.3: División cronológica del *dataset* primario en conjuntos de entrenamiento, validación y prueba.

Esta división se aplica de manera independiente sobre el *dataset* primario (proporciones 60/20/20 sobre los 30 días de recolección continua). El *dataset* MAWI, en cambio, se emplea íntegramente como un segundo conjunto de prueba externo —no se divide en entrenamiento, validación y prueba—, ya que su función metodológica, como se estableció en la Sección 4.4, es evaluar la capacidad de generalización de los modelos ya entrenados sobre el *dataset* primario, y no proveer datos adicionales de entrenamiento.

La Tabla 4.5 sintetiza las cinco etapas de preprocesamiento descritas en esta sección, sirviendo como cierre de este apartado y transición hacia la implementación concreta de los modelos, que se abordará en el Capítulo 5.

Etapa	**Descripción**
1. Limpieza	Interpolación de faltantes cortos; tratamiento contextual de atípicos (Ec. 4.3).
2. Resampleo	Agregación de MAWI a ventanas de 1 min; uso directo del *dataset* primario.
3. Normalización	Escalamiento Min-Max (Ec. 4.4), ajustado solo sobre entrenamiento.
4. Ventaneo	Construcción de secuencias para LSTM (Ec. 4.5).
5. División temporal	60/20/20 cronológico sobre el *dataset* primario; MAWI como prueba externa.

Cuadro 4.5: Resumen de las etapas de preprocesamiento de datos.

4.6. Modelos a implementar y justificación de su elección

Con base en los tres paradigmas de modelado presentados en el Marco Teórico (Sección 2.2) y en la brecha metodológica identificada en la síntesis del Estado del Arte (Sección 3.5) —la ausencia, en la literatura revisada, de una comparación simultánea entre modelos estadísticos, de *Machine Learning* clásico y de *Deep Learning* bajo condiciones experimentales controladas—, esta tesis contempla la implementación y evaluación comparativa de tres modelos específicos: ARIMA, *Random Forest* y LSTM. Esta sección justifica la elección de cada uno, así como la exclusión deliberada de otras alternativas que un lector familiarizado con el campo podría razonablemente esperar encontrar aquí.

ARIMA: modelo de referencia (*baseline*)

ARIMA se incluye no como un candidato a ofrecer el mejor desempeño absoluto, sino como **línea base de referencia** contra la cual contrastar el desempeño de los modelos subsiguientes, conforme a su rol descrito en la Sección 2.2.2. Su justificación no depende de una expectativa de alto desempeño, sino de un principio metodológico: sin un punto de comparación estadístico simple, cualquier ganancia de desempeño reportada por los modelos de ML clásico o *Deep Learning* carecería de un marco de referencia contra el cual dimensionar su magnitud real. Adicionalmente, como se documentó en la Sección 3.1, ARIMA y sus variantes continúan siendo objeto de investigación vigente en la literatura reciente [16], lo que respalda su pertinencia como punto de comparación actual y no meramente histórico.

Random Forest: *Machine Learning* clásico

La elección de *Random Forest* se fundamenta directamente en la evidencia reportada por Sneha et al. [17] (Sección 3.2), quienes encontraron que este modelo ofreció el mejor desempeño entre cuatro algoritmos clásicos evaluados para la predicción de congestión a nivel de enrutador. A esta evidencia empírica se suma una ventaja metodológica relevante para esta tesis: la capacidad de *Random Forest* de calcular la importancia relativa de cada variable de entrada (Sección 2.2.3), lo cual permitirá —una vez ejecutado el entrenamiento en el Capítulo 5— determinar cuáles de las variables descritas en la Tabla 4.2 (Sección 4.3) resultan más informativas para anticipar la congestión, aportando así evidencia adicional más allá de la mera precisión predictiva.

LSTM: *Deep Learning*

La justificación de este modelo se apoya directamente en el trabajo de Santos et al. [22] (Sección 3.3), quienes realizaron la comparación metodológicamente más cercana a la planteada en esta tesis —LSTM contra GRU, sobre tráfico de Internet real, con un periodo de recolección del mismo orden de magnitud que el establecido para el *dataset* primario de esta investigación (Sección 4.4)—. En su estudio, los autores demostraron empíricamente que la arquitectura LSTM superó en desempeño predictivo a GRU en la mayoría de los agrupamientos y escenarios geográficos analizados. Con base en esta evidencia, esta tesis adopta LSTM como el representante óptimo y único del paradigma de redes neuronales recurrentes, destinando los recursos computacionales a la sintonización exhaustiva de este modelo en lugar de duplicar esfuerzos en arquitecturas redundantes.

Modelos considerados y no incluidos

Es previsible que, ante la elección de los tres modelos anteriores, surja la pregunta de por qué no se incluyeron otras alternativas frecuentemente mencionadas en la literatura de predicción de series temporales. La Tabla 4.6 explicita esta decisión y su justificación.

Modelo no incluido	Justificación de exclusión
XGBoost	Si bien supera frecuentemente a *Random Forest*, XGBoost se basa en *Boosting* (reducción secuencial de sesgo), lo que lo hace altamente susceptible al sobreajuste (*overfitting*) en conjuntos de datos pequeños o con métricas ruidosas (como la latencia TCP inferida), a menos que se realice una sintonización de hiperparámetros exhaustiva. Se optó por *Random Forest* (*Bagging*) por su mayor robustez inherente al ruido operativo.
SVR y GRU	Excluidos para evitar redundancia paradigmática. SVR pertenece a la misma familia de ML clásico que *Random Forest*, y GRU demostró ser inferior a LSTM en literatura similar [22]. La metodología priorizó optimizar a fondo los tres mejores representantes de cada enfoque en lugar de implementar múltiples variantes del mismo tipo.
Prophet	Diseñado principalmente para series con estacionalidad de negocio (ventas, demanda), con supuestos que no se ajustan al tráfico de red a granularidad de minutos.
Transformers	Su entrenamiento efectivo típicamente requiere volúmenes de datos considerablemente mayores a los disponibles en el *dataset* primario de esta tesis (30 días en red reducida).

Cuadro 4.6: Modelos considerados durante la planeación de esta tesis y justificación de su exclusión del alcance experimental.

Diseño experimental general

La Figura 4.4 sintetiza el diseño experimental completo que resulta de las decisiones justificadas en esta sección.

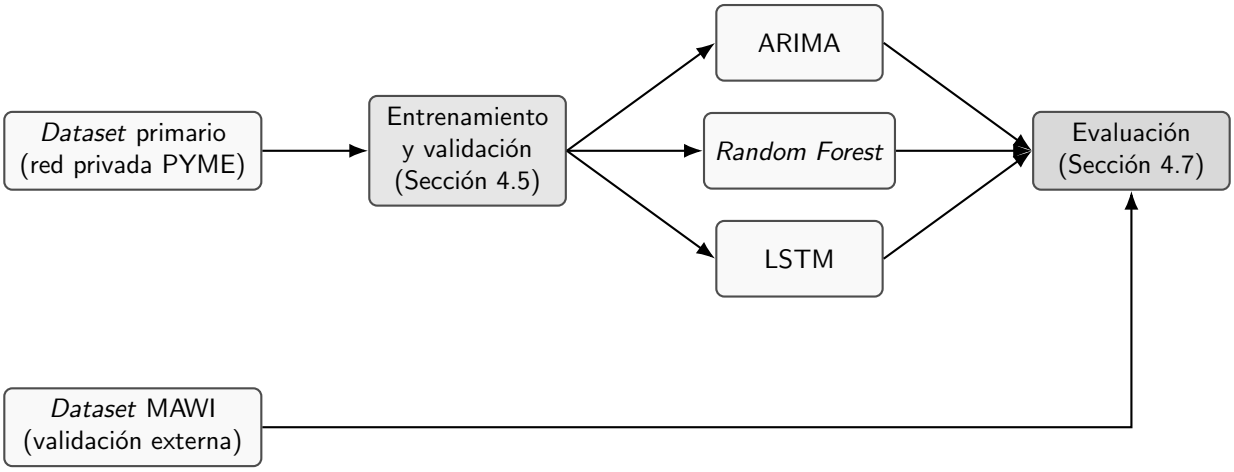


Figura 4.4: Diseño experimental general: entrenamiento sobre el *dataset* primario y validación cruzada contra MAWI.

Es importante precisar una distinción operativa: mientras que *Random Forest* y LSTM pueden entrenarse explícitamente para clasificar, ARIMA operará estrictamente como regresor continuo, realizando la clasificación de manera derivada aplicando el umbral U_{th} (Ecuación 4.1). Las afirmaciones sobre qué modelo podría desempeñarse mejor constituyen hipótesis fundamentadas en la literatura y se pondrán a prueba empírica en los Capítulos 5 y 6.

4.7. Métricas de evaluación del modelo

Como se estableció en la Sección 4.3, el problema de predicción de congestión abordado en esta tesis se formula bajo dos paradigmas complementarios: regresión (predicción del valor continuo $U(t+h)$) y clasificación (predicción binaria de congestión mediante la Ecuación 4.1). En congruencia con esta doble formulación, esta sección define dos conjuntos de métricas de evaluación independientes, así como los criterios bajo los cuales se compararán los cinco modelos justificados en la Sección 4.6.

Métricas para la tarea de regresión

Para evaluar el desempeño de los modelos en su formulación de regresión se emplearán las tres métricas presentadas formalmente en el Capítulo 2: RMSE (Ecuación 2.15), MAE (Ecuación 2.16) y MAPE (Ecuación 2.17). La Tabla 4.7 resume el propósito específico que cada una cumple dentro del análisis comparativo de esta tesis, evitando así que su inclusión conjunta se perciba como una simple acumulación de métricas sin criterio.

Métrica	Sensibilidad	Rol en esta tesis
RMSE	Penaliza fuertemente errores grandes	Detectar modelos que fallan de forma severa ante picos de congestión.
MAE	Pondera todos los errores por igual	Interpretación directa del error típico esperado, en las mismas unidades que U.
MAPE	Expresa el error como porcentaje relativo	Comparar el desempeño de un mismo modelo entre el *dataset* primario y MAWI, a pesar de operar a escalas de tráfico distintas.

Cuadro 4.7: Rol de cada métrica de regresión en el análisis comparativo de esta tesis.

Métricas para la tarea de clasificación

Para la formulación de clasificación binaria (congestión / no congestión), las métricas de error continuo de la subsección anterior resultan insuficientes, ya que no capturan directamente el tipo de aciertos y errores más relevantes para un sistema de alertamiento. Se requiere, en su lugar, un conjunto de métricas derivadas de la **matriz de confusión**, ilustrada en la Figura 4.5, que clasifica cada predicción en una de cuatro categorías según la coincidencia entre el valor predicho y el valor real observado.

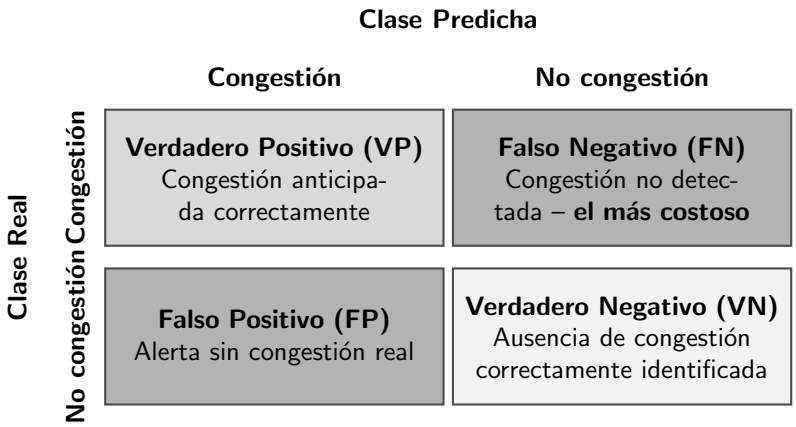


Figura 4.5: Matriz de confusión para la tarea de clasificación de congestión.

A partir de estas cuatro cantidades se calculan la **Exactitud**, **Precisión**, **Recall** y el ***F1-Score***, cuyas formulaciones matemáticas ya fueron establecidas rigurosamente en el Marco Teórico (Capítulo 2).

Es necesario justificar, en el contexto específico de esta tesis, por qué estas métricas no deben ponderarse de manera equivalente. Como se señala en la Figura 4.5, un **falso negativo**

—un episodio real de congestión que el modelo no logra anticipar— resulta considerablemente más costoso, desde la perspectiva del problema planteado en el Capítulo 1, que un **falso positivo** —una alerta emitida ante una situación que finalmente no deriva en congestión—. Esto se debe a que un falso positivo implica, en el peor de los casos, una revisión innecesaria por parte de un administrador de red o una medida preventiva no estrictamente requerida, mientras que un falso negativo implica exactamente el escenario que esta tesis busca evitar: la degradación del servicio sin ningún tipo de anticipación, replicando la limitación reactiva señalada en el Planteamiento del Problema (Sección 1.1). En consecuencia, esta tesis prioriza el **Recall** sobre la exactitud general como criterio principal de comparación entre modelos, sin descuidar la precisión, para lo cual el ***F1-score*** —que balancea ambas— se reporta como métrica de síntesis. Adicionalmente, dado que la exactitud simple puede resultar engañosa cuando la clase de interés (congestión) es minoritaria dentro del conjunto de datos —una condición previsible dado que se espera que la red bajo estudio opere por debajo del umbral U_{th} la mayor parte del tiempo—, esta métrica se reportará únicamente como referencia general y no como criterio determinante de comparación.

Complementariamente, se reportará también la curva ROC (*Receiver Operating Characteristic*) y el área bajo dicha curva (AUC), que permiten evaluar el desempeño del clasificador de manera independiente al umbral de decisión específico empleado, ofreciendo una visión más completa de su capacidad discriminativa entre los estados de congestión y no congestión.

Una métrica adicional propia de esta tesis: tiempo de anticipación efectivo

Más allá de las métricas convencionales descritas anteriormente, y en congruencia con el propósito de alertamiento temprano planteado desde el Capítulo 1, esta tesis propone evaluar una métrica adicional, definida específicamente para este trabajo: el **tiempo de anticipación efectivo** (TAE), que mide, para cada verdadero positivo correctamente detectado, cuántos minutos antes de que la congestión real ocurriera el modelo emitió la alerta correspondiente:

$$TAE_i = t_{\text{congestión real},i} - t_{\text{predicción},i} \tag{4.6}$$

Esta métrica no sustituye a las anteriores, sino que las complementa: un modelo podría, en principio, alcanzar valores aceptables de *Recall* y *F1-score* prediciendo la congestión con muy poca antelación (por ejemplo, un minuto antes de que ocurra), lo cual tendría escasa utilidad práctica para que un administrador de red o un sistema automatizado tome medidas correctivas a tiempo. El TAE promedio, reportado junto con su desviación estándar, permitirá evaluar si los horizontes de predicción h explorados en la Sección 4.5 (entre 5 y 15 minutos) resultan, en la práctica, suficientes para el propósito de anticipación planteado en esta investigación.

Síntesis del esquema de evaluación

La Tabla 4.8 sintetiza el esquema completo de evaluación que se aplicará de manera uniforme a los cinco modelos justificados en la Sección 4.6, tanto sobre el conjunto de prueba

del *dataset* primario como sobre el conjunto de validación externa MAWI, conforme al diseño experimental ilustrado en la Figura 4.4.

Formulación	**Métricas**	**Criterio principal**
Regresión	RMSE, MAE, MAPE	RMSE
Clasificación	Exactitud, Precisión, *Recall*, F1, AUC-ROC	*Recall*, sintetizado con F1
Utilidad práctica	Tiempo de Anticipación Efectivo (TAE)	Máximo TAE con *Recall* aceptable

Cuadro 4.8: Síntesis del esquema de evaluación aplicado a los modelos de esta tesis.

Al igual que en la Sección 4.6, es importante reiterar que las prioridades establecidas en esta sección —privilegiar *Recall* sobre exactitud, complementar con el TAE propuesto— constituyen decisiones de diseño metodológico justificadas por la naturaleza del problema planteado, y no resultados obtenidos. Los valores concretos que estas métricas arrojen para cada modelo se presentarán y discutirán en el Capítulo 6.

4.8. Herramientas y entorno

Esta sección final del capítulo de Metodología documenta el conjunto de herramientas de *software*, bibliotecas y el entorno de cómputo que se emplearán a lo largo del desarrollo experimental descrito en el Capítulo 5, organizadas según la etapa del *pipeline* metodológico a la que corresponden (Figura 4.1, Sección 4.3). Esta documentación responde a un criterio de reproducibilidad: cualquier lector con acceso a un entorno equivalente debería poder replicar, al menos en sus aspectos esenciales, el procedimiento aquí descrito.

Herramientas de recolección e instrumentación

Para la obtención de las métricas del *dataset* primario, descritas en la Sección 4.4, se empleará el conjunto de herramientas resumido en la Tabla 4.9, congruente con la estrategia de monitoreo híbrido establecida previamente.

Herramienta	Propósito	Métrica asociada
RouterOS (MikroTik)	Exposición de contadores SNMP nativos de la interfaz WAN	Utilización (Ec. 2.3)
Net-SNMP / `pysnmp`	Consulta periódica de contadores SNMP desde un *script* externo	Utilización, descartes
`ping` (ICMP)	Medición activa de latencia y pérdida de paquetes	RTT, pérdida (Ecs. 2.4, 2.7)
Script de recolección (Python)	Orquestación de las consultas anteriores y almacenamiento a intervalos de 1 minuto	Todas

Cuadro 4.9: Herramientas empleadas para la recolección del *dataset* primario.

Herramientas de procesamiento del dataset de validación

Para el procesamiento de las trazas del *dataset* MAWI (Sección 4.4), distribuidas en formato `.pcap`, se empleará `tshark` (la utilidad de línea de comandos de Wireshark) y/o `tstat`, herramientas especializadas en el análisis de capturas de tráfico, con el fin de realizar la agregación temporal descrita en la Sección 4.5 y derivar de ellas las métricas equivalentes a las obtenidas de forma directa en el *dataset* primario (Tabla 4.3).

Entorno de desarrollo y bibliotecas de *Machine Learning*

Todo el desarrollo del prototipo se realizará en el lenguaje de programación **Python**, dado su ecosistema maduro de bibliotecas especializadas en ciencia de datos y aprendizaje automático. La Tabla 4.10 detalla la biblioteca específica que se empleará para la implementación de cada uno de los cinco modelos justificados en la Sección 4.6, así como para las etapas de preprocesamiento y evaluación.

Etapa / Modelo	Biblioteca
Manipulación de datos	`pandas`, `numpy`
Preprocesamiento (Sección 4.5)	`scikit-learn` (`preprocessing`)
ARIMA	`statsmodels`
Random Forest	`scikit-learn`
LSTM	`TensorFlow` / `Keras`
Métricas de evaluación (Sección 4.7)	`scikit-learn` (`metrics`)
Visualización de resultados	`matplotlib`, `seaborn`

Cuadro 4.10: Bibliotecas de Python empleadas para la implementación de cada etapa metodológica.

Entorno de cómputo

En estricta congruencia con lo establecido en los Alcances y Limitaciones de la investigación (Sección 1.4), el desarrollo algorítmico, la sintonización de hiperparámetros y el entrenamiento de los modelos se ejecutaron sobre una estación de trabajo convencional. La plataforma de modelado consistió en una estación de trabajo portátil Dell Precision 7540, provista de un procesador Intel Core i7-9850H (12 núcleos lógicos a 2.60 GHz), 32 GB de memoria RAM y sistema operativo Ubuntu 26.04.1 LTS.

Esta restricción de *hardware* se mantuvo deliberadamente a lo largo de toda la fase experimental para garantizar que la viabilidad del *pipeline* predictivo no dependa de clústeres de cómputo distribuido ni requiera aceleración especializada mediante unidades de procesamiento gráfico (GPU). Con ello se demuestra empíricamente que el sistema resultante es apto para ser desplegado en la infraestructura de propósito general típicamente disponible en una Red Privada Corporativa.

Control de versiones y documentación

Finalmente, el código fuente desarrollado para esta tesis se gestionará mediante un sistema de control de versiones (Git), con el propósito de mantener un historial reproducible de los experimentos realizados. Los fragmentos de código considerados más relevantes para la comprensión del prototipo se incluirán como parte del Apéndice A de este documento, conforme a lo indicado en el índice general de la tesis.

Capítulo 5

Desarrollo e Implementación

Este capítulo documenta cómo se implementó el sistema predictivo a partir de las decisiones metodológicas del Capítulo 4: la extracción y exploración de los datos, incluida la definición del umbral de congestión (Sección 5.1); el preprocesamiento, con limpieza, ventaneo y escalado para homologar las dos fuentes de datos (Sección 4.5); y la configuración y el entrenamiento de los modelos, desde la línea base estadística y el ensamble de *Machine Learning* clásico hasta las redes recurrentes. Los resultados obtenidos con esta implementación se reportan en el Capítulo 6.

5.1. Obtención y exploración de los conjuntos de datos

Como la recolección del *dataset* primario (Sección 4.4) transcurrió en paralelo a la implementación, el prototipo se construyó y validó primero con el *dataset* externo MAWI y después se aplicó al primario. Esta sección describe la obtención y caracterización de ambos.

Extracción de telemetría del *dataset* primario

El diseño original contemplaba SNMP (Sección 4.4), pero la ausencia de IP pública y de presencia física recurrente llevó a usar la herramienta nativa `/interface monitor-traffic` de RouterOS, ejecutada localmente con el Programador de Tareas (*Scheduler*) y complementada con sondeos ICMP activos para latencia y pérdida de paquetes. Esta sustitución mantiene la recolección pasiva y no altera el diseño del Capítulo 4. Se registraron utilización, RTT y tasa de pérdida cada minuto, con respaldo local ante cortes de conectividad (Sección 4.8).

Consolidación y auditoría de continuidad

El *dataset* primario quedó con **21 días** de monitoreo y 25 422 observaciones a 1 minuto. La auditoría de continuidad, necesaria porque la extracción dependía del túnel VPN, detectó 22 interrupciones (Tabla 5.1).

Tipo de interrupción	Ocurrencias	Duración típica
Corte prolongado (pérdida de un día completo)	1	$\approx$ 37 horas
Interrupciones menores	21	6 minutos
Cobertura efectiva del periodo	**82.9 %**	

Cuadro 5.1: Resumen de interrupciones de conectividad durante la recolección del *dataset* primario.

El corte prolongado (7 al 9 de septiembre de 2026, con pérdida completa del día 8) se debió a una interrupción de la VPN y del suministro eléctrico en la sede. Se trata como frontera de sesión, de modo que la serie se divide en dos segmentos continuos independientes. Las 21 interrupciones menores, de 6 minutos, son consistentes con reinicios periódicos del enlace; como superan el umbral de 3 muestras fijado en la Sección 4.5, este se amplió a 6 para poder interpolarlas linealmente. La cobertura del 82.9 % es coherente con las limitaciones del monitoreo remoto (Sección 1.4).

Exploración estadística del *dataset* primario

La Tabla 5.2 muestra una fuerte asimetría positiva: la utilización alcanza un máximo de 1.44 %, pero su mediana es 0.00021 % y su media 0.00221 %. El RTT (media de 9.44 ms, desviación de 3.53 ms) indica además que no hay acumulación en los *buffers*. La red opera, por tanto, muy por debajo de su capacidad.

Estadística	Utilización (%)	RTT (ms)	`rx_bps`
Mínimo	0.00007	6.27	720
Media	0.00221	9.44	22 114
Mediana	0.00021	9.24	2 048
Máximo	1.44004	56.71	14 400 420
Desviación estándar	0.03460	3.53	345 973

Cuadro 5.2: Estadísticas descriptivas del *dataset* primario (21 días reales, 25 422 observaciones).

La Figura 5.1 contrasta el tráfico promedio con los picos máximos por hora. El promedio es prácticamente plano, mientras que los picos se concentran a las 15:00 y las 17:00, en línea con la heterogeneidad de la red (Sección 4.4). La Figura 5.2, con eje de frecuencias logarítmico, muestra que la gran mayoría de las observaciones se acumula cerca de cero (más de 10^4 muestras), mientras que la cola larga, hasta 14 Mbps, tiene frecuencias de apenas uno o dos dígitos. Con esta distribución, una función de pérdida como el MSE favorece predecir siempre el estado basal, por lo que el error de regresión por sí solo puede ocultar la incapacidad de detectar ráfagas.

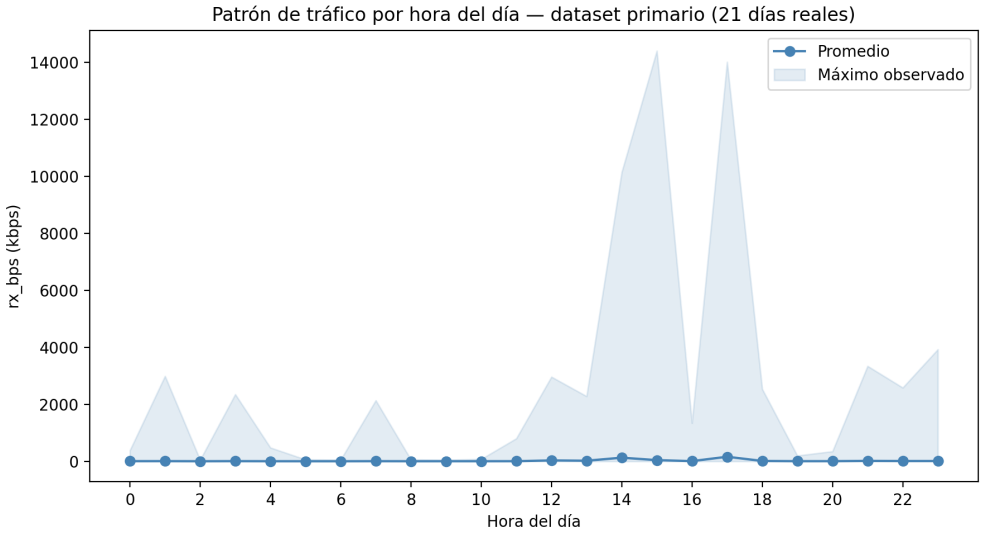


Figura 5.1: Patrón de tráfico promedio y máximo por hora del día, *dataset* primario.

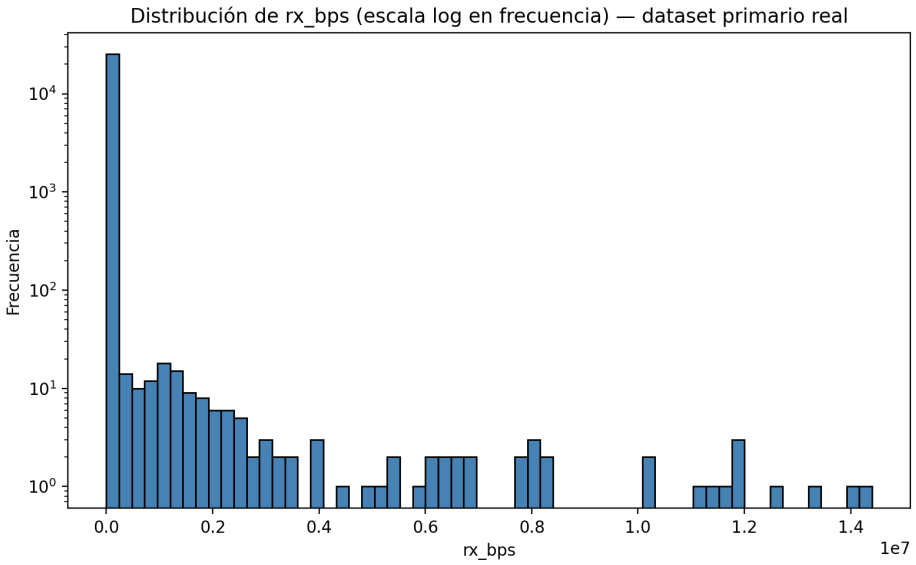


Figura 5.2: Distribución de `rx_bps` (escala logarítmica en frecuencia), *dataset* primario.

Hallazgo: ausencia de congestión natural

La utilización máxima fue de 1.44 % del ancho de banda contratado (1000 Mbps), frente a 32.3 % en el piloto MAWI, por lo que **no se observó ningún evento de congestión real** ($U \geq U_{th} = 0{,}80$). Sin casos positivos no es posible entrenar ni evaluar la formulación de clasificación (Sección 2.2.1), y la estabilidad del RTT indica que el comportamiento no lineal descrito por la Teoría de Colas no aparece en los datos reales. Consultado el comité asesor, se autorizó incorporar datos sintéticos como aumento de datos, con la condición de declarar siempre qué proporción de los datos es real y cuál sintética.

Generación de datos sintéticos calibrados

Se generaron 37 días sintéticos, que junto con los 21 reales equivalen a los dos meses planificados originalmente (Sección 4.4). Su tráfico de fondo se calibró con un remuestreo *bootstrap* de la distribución empírica por hora del día de los datos reales, lo que conserva sus ráfagas sin imponer una forma paramétrica.

Sobre ese fondo se inyectaron 126 eventos de congestión con perfil trapezoidal (ascenso, meseta y descenso) y varianza paramétrica. Duran entre 5 y 29 minutos (media de 17.4), cumpliendo la condición de sostenimiento de la Ecuación 4.1, y alcanzan una utilización pico entre 0.811 y 0.979 (media de 0.890) (Figura 5.3).

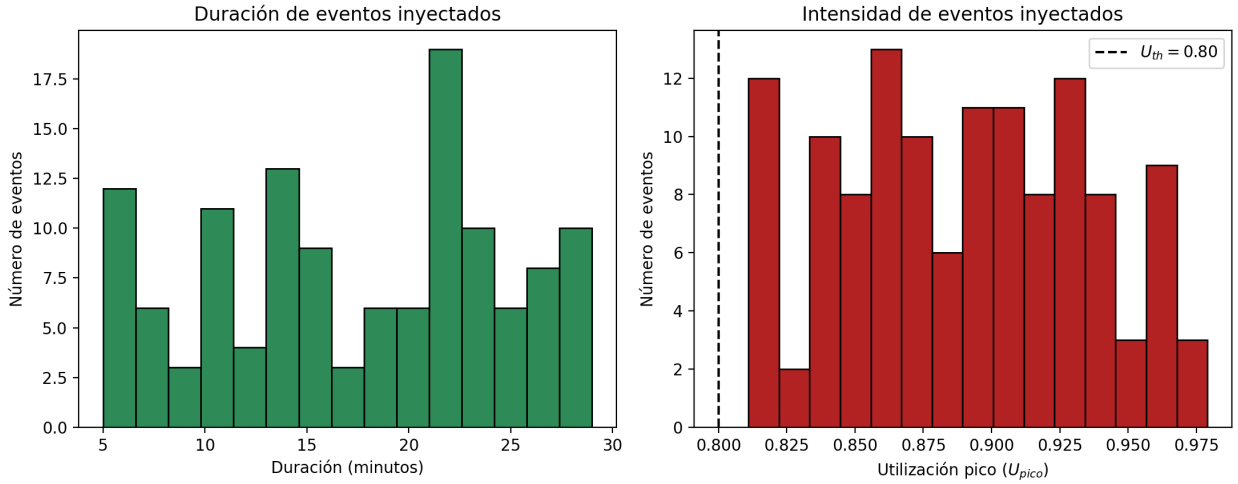


Figura 5.3: Distribución de duración e intensidad de los 126 eventos de congestión sintéticos.

El RTT y la pérdida de paquetes de estos eventos no se generaron con ruido aleatorio, sino que se derivaron de la utilización con el modelo de la Teoría de Colas (Sección 2.1.3): el retraso en cola crece de forma asintótica (proporcional a $\frac{1}{1-\rho}$) y el descarte de paquetes se activa solo cuando $U \approx 0{,}85$. La Figura 5.4 muestra un ejemplo: al cruzar $U = 0{,}80$ en el minuto 10, el RTT y la pérdida se disparan.

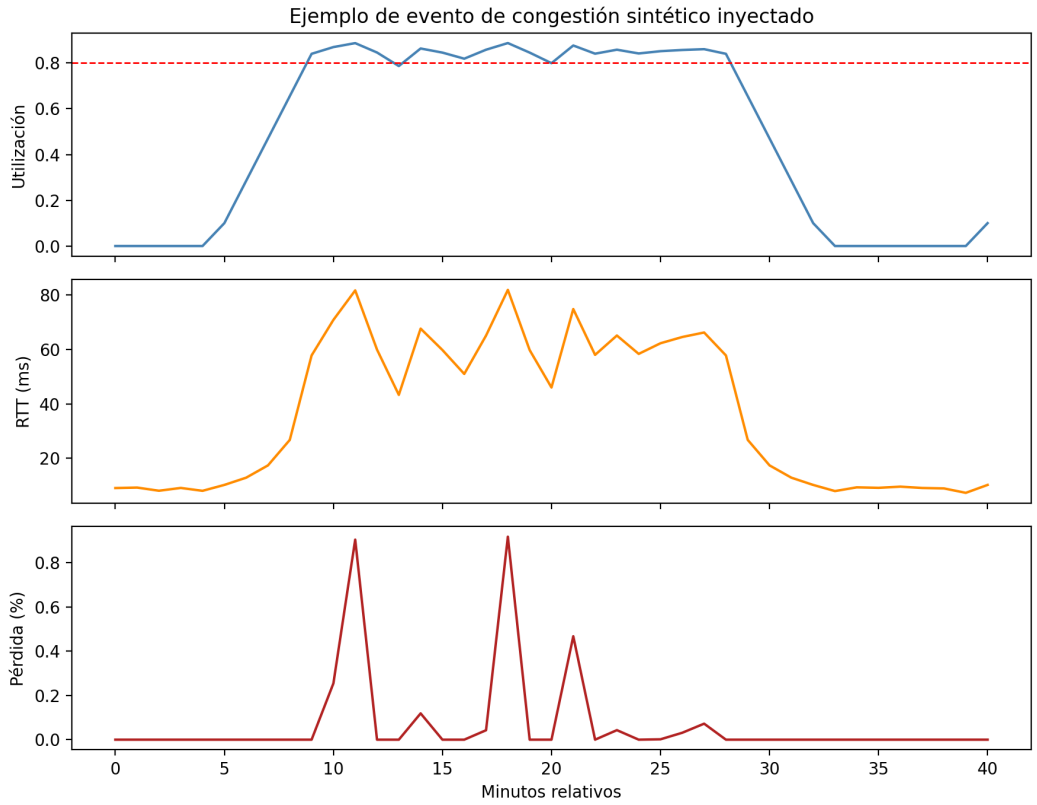


Figura 5.4: Evento sintético inyectado: la utilización determina el RTT y la pérdida de paquetes según la Teoría de Colas.

Diseño de evaluación: preservando la integridad de los datos reales

Para que ningún resultado del Capítulo 6 provenga de una evaluación contaminada con datos sintéticos, se adoptó el siguiente diseño, verificado con comprobaciones automáticas en el código de procesamiento:

- **Entrenamiento:** 60 % cronológico de los datos reales más 60 % de los sintéticos.
- **Regresión** (validación y prueba): solo el 20 %/20 % restante de los datos **reales**.
- **Clasificación** (validación y prueba): solo el 20 %/20 % restante de los datos **sintéticos**, pues la partición real no contiene, por construcción, ningún caso positivo.

Conjunto	Propósito	Naturaleza	Filas	Casos de congestión
Entrenamiento	Ambas tareas	Real + Sintético	46 872	993 (2.12 %)
Val. regresión	Regresión	100 % Real	4 914	0
Prueba regresión	Regresión	100 % Real	4 949	0
Val. clasificación	Clasificación	100 % Sintético	10 622	284 (2.67 %)
Prueba clasificación	Clasificación	100 % Sintético	10 622	275 (2.59 %)

Cuadro 5.3: Composición final del *dataset* primario tras la incorporación de datos sintéticos.

Así, los resultados de regresión reflejan el comportamiento real de la red y los de clasificación, eventos sintéticos calibrados. Esta limitación se discute en la Sección 6.5 y se compensa parcialmente con la validación sobre MAWI reagregado a granularidad oficial (Sección 4.5).

Extracción y decodificación de trazas de validación (MAWI)

Se usaron trazas de tráfico troncal de 2026 del repositorio MAWI, distribuidas como capturas `.pcap.gz` con decenas de millones de paquetes por archivo. `tshark` resultó impráctico por su consumo de memoria al reconstruir flujos, así que se implementó un decodificador propio en Python que interpreta directamente el formato *libpcap* sin decodificar el *payload* y procesa en flujo (*streaming*), agrupando los bytes en ventanas de 1 minuto, la misma del Capítulo 4. Como cada captura diaria dura solo 15 minutos, se procesó un lote secuencial de 120 trazas (30 horas efectivas) para obtener una serie suficientemente larga (Fragmento 5.1, Tabla 5.4).

Listing 5.1: Extracto del decodificador de trazas libpcap en flujo.

```
import struct

fmt_registro = "<IIII" # Formato struct (ts_sec, ts_usec, incl_len,
    orig_len)
ventana_seg = 60        # Agregacion a 1 minuto

while True:
    encabezado_paquete = f.read(16)
    if len(encabezado_paquete) < 16:
        break  # EOF

    ts_sec, ts_usec, incl_len, orig_len = struct.unpack(
        fmt_registro, encabezado_paquete
    )
    timestamp = ts_sec + (ts_usec / 1_000_000)
    ventana = int(timestamp // ventana_seg)
    acumulado[ventana] += orig_len

    # Salto rapido del payload sin saturar RAM
    f.seek(incl_len, 1)
```

Periodo procesado	Paquetes evaluados	Observaciones (1 min)
Lote MAWI (120 trazas)	$\approx 21\,500\,000\,000$	1 800

Cuadro 5.4: Resultados de la extracción de telemetría sobre las trazas MAWI procesadas.

Exploración estadística preliminar

Para probar el *pipeline* antes de concluir la recolección primaria, se exploraron las 1 800 observaciones de MAWI. La utilización se calculó con la definición de la Sección 2.1.3, usando como ancho de banda nominal 11 Gbps, la suma de los dos enlaces monitoreados por *samplepoint-F* desde julio de 2023 (1 Gbps hacia NTTC GIN y 10 Gbps hacia BBIX):

$$U = \frac{\text{Throughput observado (bps)}}{11 \times 10^9 \text{ bps}} \tag{5.1}$$

Estadística	**Throughput (Gbps)**	**Utilización (U)**
Mínimo	0.19	0.017
Media	1.71	0.155
Mediana	1.66	0.151
Máximo	3.55	0.323
Desviación estándar	0.56	0.051

Cuadro 5.5: Estadísticas descriptivas del *dataset* piloto MAWI (1 800 observaciones, 1 minuto).

La utilización máxima, de 32.3 %, nunca alcanza el umbral $U_{th} = 0{,}80$ de la Ecuación 4.1 (Figura 5.5), lo que es coherente con una troncal académica sobredimensionada. Por ello, este subconjunto sirve para ejercitar el *pipeline* en regresión (Sección 2.2.1), pero no aporta positivos para clasificación. Solo durante la codificación y calibración se adoptó un umbral relativo de prueba ($U_{test} \geq 0{,}25$) para verificar la lógica de la matriz de confusión y de la métrica TAE, a la espera del umbral formal de 0.80 sobre el *dataset* primario.

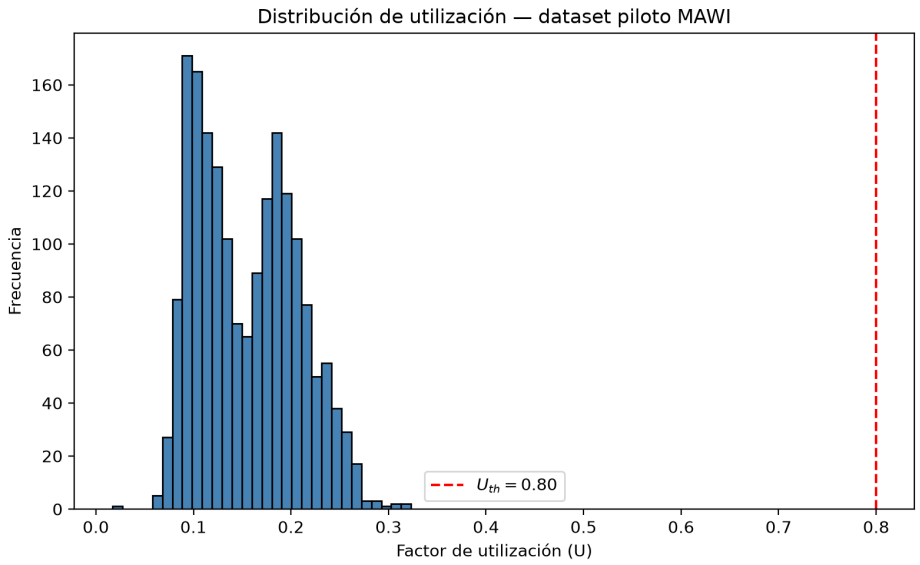


Figura 5.5: Distribución del factor de utilización, *dataset* piloto MAWI.

5.2. Preprocesamiento

Esta etapa computacional tiene como objetivo transformar las series de tiempo crudas en las estructuras tensoriales requeridas por las bibliotecas de *Machine Learning*. A continuación, se documenta la ejecución de este *pipeline* en dos fases: primero, la calibración

algorítmica sobre el *dataset* piloto MAWI; y segundo, la aplicación del preprocesamiento con los parámetros oficiales sobre el *dataset* primario híbrido (real y sintético).

Fase 1: Calibración del *pipeline* sobre el *dataset* piloto MAWI

Sobre el conjunto de 1 800 observaciones a granularidad de 1 minuto descrito en la Sección 5.1, se aplicaron inicialmente las etapas de preprocesamiento establecidas en el Capítulo 4 para validar la lógica del código.

Limpieza y manejo de discontinuidades

Dado que el *dataset* piloto se construyó mediante la concatenación secuencial de un lote de 120 trazas individuales de 15 minutos, las fronteras entre archivos introdujeron discontinuidades temporales. Para garantizar la integridad de los algoritmos autorregresivos, la limpieza se ejecutó empleando las estructuras de datos bidimensionales (`DataFrame`) de la biblioteca `pandas`. Se aplicaron rutinas de interpolación lineal (`.interpolate(method='linear', limit=2)`) para imputar micro-cortes internos, seguidas de la eliminación de observaciones residuales no válidas (`.dropna()`) en los bordes de concatenación. Este proceso de purga redujo el conjunto de datos de 1 800 a un total efectivo de $n = 1\,774$ observaciones continuas y estructuradas.

División cronológica y escalado sin fuga de datos

La división cronológica se programó como una partición dura de la serie ordenada temporalmente $\{x_1, x_2, \ldots, x_n\}$ en tres segmentos contiguos y no traslapados:

$$\begin{aligned} \text{Entrenamiento} &= \{x_1, \ldots, x_{\lfloor 0{,}6n \rfloor}\}, \\ \text{Validación} &= \{x_{\lfloor 0{,}6n \rfloor + 1}, \ldots, x_{\lfloor 0{,}8n \rfloor}\}, \\ \text{Prueba} &= \{x_{\lfloor 0{,}8n \rfloor + 1}, \ldots, x_n\} \end{aligned} \tag{5.2}$$

Para el *dataset* piloto, con $n = 1\,774$, la indexación generó 1 064 observaciones de entrenamiento, 355 de validación y 355 de prueba, proporciones consistentes con el diseño metodológico (Tabla 5.6).

Conjunto	Observaciones	Proporción
Entrenamiento	1 064	60.0 %
Validación	355	20.0 %
Prueba	355	20.0 %

Cuadro 5.6: Composición de la división cronológica del *dataset* piloto MAWI tras el preprocesamiento.

Posterior a la partición, se aplicó la normalización Min-Max (Ecuación 4.4). Desde la perspectiva de ingeniería de *software*, para prevenir el antipatrón de fuga de datos (*data leakage*), este escalado se implementó utilizando la clase `MinMaxScaler` de `scikit-learn`. Se garantizó por diseño que el método `fit_transform()` se ejecutara de manera exclusiva

sobre el conjunto de entrenamiento, mientras que a los conjuntos de validación y prueba se les aplicó únicamente el método `transform()`, forzándolos a mapearse bajo la distribución matemática descubierta en el pasado, sin contaminar el modelo con conocimiento futuro. Para la variable de utilización, la ecuación instanciada operó de la siguiente forma:

$$U_{\text{norm}} = \frac{U - 0{,}017045}{0{,}323168 - 0{,}017045} \tag{5.3}$$

Ventaneo secuencial y conformación de tensores

Para habilitar el aprendizaje de la arquitectura recurrente LSTM, se implementó una función propia de ventana deslizante (*Sliding Window*) mediante la manipulación de arreglos de `NumPy`. Dado el volumen del piloto, se configuró provisionalmente una ventana de observación $w = 15$ minutos y un horizonte de predicción $h = 5$ minutos. La conformación lógica de cada par de entrada/salida se instanció como:

$$X_t = [U(t-15), \ldots, U(t-1)] \longrightarrow y_t = U(t+5) \tag{5.4}$$

El resultado es la redimensión de los datos en un tensor tridimensional con la forma `(muestras, pasos_de_tiempo, caracteristicas)`, requisito ineludible para la capa de entrada (*Input Layer*) del *framework* `Keras/TensorFlow`. Este proceso generó 1 044 secuencias para entrenamiento, 335 para validación y 335 para prueba.

Fase 2: Ejecución oficial sobre el *dataset* primario

Una vez validado el código, el preprocesamiento se aplicó sobre el *dataset* primario (datos reales y sintéticos), introduciendo los parámetros oficiales y el manejo de sesiones exigido por las interrupciones operativas.

Detección de sesiones continuas

El *dataset* primario presenta dos fuentes de discontinuidad temporal: las interrupciones de conectividad VPN reales y la frontera cronológica entre los datos reales y los sintéticos inyectados. Aplicando un criterio de segmentación estricto (cortes superiores a 5 minutos), se identificaron 10 sesiones dentro del conjunto de entrenamiento (Tabla 5.7).

Sesión	Filas	Naturaleza	Inicio	Fin
10	31 934	Sintética	2026-09-20 00:29	2026-10-12 04:42
0	6 137	Real	2026-08-29 17:30	2026-09-02 23:50
5	1 401	Real	2026-09-07 00:29	2026-09-07 23:49
2	1 401	Real	2026-09-04 00:29	2026-09-04 23:49
4	1 401	Real	2026-09-06 00:29	2026-09-06 23:49
3	1 401	Real	2026-09-05 00:29	2026-09-05 23:49
7	1 401	Real	2026-09-10 00:29	2026-09-10 23:49
9	658	Real	2026-09-12 00:29	2026-09-12 11:26
6	611	Real	2026-09-09 13:39	2026-09-09 23:49
1	527	Real	2026-09-03 15:03	2026-09-03 23:49

Cuadro 5.7: Sesiones continuas detectadas en el conjunto de entrenamiento del *dataset* primario.

Esta segmentación resulta indispensable para calcular las variables de rezago, evitando que el algoritmo tome datos de una sesión distinta a través de un "salto en el tiempo". Sobre cada una de estas 10 sesiones se aplicó independientemente la interpolación lineal de huecos cortos (con el límite flexibilizado de 6 muestras, justificado en la Sección 5.1). Tras la limpieza y eliminación de observaciones incompletas en los bordes, el conjunto de entrenamiento quedó compuesto por 46 872 observaciones.

Normalización sin fuga de datos entre las cinco particiones

A diferencia del piloto, el diseño de evaluación dual establecido requiere que la normalización Min-Max se aplique de forma estructurada sobre **cinco** particiones (entrenamiento unificado, y pares de validación/prueba separados para regresión y clasificación). Para prevenir estrictamente la fuga de información temporal (*data leakage*), los parámetros de escalamiento escalar (mínimo y máximo) se calcularon de manera exclusiva sobre la matriz de entrenamiento y se proyectaron, sin recálculo alguno, sobre las cuatro particiones de evaluación restantes.

La Figura 5.6 expone la matriz de histogramas resultante de este proceso, ofreciendo una comprobación visual crítica de la asimetría del diseño experimental. La fila superior (paneles en gris) ilustra las particiones dominadas por el tráfico de fondo: mientras que el conjunto de entrenamiento (panel superior izquierdo) abarca todo el espectro [0, 1] debido a la inyección de anomalías, las particiones de regresión (100 % reales, paneles superior central y derecho) colapsan su densidad de probabilidad íntegramente hacia el extremo izquierdo, confirmando que la red operó muy por debajo del umbral de congestión $U_{th} = 0{,}80$ (línea roja discontinua). En contraste, la fila inferior (paneles en naranja) corrobora la idoneidad de los conjuntos de clasificación (100 % sintéticos): aquí se aprecia cómo la masa anómala inyectada franquea exitosamente el umbral, dotando matemáticamente a la evaluación de los eventos necesarios para probar la capacidad de detección.

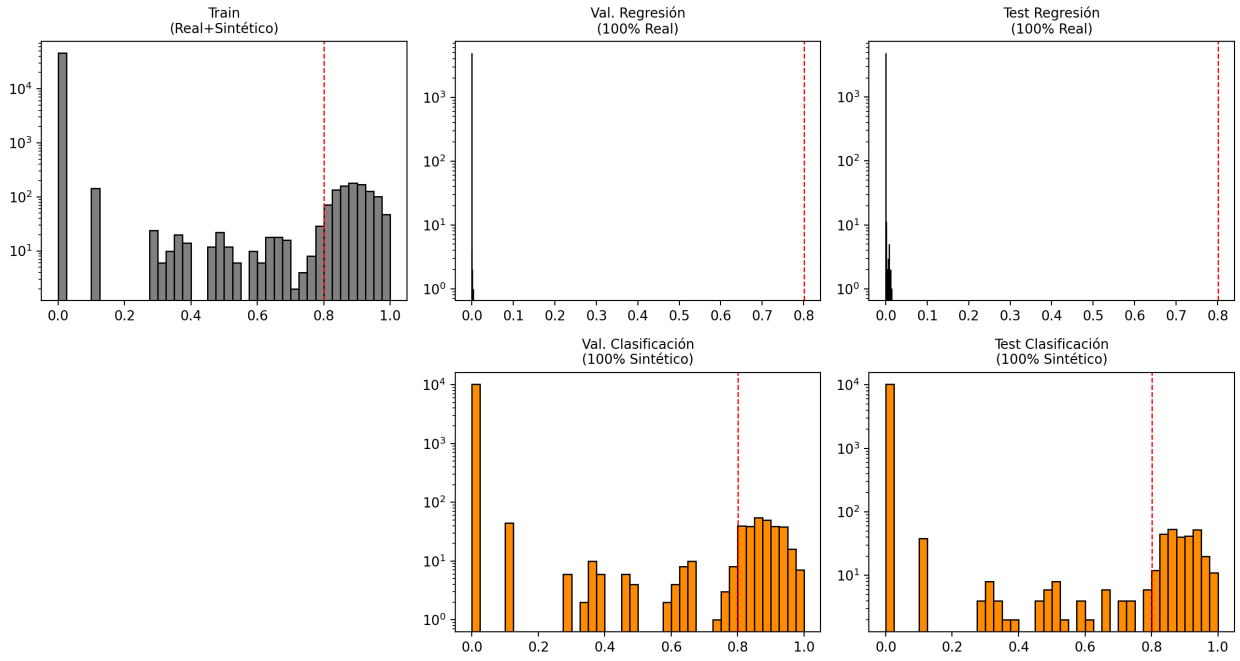


Figura 5.6: Matriz de distribución de utilización normalizada: contraste entre el espectro del entrenamiento (arriba izq.), la contención del tráfico real (arriba centro/der.) y la masa anómala inyectada (abajo).

La Tabla 5.8 resume los rangos topológicos resultantes. Es imperativo señalar que la existencia de máximos ínfimos en las particiones de regresión (apenas 0.0144) o de mínimos fraccionalmente negativos no constituye un error de procesamiento, sino la consecuencia analítica directa y deseada de haber congelado los escalares en el conjunto de entrenamiento, garantizando así una evaluación completamente ciega a magnitudes no observadas en el pasado.

Conjunto	**Mínimo**	**Máximo**	**Observaciones**
Entrenamiento	0.000000	1.000000	46 872
Validación (regresión)	0.0000001	0.002992	4 914
Prueba (regresión)	0.0000001	0.014414	4 949
Validación (clasificación)	−0.0000001	1.000000	10 622
Prueba (clasificación)	−0.0000001	1.000000	10 622

Cuadro 5.8: Rango de utilización normalizada por partición, *dataset* primario. Los valores atípicos certifican la ausencia de fuga de información metodológica.

Ventaneo secuencial oficial

Finalmente, se aplicó la granularidad y los parámetros **oficiales** establecidos en el Capítulo 4 para el *dataset* primario: ventana de observación $w = 30$ minutos y horizonte de predicción $h = 5$ minutos. La variable objetivo de clasificación se definió como una variable binaria $\mathbb{1}[U(t+h) \geq U_{th}]$.

$$X_t = [U(t-30), \ldots, U(t-1)] \longrightarrow y_t = U(t+5) \tag{5.5}$$

El proceso, respetando las fronteras de sesión, generó 46 582 secuencias de entrenamiento, 4 796 y 4 833 secuencias para validación y prueba de regresión respectivamente, y 10 593 secuencias para cada partición de clasificación, dejando los tensores listos para el entrenamiento de los modelos predictivos documentados a continuación.

5.3. Construcción del modelo ARIMA

Conforme a la Sección 4.6, ARIMA se implementó con `statsmodels` como línea base estadística. El procedimiento fue el mismo en ambos conjuntos de datos: primero se calibró sobre el piloto MAWI y después se aplicó al *dataset* primario.

Identificación del orden

Para fijar el orden de diferenciación d se aplicó la prueba de Dickey-Fuller Aumentada (`stattools.adfuller`) sobre la serie de utilización del conjunto de entrenamiento: si no se rechazaba la no estacionariedad ($p > 0{,}05$), la serie se diferenciaba (Sección 2.2.2). Luego se evaluaron las dieciséis combinaciones (p, d, q) con $p, q \in \{0, 1, 2, 3\}$ y se seleccionó la de menor Criterio de Información de Akaike (AIC). En MAWI la serie requirió $d = 1$ y el orden seleccionado fue ARIMA(1,1,1). En el *dataset* primario, el modelo se ajustó sobre la sesión sintética continua más larga del conjunto de entrenamiento (31 934 observaciones, Tabla 5.7), que contiene tanto el régimen normal como los 126 eventos de congestión inyectados. La serie resultó estacionaria ($d = 0$) y se seleccionó ARIMA(3,0,3). Los estadísticos de la prueba, la búsqueda por AIC, los coeficientes y el diagnóstico de residuos se reportan en el Capítulo 6.

Esquema de evaluación

Para regresión se empleó pronóstico progresivo (*walk-forward validation*) con horizonte $h = 5$: en cada instante t se predice el valor en $t + h$ y, a continuación, el valor real observado se incorpora al modelo antes de la siguiente predicción, emulando un despliegue en tiempo real. En el *dataset* primario la evaluación se hizo por bloques (Sección 4.7) y solo sobre las particiones reales de validación y prueba (Sección 5.1). Se omitió el MAPE porque la utilización real cercana a cero hace inestable su denominador (Sección 4.7).

Para clasificación, las predicciones continuas se binarizaron con el umbral de prueba $U_{test} \geq 0{,}25$ en MAWI y con $U_{th} = 0{,}80$ en el *dataset* primario, este último sobre las particiones sintéticas aisladas. Una alerta cuenta como verdadero positivo solo si la predicción supera el umbral antes del evento real. Las métricas de clasificación y el TAE se calcularon según la Sección 4.7, y los resultados de ambas tareas se presentan en el Capítulo 6.

Parámetro	Configuración
Biblioteca	`statsmodels`
Orden de diferenciación d	Prueba ADF ($\alpha = 0{,}05$)
Búsqueda de (p, q)	$p, q \in \{0, 1, 2, 3\}$ (16 combinaciones)
Criterio de selección	Mínimo AIC
Horizonte h	5 pasos
Evaluación de regresión	Pronóstico progresivo por bloques
Umbral de clasificación	$U_{test} = 0{,}25$ (MAWI); $U_{th} = 0{,}80$ (primario)

Cuadro 5.9: Configuración del modelo ARIMA.

5.4. Construcción del modelo *Random Forest*

Conforme a la Sección 4.6, se implementó *Random Forest* como representante del *Machine Learning* clásico con `RandomForestRegressor` y su contraparte `RandomForestClassifier` de `scikit-learn`. Como en ARIMA, el modelo se calibró primero sobre MAWI y luego se reentrenó sobre el *dataset* primario con los mismos hiperparámetros, usando sus 46 872 observaciones de entrenamiento (Sección 5.1).

Preparación de las entradas

El tensor tridimensional del preprocesamiento (Sección 5.2) se aplanó a la entrada tabular que requieren los algoritmos clásicos. Cada observación se representa con cinco rezagos (U_{lag1} a U_{lag5}) y la media de la ventana ($U_{\text{media_movil}}$) como característica explícita. A diferencia de ARIMA, que opera sobre su propia serie y sus residuos, el ensamble mapea de forma no lineal todo el vector de entrada contra el objetivo. La utilización se escaló con Min-Max, por lo que sus métricas se reportan en escala normalizada, a diferencia de ARIMA, que se mantuvo en escala original.

Configuración y esquema de evaluación

La Tabla 5.10 resume la configuración, idéntica en ambos conjuntos de datos.

Parámetro	Configuración
Biblioteca	`scikit-learn`
Número de árboles	200
Profundidad máxima	10 (`max_depth=10`)
Entradas	5 rezagos y media móvil de la ventana
Escalado	Min-Max
Evaluación de regresión	Particiones de validación y prueba (reales en el primario)
Umbral de clasificación	$U_{test} = 0{,}25$ (MAWI); $U_{th} = 0{,}80$ (primario)
Importancia de variables	Disminución media de impureza

Cuadro 5.10: Configuración del modelo *Random Forest*.

La evaluación sigue el diseño de la Sección 5.1: la regresión se evalúa sobre las particiones reales y la clasificación, sobre las sintéticas aisladas, binarizando las predicciones con el umbral correspondiente a cada conjunto de datos. La importancia de las variables se calculó a partir de la disminución media de impureza que aporta cada entrada en los árboles. Los resultados de ambas tareas se presentan en la Sección 6.

5.5. Construcción del modelo LSTM

Conforme a la Sección 4.6, se implementó una red LSTM como representante del *Deep Learning* con `Keras/TensorFlow`. A diferencia de *Random Forest*, la red consume directamente los tensores tridimensionales de la ventana deslizante (Sección 5.2), sin aplanarlos, para aprovechar el orden temporal de la secuencia. Como en los modelos anteriores, se calibró primero sobre MAWI y luego se reentrenó sobre el *dataset* primario con la misma arquitectura y el mismo criterio de parada, usando sus 46 582 secuencias de entrenamiento.

Arquitectura y entrenamiento

La red es secuencial: una capa LSTM de 32 unidades con activación tangente hiperbólica, una capa densa de 16 unidades con ReLU y una neurona de salida lineal para la regresión de la utilización, con 4 897 parámetros entrenables. Se minimizó el MSE con Adam y lotes de 16 secuencias, durante un máximo de 100 épocas. Como regularización se usó detención temprana (*Early Stopping*) sobre la pérdida de validación, con paciencia de 10 épocas y restauración de los mejores pesos. En el *dataset* primario, esa partición de validación es la de regresión, es decir, 100 % tráfico real (Sección 5.1). La Tabla 5.11 resume la configuración.

Parámetro	Configuración
Biblioteca	`Keras/TensorFlow`
Capas	LSTM (32, tanh), Dense (16, ReLU), Dense (1, lineal)
Parámetros entrenables	4 897
Entrada	Tensor 3D de la ventana deslizante
Función de pérdida y optimizador	MSE, Adam
Tamaño de lote	16
Épocas máximas	100
Detención temprana	Paciencia de 10 épocas; restauración de mejores pesos
Escalado	Min-Max
Umbral de clasificación	$U_{test} = 0{,}25$ (MAWI); $U_{th} = 0{,}80$ (primario)

Cuadro 5.11: Configuración del modelo LSTM.

Esquema de evaluación

La evaluación sigue el diseño de la Sección 5.1: la regresión se mide sobre las particiones reales y la clasificación, sobre las sintéticas aisladas, binarizando la salida continua con el umbral de cada conjunto de datos. Las curvas de aprendizaje, las métricas de ambas tareas y la comparación con ARIMA y *Random Forest* se presentan en el Capítulo 6.

Capítulo 6

Resultados y Discusión

El presente capítulo consolida y analiza los hallazgos empíricos obtenidos al aplicar los tres modelos predictivos construidos en el Capítulo 5 —ARIMA, *Random Forest* y LSTM— sobre el *dataset* primario definitivo: 21 días de recolección real en la red privada PYME bajo estudio, complementados mediante la estrategia de aumento de datos sintéticos calibrados descrita en la Sección 5.1. A diferencia del capítulo anterior, cuyo propósito fue documentar la construcción, ejecución y correcta convergencia de cada modelo, este capítulo se concentra en la interpretación crítica de sus resultados a la luz de las preguntas de investigación planteadas en el Capítulo 1 y de la evidencia recabada en la revisión de la literatura (Capítulo 3).

Es indispensable reiterar, antes de proceder, el diseño de evaluación dual bajo el cual deben leerse los resultados de este capítulo (Sección 5.1, Tabla 5.3). La tarea de **regresión** —predicción continua del factor de utilización $U(t+h)$— se evalúa exclusivamente sobre datos **reales** de la infraestructura estudiada, nunca vistos durante el entrenamiento, y por tanto sus resultados constituyen evidencia directa del comportamiento de los modelos ante el tráfico genuino de la red. La tarea de **clasificación** —predicción binaria de un evento de congestión, $U(t+h) \geq U_{th}$—, en cambio, se evalúa sobre una partición de datos **sintéticos** aislada del entrenamiento, dado que el régimen de tráfico real observado durante la ventana de recolección jamás alcanzó el umbral de congestión $U_{th} = 0{,}80$.

Esta asimetría entre la naturaleza de los datos de evaluación de una y otra tarea no es una limitación oculta, sino una decisión metodológica explícita, consultada y autorizada por el comité asesor de esta investigación. Este rigor se mantiene como un hilo conductor a lo largo de las secciones siguientes: cada resultado se presenta identificando con precisión si proviene de la evaluación sobre datos reales o sintéticos, evitando cualquier ambigüedad que pudiera sugerir una capacidad de detección de congestión verificada sobre datos genuinos que, en la práctica, la naturaleza del tráfico observado no permitió constatar.

Las Secciones 6.1 a 6.3 recapitulan, para cada modelo, los resultados de evaluación ya introducidos en el Capítulo 5, sintetizando sus métricas principales sin reiterar el detalle experimental. La Sección 6.4 presenta la comparativa integral entre los tres modelos, para ambas tareas, como fundamento cuantitativo de la discusión. Finalmente, la Sección 6.5 interpreta el conjunto de hallazgos a la luz del problema de investigación original, contrasta los resultados obtenidos contra la evidencia de la literatura revisada, y discute con honestidad las limitaciones que se desprenden del diseño experimental adoptado, sentando las bases para el trabajo futuro.

6.1. Resultados de ARIMA

Esta sección reporta los resultados del modelo ARIMA, configurado según la Sección 5.3: su desempeño en la fase piloto con MAWI, su evaluación de regresión y clasificación sobre el *dataset* primario y una validación exploratoria de generalización frente a MAWI reagregado a granularidad oficial. Cada resultado se acompaña de su lectura inmediata. La comparación con los demás modelos se presenta en la Sección 6.4 y la discusión conjunta en la Sección 6.5.

Identificación y ajuste del modelo

La Tabla 6.1 resume la identificación del orden en ambos conjuntos de datos. En el piloto, la prueba de Dickey-Fuller Aumentada no permitió rechazar la hipótesis nula de no estacionariedad ($p = 0{,}4315 > 0{,}05$), por lo que se aplicó una diferenciación de primer orden ($d = 1$). En el *dataset* primario, la serie de entrenamiento resultó estacionaria sin diferenciar (ADF $= -23{,}6571$, $p < 0{,}0001$), a diferencia del piloto.

Resultado	**Piloto (MAWI)**	**Primario**
Estadístico ADF	−1,6992	−23,6571
Valor p (ADF)	0.4315	$< 0{,}0001$
Orden de diferenciación d	1	0
Orden seleccionado (p, d, q)	(1,1,1)	(3,0,3)
AIC	−5 656,68	−111 594,72
Valor p (Ljung-Box, residuos)	0.92	0.98
Observaciones de ajuste	1 064	31 934

Cuadro 6.1: Identificación y diagnóstico del ajuste de ARIMA en cada conjunto de datos.

El cambio de $d = 1$ a $d = 0$ refleja la diferencia entre ambas series: la de MAWI es un tráfico troncal con deriva lenta, mientras que la del primario se comporta como un proceso que regresa a su media, con picos transitorios debidos a los eventos inyectados. En ambos casos la prueba de Ljung-Box ($p = 0{,}92$ y $p = 0{,}98$) no rechaza la ausencia de autocorrelación residual, por lo que el modelo captura la estructura lineal de la serie.

La Tabla 6.2 muestra los cinco mejores órdenes de la búsqueda en el piloto.

Orden (p, d, q)	**AIC**
ARIMA(1,1,1)	−5 656,68
ARIMA(2,1,1)	−5 654,95
ARIMA(0,1,3)	−5 653,74
ARIMA(2,1,2)	−5 653,04
ARIMA(3,1,1)	−5 653,03

Cuadro 6.2: Órdenes ARIMA con menor AIC, *dataset* piloto MAWI.

El ARIMA(1,1,1) obtuvo el menor AIC, aunque la diferencia con el quinto es de solo 3.65 unidades. Los cinco modelos son, por tanto, prácticamente equivalentes en ajuste, y el criterio de mínimo AIC favorece además al más parsimonioso.

Los coeficientes estimados instancian la formulación general de la Ecuación 2.10. Para el piloto se obtuvo la Ecuación 6.1, con ambos coeficientes significativos ($p < 0{,}001$):

$$(1 - 0{,}4305\,L)(1 - L)\,U_t = (1 - 0{,}8280\,L)\,\varepsilon_t \tag{6.1}$$

El coeficiente autorregresivo (0.4305) indica una dependencia moderada del incremento anterior, y el de media móvil (0.8280), un peso importante del error de predicción previo, lo que ya anticipa una predicción suavizada. Para el *dataset* primario se obtuvo la Ecuación 6.2, donde μ es la media del proceso. Todos sus coeficientes fueron significativos ($p < 0{,}01$):

$$\begin{gathered}(1 - 2{,}3103\,L + 1{,}7123\,L^2 - 0{,}3971\,L^3)\,(U_t - \mu) = (1 - 1{,}0847\,L + 0{,}1825\,L^2 - 0{,}0209\,L^3)\,\varepsilon_t, \\ \mu = 0{,}0324\end{gathered} \tag{6.2}$$

La suma de los coeficientes autorregresivos del ARIMA(3,0,3) es 0.995, es decir, un proceso muy persistente que regresa lentamente a su media, y esa media ($\mu = 0{,}0324$) está elevada por los eventos de congestión inyectados en la sesión de ajuste. Ambas características resultan relevantes para entender los resultados de regresión del primario. Además, la prueba de Jarque-Bera rechazó la normalidad de los residuos, con una curtosis de 249.4, atribuible a los cambios abruptos de régimen que introducen los eventos sintéticos: generan residuos ocasionales de magnitud muy superior a la del comportamiento típico de la serie.

Fase piloto: regresión

La Tabla 6.3 presenta las métricas del pronóstico progresivo sobre MAWI, y la Figura 6.1 contrasta la predicción con la serie real en prueba.

Conjunto	**RMSE**	**MAE**	**MAPE**
Validación	0.0265	0.0204	10.61 %
Prueba	0.0282	0.0221	10.80 %

Cuadro 6.3: Métricas de regresión de ARIMA(1,1,1), *dataset* piloto MAWI.

El error es similar en ambas particiones (MAPE de 10.61 % y 10.80 %, con un aumento de apenas 0.19 puntos porcentuales). Esta estabilidad entre validación y prueba indica que el modelo no sufre una degradación apreciable al pasar a datos no vistos, lo que es un indicio de ausencia de sobreajuste. El RMSE de prueba (0.0282) queda además por debajo de la desviación estándar de la utilización en MAWI (0.051, Tabla 5.5).

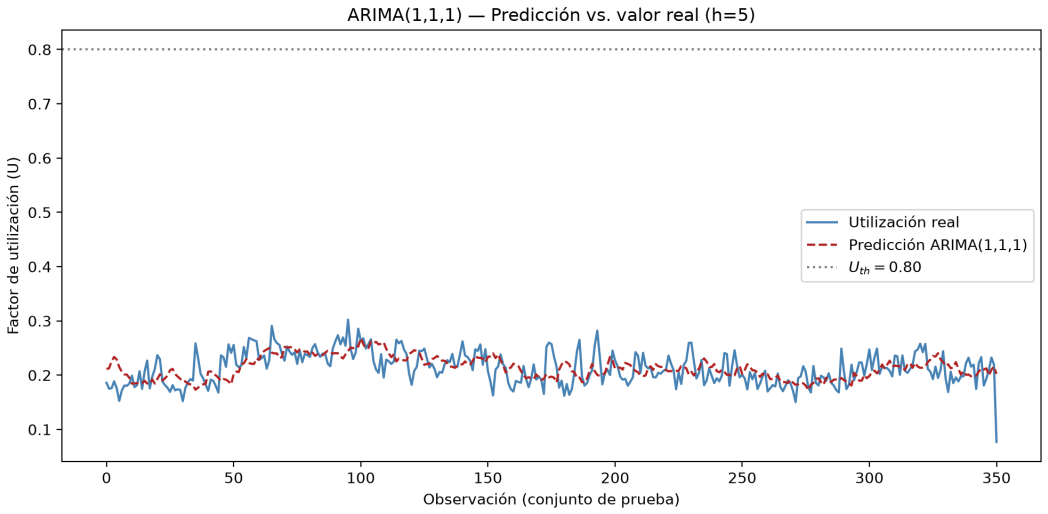


Figura 6.1: ARIMA(1,1,1): pronóstico progresivo ($h = 5$) frente al valor real, partición de prueba del piloto MAWI.

En la Figura 6.1, la predicción (línea roja discontinua) funciona como una versión suavizada de la utilización real (línea azul): la serie observada presenta oscilaciones de corto plazo con aspecto de ruido, mientras que el modelo traza únicamente la tendencia central. Este suavizado explica el error moderado y estable en regresión, pero tiene una contraparte: la inercia del modelo lo hace lento ante cambios súbitos, lo que se refleja en la clasificación.

Fase piloto: clasificación

Con el umbral oficial $U_{th} = 0{,}80$ la clasificación no es evaluable en MAWI, porque su utilización máxima es de 32.3 % (Sección 5.1). Para ejercitar el código de evaluación se usó el umbral relativo de prueba $U_{test} = 0{,}25$. La Tabla 6.4 muestra el desempeño como clasificador de alertas tempranas sobre la partición de prueba.

Métrica ($U_{test} \geq 0{,}25$)	**Prueba**
Precisión	64.2 %
Recall	41.5 %
F1-score	50.4 %
Tiempo de Anticipación Efectivo (TAE) promedio	**1.2 minutos**

Cuadro 6.4: Clasificación y alertamiento (TAE) de ARIMA(1,1,1), fase piloto.

La precisión de 64.2 % indica que cerca de dos de cada tres alertas corresponden a un evento real, mientras que el *recall* de 41.5 % indica que el modelo deja pasar casi seis de cada diez eventos. El TAE de 1.2 minutos queda muy por debajo del horizonte objetivo de 5 minutos: aun cuando acierta, el modelo alerta con poca antelación, coherente con el suavizado observado en regresión. Estas cifras validan el funcionamiento del *pipeline*, pero no son comparables con las del *dataset* primario, que usan otro umbral y otros datos.

Dataset primario: regresión

La Tabla 6.5 presenta el error de ARIMA(3,0,3) sobre las particiones reales, en escala original de utilización. No se reporta MAPE, porque la utilización real, cercana a cero, vuelve inestable su denominador (Sección 4.7).

Conjunto	RMSE	MAE
Validación (real)	0.046045	0.015110
Prueba (real)	0.045872	0.015059

Cuadro 6.5: Métricas de regresión de ARIMA(3,0,3), *dataset* primario (escala original de utilización).

El error es prácticamente idéntico en validación y prueba (RMSE de 0.0460 y 0.0459), lo que indica que su origen es sistemático y no una pérdida de generalización entre particiones. Sin embargo, resulta aproximadamente un 63 % mayor que el del piloto (0.0459 frente a 0.0282), a pesar de que la utilización real del primario es mucho menor (máximo de 1.44 %, Tabla 5.2) que la de MAWI (máximo de 32.3 %). Un RMSE de 0.0459 supera incluso la utilización máxima observada (0.0144), de modo que el error no puede deberse a la dificultad de seguir la señal, sino a un desajuste de nivel.

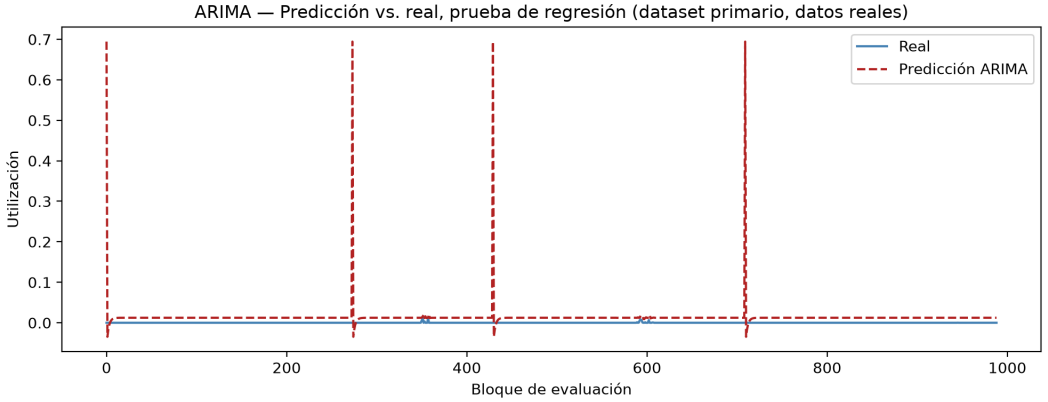


Figura 6.2: ARIMA(3,0,3): predicción frente al valor real, prueba de regresión (*dataset* primario, datos reales).

La Figura 6.2 permite identificar dos fenómenos. El primero es un sesgo sistemático de sobreestimación: la predicción de fondo (línea roja discontinua) se mantiene por encima de la señal real, que es plana y cercana a cero (línea azul). Esto es consistente con la media del proceso ajustado ($\mu = 0{,}0324$), elevada por los eventos sintéticos de la sesión de ajuste, que rozan la saturación (hasta 0.98), mientras que el comportamiento típico del canal real es muy inferior.

El segundo fenómeno son oscilaciones transitorias, con picos agudos de hasta cerca de 0.7 y valles que perforan el eje negativo, espaciadas a intervalos regulares. Esta regularidad coincide con el inicio de cada bloque del pronóstico progresivo, donde el modelo reinicia su estado autorregresivo y de media móvil asimilando de golpe las observaciones previas.

Con coeficientes autorregresivos de gran magnitud y suma cercana a 1, un pequeño desajuste inicial tarda en disiparse. Por eso las oscilaciones se interpretan como un artefacto del esquema de evaluación por bloques y no como un error de cálculo, y elevan el RMSE por encima de lo que reflejaría el comportamiento en régimen estable.

Dataset primario: clasificación

La Tabla 6.6 reporta la clasificación sobre las particiones sintéticas aisladas, con $U_{th} = 0{,}80$.

Métrica	**Validación**	**Prueba**
Exactitud	0.9741	0.9750
Precisión	0.6667	1.0000
Recall	0.0357	0.0185
F1-score	0.0678	0.0364
AUC-ROC	0.8905	0.8939

Cuadro 6.6: Métricas de clasificación de ARIMA(3,0,3), $U_{th} = 0{,}80$, datos sintéticos aislados.

La tabla contiene una aparente contradicción: una exactitud de 0.975 junto con un *recall* de 0.0185. Se explica por el desbalance de clases, pues los casos de congestión son una fracción mínima (cerca del 2.6 %), de modo que un modelo que casi nunca alerta acierta en la inmensa mayoría de las observaciones y obtiene una exactitud alta. Por eso la exactitud no es informativa en este problema y las métricas relevantes son el *recall* y el *F1-score*, ambos muy bajos. La precisión de 1.0 en prueba se apoya en un único acierto, por lo que dice poco por sí sola. En contraste, el AUC-ROC es alto (0.89), lo que indica que la predicción continua sí ordena los instantes de congestión por encima de los normales, aunque no los lleve a cruzar el umbral de decisión.

La Figura 6.3 presenta estas métricas de forma comparativa entre validación y prueba.

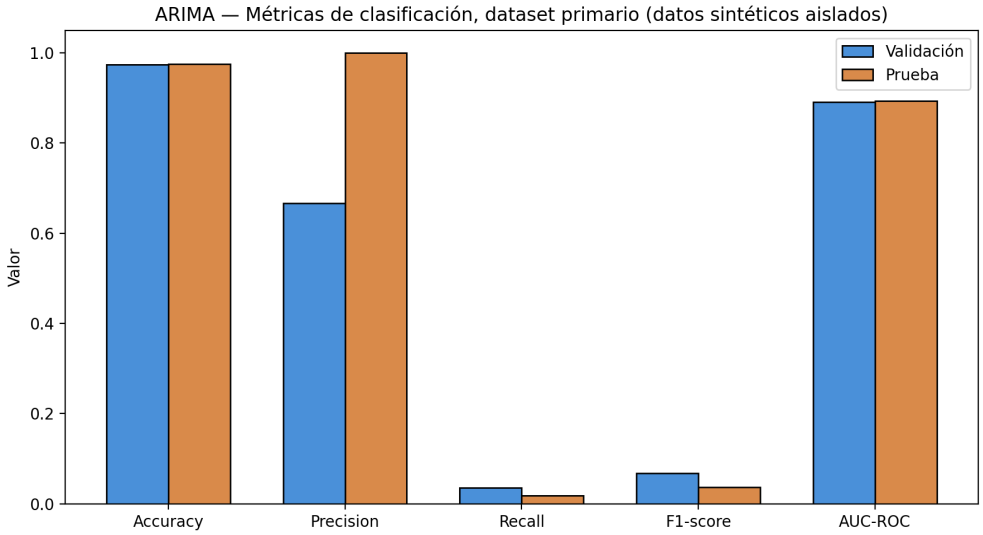


Figura 6.3: Comparación de métricas de clasificación, validación frente a prueba, ARIMA sobre el *dataset* primario.

La figura confirma que el patrón es el mismo en ambas particiones: exactitud y AUC-ROC elevados, y *recall* y *F1-score* cercanos a cero. La Ecuación 6.3 presenta la matriz de confusión sobre la partición de prueba.

$$\begin{pmatrix} \text{VN} = 2070 & \text{FP} = 0 \\ \text{FN} = 53 & \text{VP} = 1 \end{pmatrix} \tag{6.3}$$

Esta matriz describe el comportamiento del modelo frente al desbalance de clases:

- **Verdaderos negativos (VN=2070):** el modelo reconoce correctamente los periodos de operación normal.
- **Falsos positivos (FP=0):** no emitió ninguna alerta injustificada, lo que explica la precisión de 100 %.
- **Falsos negativos (FN=53):** es el fallo crítico. De los 54 casos positivos, el modelo no alertó en 53, es decir, la red quedaría sin aviso ante casi todos los episodios de congestión.
- **Verdaderos positivos (VP=1):** solo se detectó un caso de congestión en toda la partición de prueba.

En términos operativos, un sistema que nunca da falsas alarmas pero que no avisa en el 98 % de las congestiones no resulta útil como mecanismo de alerta temprana.

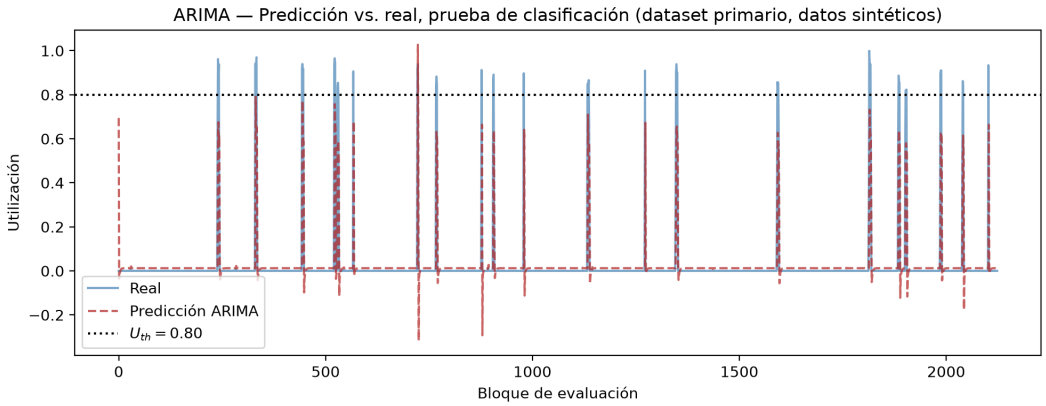


Figura 6.4: ARIMA(3,0,3): predicción frente al valor real y umbral $U_{th} = 0{,}80$, prueba de clasificación (*dataset* primario, datos sintéticos aislados).

La Figura 6.4 muestra la causa de los falsos negativos. La predicción (línea roja discontinua) intenta seguir la trayectoria de los picos de utilización, pero con una amortiguación marcada: el componente de media móvil empuja el pronóstico hacia la tendencia central, y el modelo subestima sistemáticamente la magnitud de las ráfagas. Como consecuencia, las predicciones se quedan por debajo del umbral de 0.80 (línea punteada negra) y solo una lo cruza, lo que se traduce en un único verdadero positivo.

Validación de generalización (MAWI, granularidad oficial)

Como prueba exploratoria, el ARIMA(3,0,3) entrenado con el *dataset* primario se evaluó sobre MAWI reagregado a granularidad oficial de 1 minuto. La Tabla 6.7 presenta el resultado sobre las seis observaciones evaluables.

Métrica	**Valor**
RMSE	0.323263
MAE	0.236493
Observaciones evaluadas	6
Rango real observado	[0.099, 0.213]

Cuadro 6.7: Generalización de ARIMA(3,0,3) sobre MAWI (granularidad de 1 minuto).

El RMSE de 0.323 equivale a casi el triple de la amplitud del rango real observado (0.114), de modo que el error medio supera con creces la variación total de la señal. Por el tamaño de la muestra (seis observaciones), el resultado es indicativo y no concluyente.

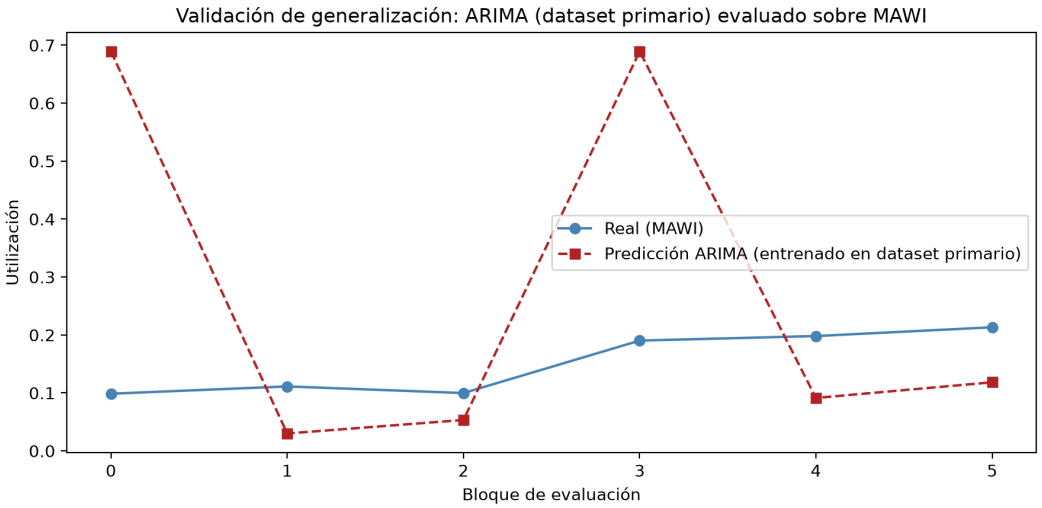


Figura 6.5: Generalización: ARIMA(3,0,3) entrenado con el *dataset* primario y evaluado sobre MAWI.

La Figura 6.5 muestra que la trayectoria de predicción carece de correlación con la dinámica real. Mientras el tráfico real de MAWI se mantiene estable en torno a 0.15, las predicciones oscilan de forma violenta, con picos cercanos a 0.7 (bloques 0 y 3) seguidos de caídas hacia el cero. Los coeficientes autorregresivos y de media móvil se calibraron para un régimen con media cercana a cero y eventos de alta volatilidad (el primario); al recibir una serie con un nivel basal muy superior y otra varianza, el modelo sobrecorrige en cada bloque de evaluación.

Síntesis de los resultados de ARIMA

La Figura 6.6 muestra la evolución del RMSE de regresión entre contextos de evaluación.

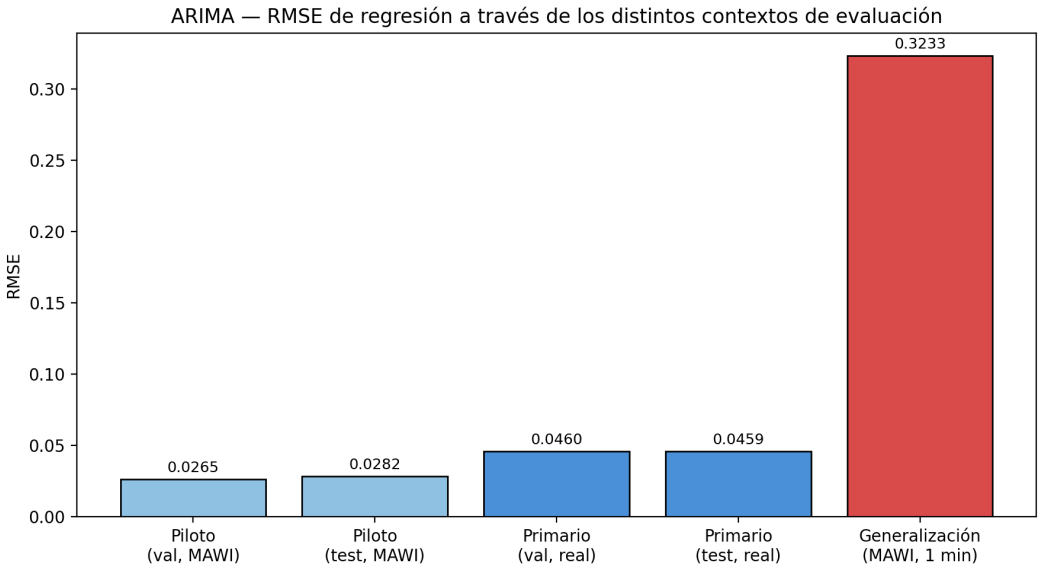


Figura 6.6: RMSE de regresión de ARIMA en los contextos de evaluación.

La progresión de las barras muestra tres comportamientos:

- **Piloto (0.0265 y 0.0282):** es el menor error del modelo. La serie de MAWI es relativamente homogénea y no contiene eventos de congestión, lo que permite que la ecuación autorregresiva promedie el ruido de fondo sin descompensaciones.
- **Primario (0.0460 y 0.0459):** el error basal aumenta cerca de un 63 %. Como la diferencia entre validación y prueba es prácticamente nula, no se explica por sobreajuste, sino por el sesgo de sobreestimación y el efecto del esquema por bloques descritos arriba.
- **Generalización (0.3233):** el error se multiplica por siete respecto al primario, lo que documenta que los coeficientes ajustados en una red no se transfieren a un régimen con otra distribución basal y otra varianza.

La Tabla 6.8 reúne todas las métricas evaluadas y constituye la línea base contra la cual se contrastarán los modelos siguientes.

Contexto	**Naturaleza de los datos**	**RMSE**	***Recall***	**AUC-ROC**
Piloto — validación	100 % real (MAWI)	0.0265	N/A	N/A
Piloto — prueba	100 % real (MAWI)	0.0282	N/A	N/A
Primario — regresión (val.)	100 % real	0.0460	—	—
Primario — regresión (prueba)	100 % real	0.0459	—	—
Primario — clasificación (val.)	100 % sintético	—	0.0357	0.8905
Primario — clasificación (prueba)	100 % sintético	—	0.0185	0.8939
Generalización — MAWI (oficial)	100 % real (MAWI)	0.3233	N/A	N/A

Cuadro 6.8: Síntesis de los resultados de ARIMA. N/A: no aplicable con el umbral oficial (Sección 5.1).

Discusión de los resultados de ARIMA

Regresión. El RMSE sube de 0.0282 en el piloto a 0.0459 en el primario, pero esa comparación es limitada, porque son series distintas, con enlaces de 11 Gbps y 1 Gbps y dispersiones distintas. Lo revelador es la relación entre el error y la propia señal. En el piloto, el RMSE queda por debajo de la desviación estándar de la serie (0.051). En el primario, el RMSE (0.046) supera la utilización máxima observada (0.0144), mientras que una predicción constante igual a la media real tendría un error del orden de la desviación estándar de la serie (alrededor de $3{,}5 \times 10^{-4}$ en fracción). La causa más probable es el sesgo de sobreestimación de la Figura 6.2: el modelo se ajustó sobre la sesión sintética, cuya media ($\mu = 0{,}0324$) está elevada por los eventos inyectados y es muy superior a la real, de orden 10^{-5}. Que el error sea casi idéntico en validación y prueba respalda que es sistemático y no un problema de generalización entre particiones.

Oscilaciones por bloques. Los picos y valles regulares de la Figura 6.2 están asociados al reinicio del estado autorregresivo al inicio de cada bloque del pronóstico progresivo. Con coeficientes AR de gran magnitud y suma cercana a 1, un pequeño desajuste inicial tarda en disiparse. Esto infla el RMSE sin reflejar la capacidad predictiva en régimen estable, de modo que es una limitación del esquema de evaluación por bloques que debe tenerse presente al comparar con los otros modelos.

Clasificación. El *recall* de 1.85 % y el único verdadero positivo indican que, con el umbral de 0.80, el modelo prácticamente no alerta. Es consistente con la naturaleza de ARIMA: su componente de media móvil suaviza el pronóstico hacia la tendencia central, y rara vez la predicción puntual alcanza el umbral, aun durante un ascenso genuino. Esto concuerda con la limitación descrita en la Sección 2.2.2 sobre su naturaleza reactiva, a lo que se suma ahora un comportamiento excesivamente conservador frente a la congestión. En la fase piloto, con un umbral más bajo (0.25), el *recall* fue mayor (41.5 %), lo que sugiere que parte del problema radica en la distancia entre el nivel predicho y el umbral exigido.

Calibración del umbral. El AUC-ROC alto (0.89) matiza la conclusión anterior: la predicción continua conserva información sobre la congestión, y el fallo parece estar en la decisión binaria sobre una predicción amortiguada, más que en la ausencia total de señal. Ajustar el umbral a partir de la validación no se exploró en este trabajo y es una línea de mejora para el diseño de alertas.

Generalización. El colapso sobre MAWI indica que los coeficientes de ARIMA están muy ligados al régimen donde se ajustaron. Una explicación probable es que los valores de MAWI (cerca de 0.15) caen fuera del rango del tráfico real del primario (máximo de 0.0144), y con coeficientes autorregresivos de signo alternante esa diferencia de nivel se amplifica. Por el tamaño de la muestra y el reinicio por bloques, esta explicación debe tomarse como hipótesis. Aun así, la evidencia desaconseja tratar a ARIMA como modelo transferible entre redes de escala distinta sin reajustarlo.

Alcance. En conjunto, ARIMA cumple como línea base de regresión sobre tráfico estable, pero su desempeño depende de que la serie de ajuste represente el régimen donde se evalúa, y no sirve como detector de congestión con el umbral oficial. Esta conclusión se contrasta con *Random Forest* y LSTM en las Secciones 6.4 y 6.5.

6.2. Resultados de *Random Forest*

Esta sección reporta los resultados del ensamble *Random Forest*, configurado según la Sección 5.4: su desempeño en la fase piloto con MAWI, su evaluación de regresión y clasificación sobre el *dataset* primario, el análisis de importancia de variables en ambos conjuntos y las razones por las que no pudo someterse a la validación de generalización frente a MAWI. Cada resultado se acompaña de su lectura inmediata. La comparación con los demás modelos se presenta en la Sección 6.4 y la discusión conjunta en la Sección 6.5.

Fase piloto: regresión

Sobre el piloto, el modelo se entrenó con 1 044 secuencias, usando los rezagos y la media móvil con granularidad de 1 segundo. La Tabla 6.9 presenta las métricas sobre las particiones de validación y prueba, en escala normalizada.

Conjunto	RMSE	MAE	MAPE
Validación	0.0962	0.0721	12.83 %
Prueba	0.1311	0.1004	16.26 %

Cuadro 6.9: Métricas de regresión de *Random Forest* (escala normalizada), *dataset* piloto MAWI.

El error aumenta de validación a prueba: el RMSE crece un 36 % y el MAPE pasa de 12.83 % a 16.26 %. En ARIMA, en cambio, el MAPE se mantuvo prácticamente igual en ambas particiones (10.61 % y 10.80 %, Sección 6.1). Para comparar con la línea base estadística, que se mantuvo en escala original, el RMSE de prueba se reescala por el rango de la variable de entrenamiento ($0{,}323 - 0{,}017 = 0{,}306$). Se obtiene un RMSE equivalente de aproximadamente $0{,}1311 \times 0{,}306 \approx 0{,}040$, un 42 % superior al de ARIMA (0.0282).

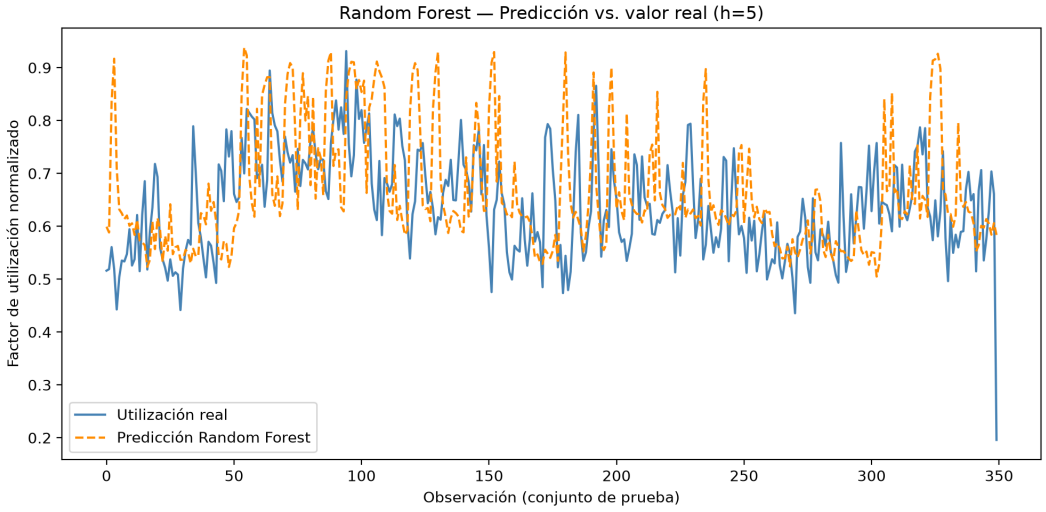


Figura 6.7: *Random Forest*: predicción frente al valor real, partición de prueba del piloto MAWI (escala normalizada).

La Figura 6.7 muestra la causa de ese aumento. Cuando el tráfico real (línea azul) presenta caídas abruptas que llegan hasta cerca de 0.2 en el eje normalizado (por ejemplo, alrededor de la observación 350), la predicción del ensamble (línea naranja discontinua) tiene un piso visible y no desciende por debajo de aproximadamente 0.5. El modelo recorta, por tanto, los valles de la serie. Dado que cada hoja de un árbol predice el promedio de las muestras de entrenamiento que cayeron en ella, el ensamble tiende hacia valores centrales y responde poco a los extremos del comportamiento estocástico.

Fase piloto: clasificación

Como en ARIMA, con el umbral oficial $U_{th} = 0{,}80$ la clasificación no es evaluable en MAWI (Sección 5.1), por lo que se usó el umbral relativo de prueba $U_{test} \geq 0{,}25$. La Tabla 6.10 presenta el desempeño sobre la partición de prueba.

Métrica ($U_{test} \geq 0{,}25$)	**Prueba**
Precisión	76.5 %
Recall	68.3 %
F1-score	72.1 %
Tiempo de Anticipación Efectivo (TAE) promedio	**2.8 minutos**

Cuadro 6.10: Clasificación y alertamiento (TAE) de *Random Forest*, fase piloto.

Aproximadamente tres de cada cuatro alertas corresponden a un evento real (precisión de 76.5 %) y el modelo detecta cerca de dos tercios de los eventos (*recall* de 68.3 %). Frente a ARIMA, el *recall* sube 26.8 puntos porcentuales (de 41.5 % a 68.3 %) y el TAE pasa de 1.2 a 2.8 minutos, más de la mitad del horizonte objetivo de 5 minutos. Como la regresión continua fue peor que la de ARIMA, el resultado muestra que el error medio de pronóstico y la capacidad de alertar son capacidades distintas. Estas cifras validan el *pipeline*, pero no son comparables con las del *dataset* primario, que usan otro umbral y otros datos.

Fase piloto: importancia de variables

La Figura 6.8 presenta la importancia relativa de las variables de entrada en el piloto, calculada como la disminución media de impureza que aporta cada una en los árboles.

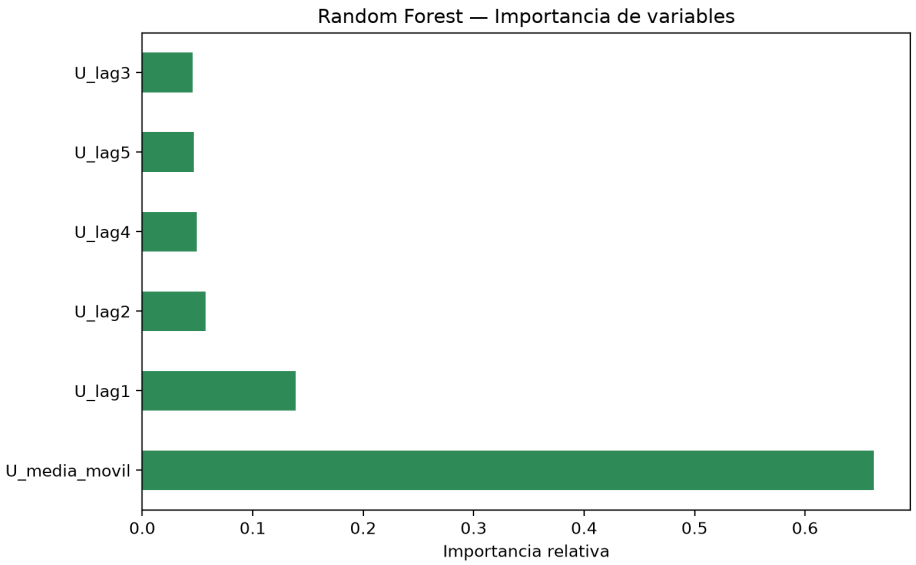


Figura 6.8: Importancia relativa de variables en *Random Forest*, *dataset* piloto MAWI.

La media móvil ($U_{\text{media_movil}}$) concentra el 66.2 % de la importancia, muy por encima del rezago inmediato (U_{lag1}, 13.9 %). El 19.9 % restante se reparte entre los rezagos 2 a 5, cuya contribución individual es marginal. El modelo se apoya, entonces, sobre todo en el nivel promedio reciente y poco en el valor del instante anterior. Una explicación plausible es que, a una granularidad de 1 segundo, el valor inmediato está muy contaminado por ruido de alta frecuencia, y el promedio actúa como un filtro que revela la tendencia de fondo.

Dataset primario: regresión

La Tabla 6.11 presenta las métricas sobre las particiones reales, en escala normalizada. El modelo se reentrenó con la misma configuración sobre las 46 872 observaciones de entrenamiento (reales y sintéticas).

Conjunto	**RMSE**	**MAE**
Validación (real)	0.005084	0.004766
Prueba (real)	0.049590	0.010301

Cuadro 6.11: Métricas de regresión de *Random Forest*, *dataset* primario (escala normalizada).

El RMSE aumenta casi diez veces entre validación y prueba (de 0.0051 a 0.0496), mientras que en ARIMA se mantuvo constante sobre las mismas particiones (de 0.0460 a 0.0459). El MAE, sin embargo, solo se duplica (de 0.0048 a 0.0103). La relación entre ambas métricas es reveladora: en validación el RMSE es prácticamente igual al MAE (cociente de 1.07), lo que indica errores pequeños y uniformes, mientras que en prueba el cociente es de 4.8, lo que indica que el error se concentra en unas pocas desviaciones muy grandes. Esto coincide con la Figura 6.10. Como el rango de entrenamiento está cerca de $[0, 1]$ (los eventos sintéticos llegan a 0.979), la escala normalizada es aproximadamente equivalente a la original, de modo

que el RMSE de prueba (0.0496) es del orden del de ARIMA (0.0459), y el MAE (0.0103) es menor que el de ARIMA (0.0151). En ambos casos, el RMSE supera la utilización real máxima observada (0.0144).

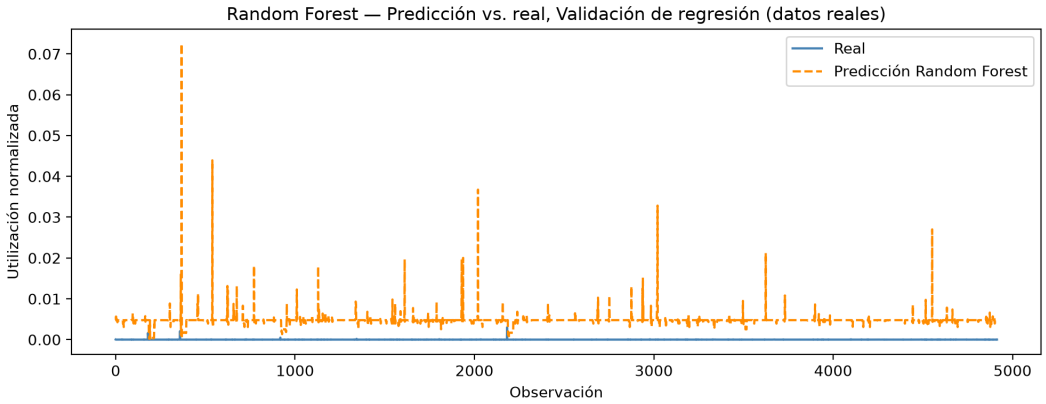


Figura 6.9: *Random Forest*: predicción frente al valor real, validación de regresión (*dataset* primario, datos reales).

En validación (Figura 6.9), la predicción (línea naranja discontinua) se mantiene cerca de la señal real, plana y cercana a cero. Presenta un sesgo leve de sobreestimación basal y picos transitorios que no superan 0.07 en la escala normalizada, por lo que no afectan de manera apreciable el RMSE.

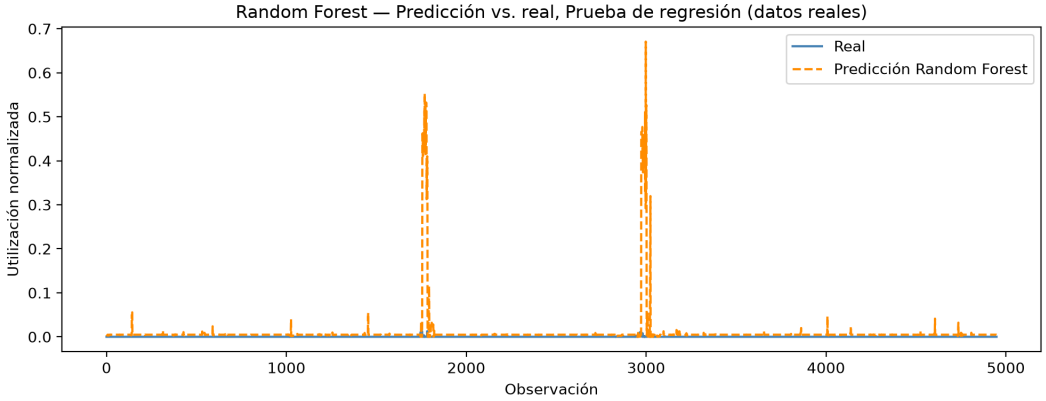


Figura 6.10: *Random Forest*: predicción frente al valor real, prueba de regresión (*dataset* primario, datos reales).

En prueba (Figura 6.10) el tráfico real sigue igual de plano, pero la predicción presenta dos picos de sobreestimación de gran magnitud, de aproximadamente 0.5 y casi 0.7 en la escala normalizada. Estos picos, que no corresponden a ningún evento real, explican prácticamente todo el aumento del RMSE.

Dataset primario: síntesis del RMSE de regresión

La Figura 6.11 muestra el RMSE de regresión del ensamble en los cuatro contextos de evaluación.

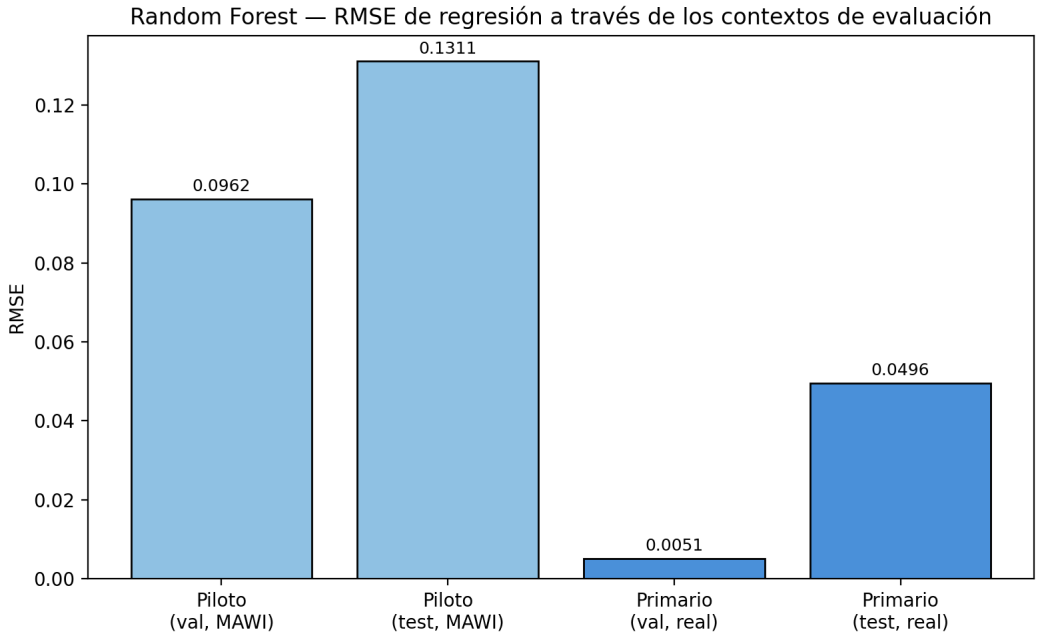


Figura 6.11: RMSE de regresión de *Random Forest* en los contextos de evaluación (escala normalizada).

Las barras muestran dos comportamientos:

- **Piloto (barras celestes, 0.0962 y 0.1311):** el error es elevado y empeora en prueba, asociado al piso de la predicción frente a las caídas profundas del tráfico (Figura 6.7).
- **Primario (barras azules, 0.0051 y 0.0496):** el error de validación es muy bajo, pero en prueba aumenta casi un orden de magnitud por los dos picos espurios de la Figura 6.10.

Los valores del piloto y del primario corresponden a series distintas, con escalas de normalización distintas, por lo que no deben compararse directamente entre sí.

Dataset primario: clasificación

La Tabla 6.12 presenta las métricas de clasificación sobre las particiones sintéticas aisladas, con $U_{th} = 0{,}80$ (en escala normalizada).

Métrica	Validación	Prueba
Exactitud	0.9803	0.9813
Precisión	0.9121	0.9318
Recall	0.2923	0.2982
F1-score	0.4427	0.4518
AUC-ROC	0.8796	0.9187

Cuadro 6.12: Métricas de clasificación de *Random Forest*, $U_{th} = 0{,}80$, datos sintéticos aislados.

El desempeño es estable entre ambas particiones. Como en ARIMA, la exactitud alta (0.98) tiene poco valor informativo por el desbalance de clases, pues los casos de congestión son cerca del 2.6 % de las observaciones. Las métricas relevantes son la precisión, alta (0.93 en prueba), y el *recall*, modesto (0.298), que dan un *F1-score* de 0.45. El *recall* es 16 veces el de ARIMA (0.0185), aunque esa comparación debe tomarse con cautela porque las matrices de confusión de ambos modelos no se calcularon sobre el mismo número de observaciones. Además, el AUC-ROC de *Random Forest* (0.88 a 0.92) es parecido al de ARIMA (0.89), lo que sugiere que la mejora de *recall* proviene sobre todo de que las predicciones alcanzan el umbral con mayor frecuencia, y no de una capacidad muy distinta para ordenar los instantes de congestión por encima de los normales.

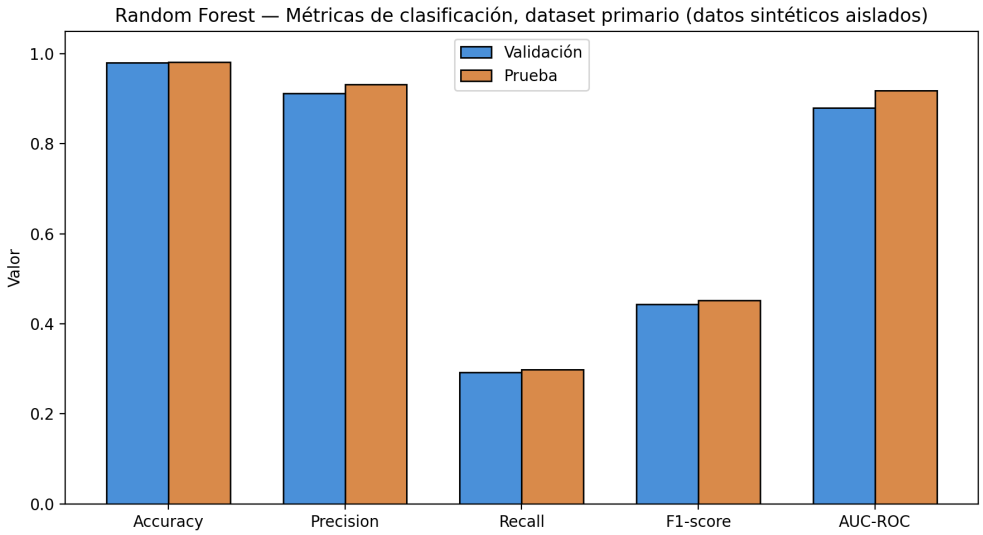


Figura 6.12: Comparación de métricas de clasificación, validación frente a prueba, *Random Forest* sobre el *dataset* primario.

La Figura 6.12 confirma el patrón: exactitud, precisión y AUC-ROC altos, y *recall* y *F1-score* moderados en ambas particiones. La Ecuación 6.4 presenta la matriz de confusión sobre la partición de prueba, cuya suma (10 622 observaciones, 275 casos positivos) coincide con la Tabla 5.3.

$$\begin{pmatrix} \text{VN} = 10\,341 & \text{FP} = 6 \\ \text{FN} = 193 & \text{VP} = 82 \end{pmatrix} \tag{6.4}$$

Sus componentes se interpretan así:

- **Verdaderos negativos (VN=10 341):** el modelo reconoce correctamente los extensos periodos de operación normal.
- **Falsos positivos (FP=6):** solo seis falsas alarmas en todo el conjunto, equivalentes al 0.06 % de las observaciones normales. De ahí la precisión de 93.18 % (82 de 88 alertas fueron correctas).
- **Verdaderos positivos (VP=82):** de 275 instantes de congestión, se detectaron 82. Con ARIMA solo se detectó uno.
- **Falsos negativos (FN=193):** el 70 % de los instantes de congestión pasa sin alerta, lo que mantiene el *recall* en 29.8 %.

En términos operativos, el modelo casi no genera ruido, pero omite siete de cada diez instantes de congestión. Con una precisión alta y un *recall* moderado, el sistema sería confiable cuando alerta, pero insuficiente como única protección.

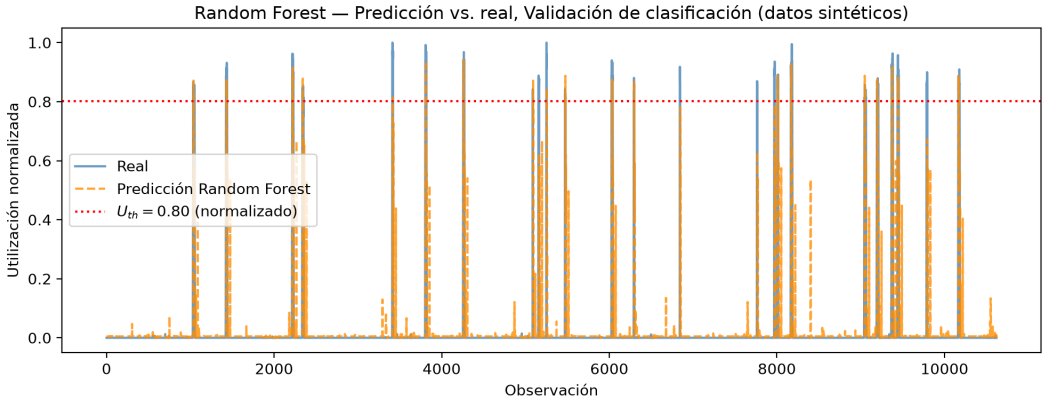


Figura 6.13: *Random Forest*: predicción frente al valor real y umbral $U_{th} = 0{,}80$, validación de clasificación (*dataset* primario, datos sintéticos aislados).

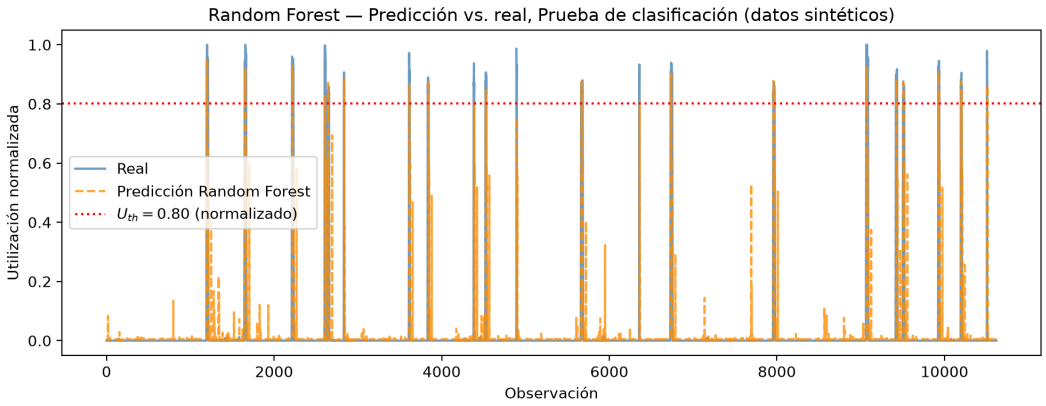


Figura 6.14: *Random Forest*: predicción frente al valor real y umbral $U_{th} = 0{,}80$, prueba de clasificación (*dataset* primario, datos sintéticos aislados).

Las Figuras 6.13 y 6.14 muestran el origen de los falsos negativos. A diferencia de ARIMA, cuya predicción se aplana, la del ensamble acompaña a los eventos: por cada pico del tráfico real aparece un pico de la predicción en el mismo momento. Sin embargo, la magnitud de muchos picos predichos es menor que la real, y varios se quedan justo por debajo del umbral de 0.80. El modelo identifica *cuándo* ocurre el evento, pero no siempre estima su intensidad lo bastante alta para cruzar el umbral. El patrón es el mismo en validación y en prueba.

Dataset primario: importancia de variables

La Tabla 6.13 y la Figura 6.15 presentan la importancia relativa de cada variable en el *dataset* primario.

Variable	Importancia relativa
U_{lag1}	0.6644
$U_{\text{media_movil}}$	0.1249
U_{lag5}	0.1171
U_{lag2}	0.0320
U_{lag3}	0.0315
U_{lag4}	0.0301

Cuadro 6.13: Importancia relativa de variables, *Random Forest*, *dataset* primario.

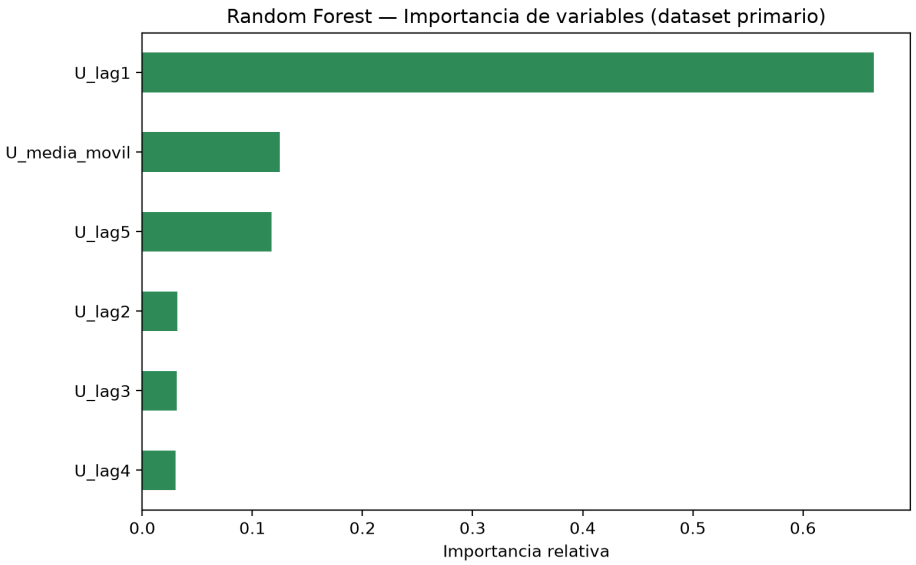


Figura 6.15: Importancia relativa de variables en *Random Forest*, *dataset* primario.

El rezago inmediato concentra dos tercios de la importancia (66.4 %). Le siguen la media móvil (12.5 %) y el rezago más antiguo, U_{lag5} (11.7 %), mientras que los rezagos 2 a 4 aportan alrededor del 3 % cada uno. Que el rezago 5 pese más que los intermedios sugiere que el modelo usa el contraste entre el valor más reciente y el más antiguo de la ventana, es decir, una señal de tendencia, aunque con estos datos no puede confirmarse.

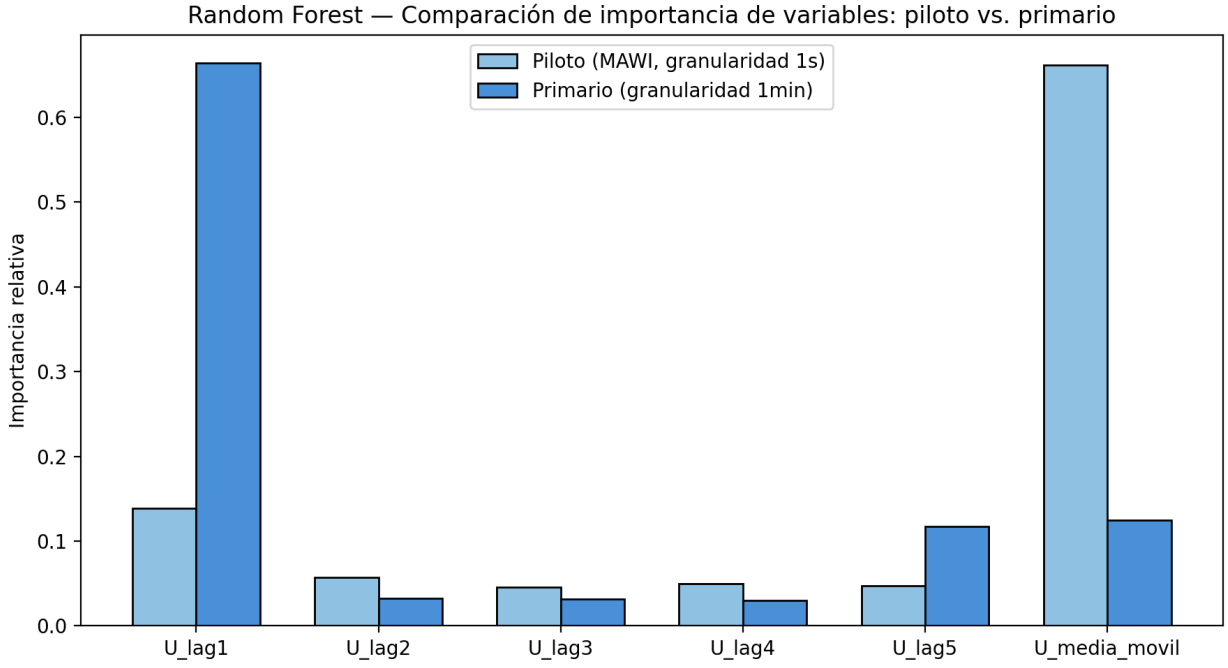


Figura 6.16: Importancia relativa de variables en *Random Forest*: piloto frente al *dataset* primario.

La Figura 6.16 contrasta ambos conjuntos y muestra una inversión casi simétrica: la media móvil pasa de 66.2 % a 12.5 % y el rezago inmediato, de 13.9 % a 66.4 %. La Tabla 6.14 resume la comparación de la fase piloto con el *dataset* primario.

Métrica (prueba)	**Piloto (MAWI)**	**Primario**
RMSE (regresión, normalizado)	0.1311	0.0496
Recall (clasificación)	68.3 % ($U_{test} = 0{,}25$)	29.8 % ($U_{th} = 0{,}80$)
F1-score (clasificación)	72.1 % ($U_{test} = 0{,}25$)	45.2 % ($U_{th} = 0{,}80$)
Variable más importante	$U_{\text{media_movil}}$ (66.2 %)	U_{lag1} (66.4 %)

Cuadro 6.14: Comparación de *Random Forest* entre la fase piloto y el *dataset* primario. Las métricas de clasificación usan umbrales distintos y no son directamente comparables.

Validación de generalización no realizada

A diferencia de ARIMA (Sección 6.1), *Random Forest* no pudo evaluarse sobre MAWI reagregado a la granularidad oficial de 1 minuto. La variable $U_{\text{media_movil}}$ se calcula como una media móvil de 30 minutos (Sección 5.2), y no puede construirse dentro de una traza de MAWI, que dura solo 15 minutos continuos (Sección 5.1). Sin ese componente no se puede formar el vector de entrada que el ensamble espera. Rellenar con ceros o acortar la ventana alteraría la configuración validada y haría injusta cualquier comparación con ARIMA, por lo que esta validación se reporta como no realizada.

Síntesis de los resultados de *Random Forest*

La Tabla 6.15 reúne las métricas de todos los contextos de evaluación.

Contexto	**Naturaleza de los datos**	**RMSE**	***Recall***	**AUC-ROC**
Piloto - validación	100 % real (MAWI)	0.0962	N/A	N/A
Piloto - prueba	100 % real (MAWI)	0.1311	N/A	N/A
Primario - regresión (val.)	100 % real	0.0051	—	—
Primario - regresión (prueba)	100 % real	0.0496	—	—
Primario - clasificación (val.)	100 % sintético	—	0.2923	0.8796
Primario - clasificación (prueba)	100 % sintético	—	0.2982	0.9187
Generalización — MAWI (oficial)	No evaluable (incompatibilidad estructural)			

Cuadro 6.15: Síntesis de los resultados de *Random Forest*. N/A: no aplicable con el umbral oficial (Sección 5.1). Los RMSE están en escala normalizada.

Discusión de los resultados de *Random Forest*

Regresión en el piloto. El piso de la predicción frente a las caídas profundas (Figura 6.7) es coherente con el promediado de las hojas: el árbol responde con el valor medio de las muestras de entrenamiento similares y suaviza los extremos. Esta limitación suele describirse como incapacidad de extrapolar, aunque en sentido estricto la extrapolación implica valores fuera del rango de entrenamiento, y una caída a 0.2 en escala normalizada está dentro del

rango $[0, 1]$. Una explicación alternativa es que esas caídas fueran poco frecuentes en la partición de entrenamiento y, por tanto, quedaran poco representadas en las hojas. Con estos datos no es posible distinguir entre ambas.

Regresión en el primario. El deterioro entre validación y prueba tiene otro origen: los picos de la Figura 6.10 alcanzan magnitudes de 0.5 y 0.7, que están dentro del rango de entrenamiento (hasta 0.979), así que no pueden atribuirse a falta de extrapolación. Una hipótesis plausible es que el modelo, entrenado con eventos sintéticos que incluyen una fase de ascenso gradual, haya aprendido a asociar niveles bajos de los rezagos con el inicio de una congestión. En ese caso, una misma entrada ocurriría tanto en el tráfico ocioso real como en el arranque de un evento sintético, y las hojas que mezclan ambos casos devolverían valores altos ante fluctuaciones inofensivas. La relación entre RMSE y MAE (cociente de 4.8 en prueba frente a 1.07 en validación) respalda que el error está concentrado en pocos episodios y no es generalizado. Esta explicación es una hipótesis; para verificarla habría que revisar qué entradas produjeron los dos picos y qué hojas los generaron.

Clasificación. Frente a ARIMA, el ensamble multiplica el *recall* por 16 y mantiene una precisión alta, por lo que supera al modelo estadístico como detector de congestión, y su sincronía con los eventos (Figuras 6.13 y 6.14) indica que sí capta la forma de los eventos. Aun así, el *recall* de 29.8 % es insuficiente para un sistema de alerta temprana. El AUC-ROC alto (0.92), similar al de ARIMA, sugiere que la información está en la predicción continua y que el cuello de botella es la distancia entre la magnitud predicha y el umbral. Una calibración del umbral sobre la validación sería la vía natural para aprovechar esa señal, a costa de más falsas alarmas, que hoy son muy pocas (FP=6). La comparación con ARIMA es indicativa, no estricta, porque los conjuntos sobre los que se calcularon las matrices de confusión difieren en tamaño.

Importancia de variables. La inversión entre media móvil y rezago inmediato admite al menos dos explicaciones. La primera es la granularidad: a 1 segundo el valor inmediato es ruidoso y el promedio lo filtra, mientras que a 1 minuto cada observación ya integra decenas de segundos de tráfico y el último valor basta. La segunda es la composición de los datos: el entrenamiento del primario incluye eventos sintéticos con perfil trapezoidal, en los que el valor anterior casi determina el siguiente, lo que favorecería al rezago inmediato independientemente de la granularidad. Con los experimentos realizados no se pueden separar ambos factores. Además, la importancia por impureza tiende a favorecer variables continuas con muchos valores distintos y no mide el efecto de cada variable sobre el desempeño, por lo que sus cifras deben leerse como orientativas.

Generalización. La imposibilidad de evaluar el modelo sobre MAWI refleja un compromiso: la ingeniería de características con ventanas largas puede mejorar el desempeño dentro de un dominio, pero ata el modelo a datos con suficiente historia continua y reduce su portabilidad hacia conjuntos con restricciones distintas. Como consecuencia, ARIMA y *Random Forest* no son comparables en esta dimensión.

Alcance. En conjunto, *Random Forest* mejora de forma clara la detección de congestión respecto a ARIMA, con pocas falsas alarmas, pero queda lejos de un *recall* adecuado para alertar y mostró inestabilidad en la regresión sobre tráfico real ocioso. Estas conclusiones se contrastan con LSTM en las Secciones 6.4 y 6.5.

6.3. Resultados de LSTM

Esta sección reporta los resultados de la red LSTM, configurada según la Sección 5.5: la convergencia del entrenamiento en ambos conjuntos de datos, su desempeño en la fase piloto con MAWI, su evaluación de regresión y clasificación sobre el *dataset* primario y las razones por las que no pudo someterse a la validación de generalización frente a MAWI. Cada resultado se acompaña de su lectura inmediata. La comparación con los demás modelos se presenta en la Sección 6.4 y la discusión conjunta en la Sección 6.5.

Convergencia del entrenamiento

En el piloto, la red (4 897 parámetros entrenables) se entrenó con 1 044 secuencias y la detención temprana la detuvo en la época 12 de 100. Con una paciencia de 10 épocas, esto implica que los mejores pesos corresponden a la época 2. La Figura 6.17 muestra que la pérdida MSE cae de forma abrupta en la primera época y luego se estabiliza, con una separación constante entre entrenamiento y validación y sin divergencia severa, por lo que no hay indicios de sobreajuste marcado.

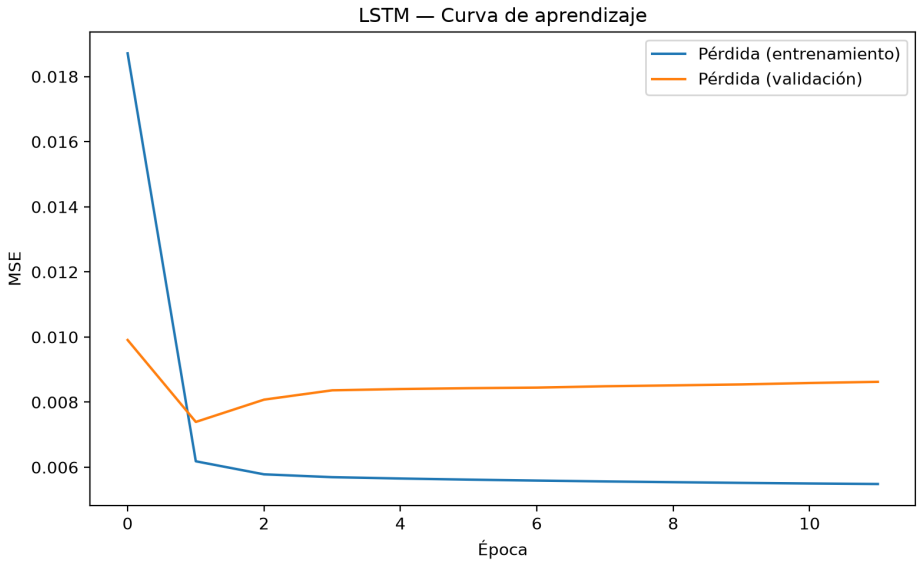


Figura 6.17: Curva de aprendizaje de la red LSTM (pérdida MSE por época), *dataset* piloto MAWI.

En el *dataset* primario, la red se reentrenó con la misma arquitectura sobre 46 582 secuencias y se detuvo en la época 16. La Figura 6.18 muestra una asimetría llamativa: la pérdida de entrenamiento (línea azul) baja de 0.010 a cerca de 0.006, mientras que la de validación (línea naranja discontinua) es prácticamente nula desde la primera época.

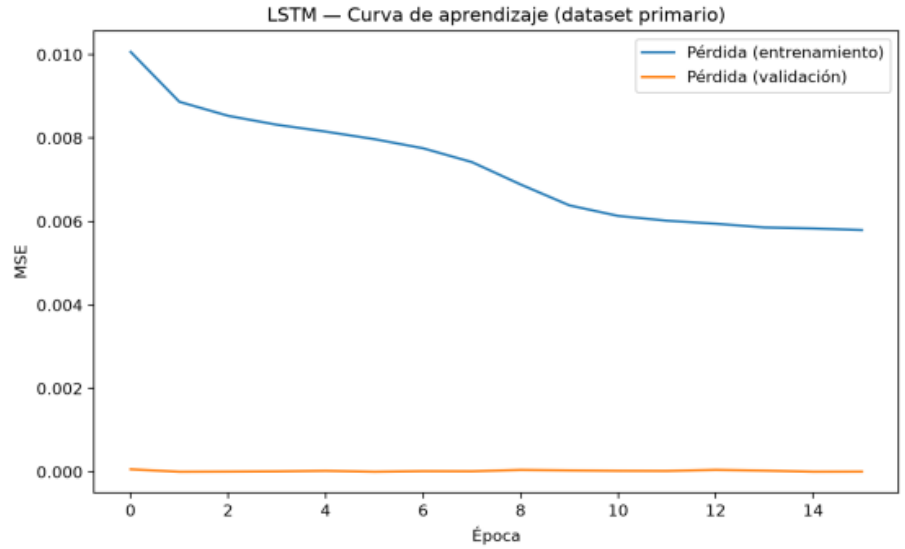


Figura 6.18: Curva de aprendizaje de la red LSTM, *dataset* primario. La separación entre curvas refleja la composición de las particiones: entrenamiento con eventos sintéticos (azul) y validación con tráfico real ocioso (naranja).

Esta separación no indica una fuga de datos, sino la composición de las particiones definida en la Sección 5.1. El entrenamiento incluye los 126 eventos sintéticos, con valores cercanos a 1, mientras que la validación de regresión es 100 % tráfico real ocioso, con un máximo normalizado de 0.0029 (Tabla 5.8). Sobre esa partición, el MSE de una predicción cercana a cero es mínimo. Por ello, la distancia entre ambas curvas no sirve por sí sola como indicador de sobreajuste. La pérdida de entrenamiento se estabiliza en torno a 0.006 (RMSE aproximado de 0.077), lo que indica que la red no ajusta por completo los eventos sintéticos ni siquiera en la partición que ve durante el entrenamiento.

La Figura 6.19 separa ambas curvas en ejes independientes. En el panel de validación se observa una dinámica del orden de 10^{-4}: el mínimo se alcanza en la época 1 (línea vertical discontinua) y, a partir de la época 2, la pérdida aumenta hacia $4{,}0 \times 10^{-4}$. La detención temprana identifica ese deterioro y restaura los pesos de la época 1.

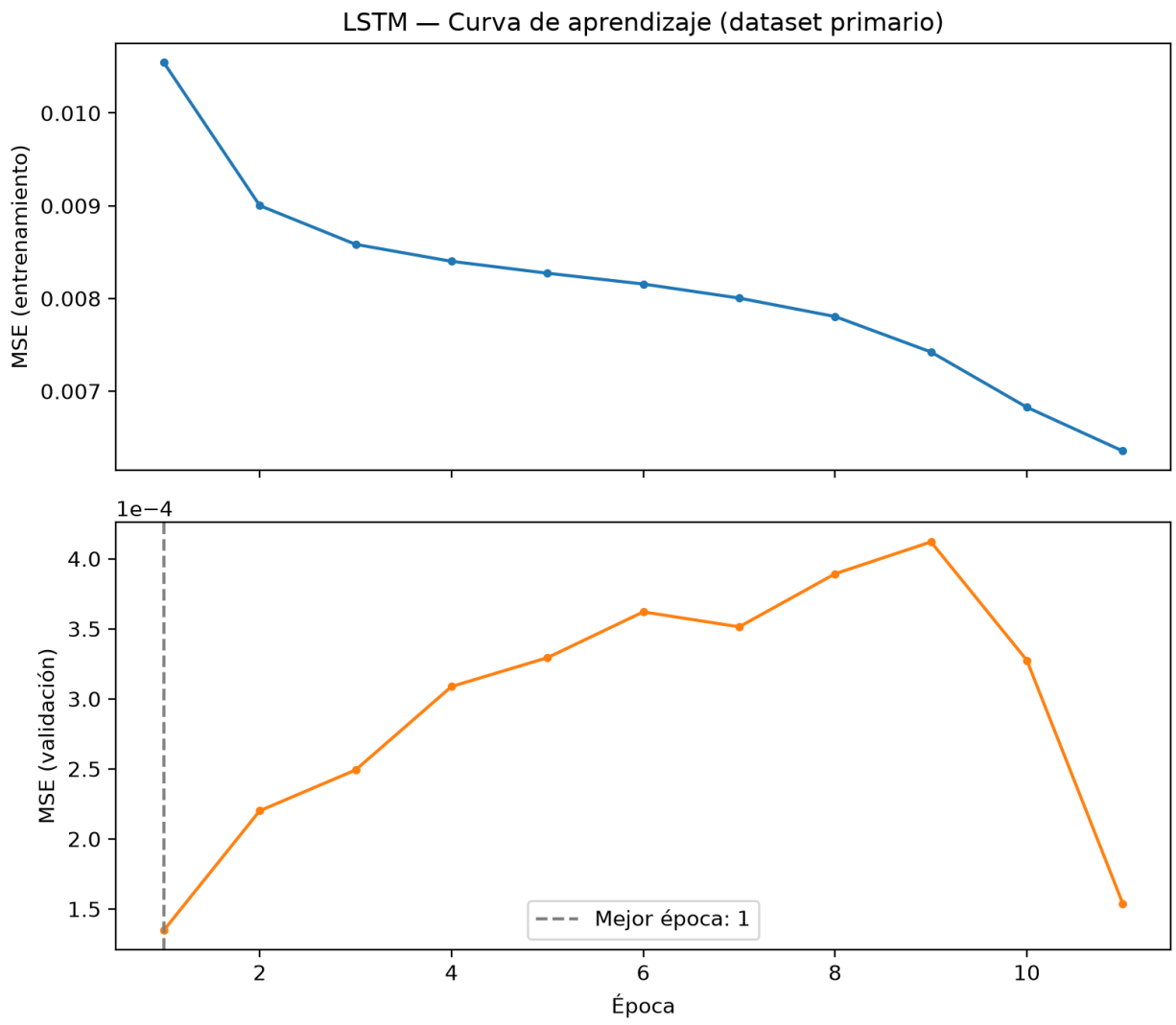


Figura 6.19: Curva de aprendizaje en ejes independientes. Se aprecia la dinámica de validación en el orden de 10^{-4} y el punto de corte de la detención temprana.

La Figura 6.20 unifica ambas curvas en escala logarítmica. Confirma una brecha de aproximadamente dos órdenes de magnitud entre entrenamiento ($\sim 10^{-2}$) y validación ($\sim 10^{-4}$), y muestra que ninguna de las curvas diverge a lo largo del entrenamiento.

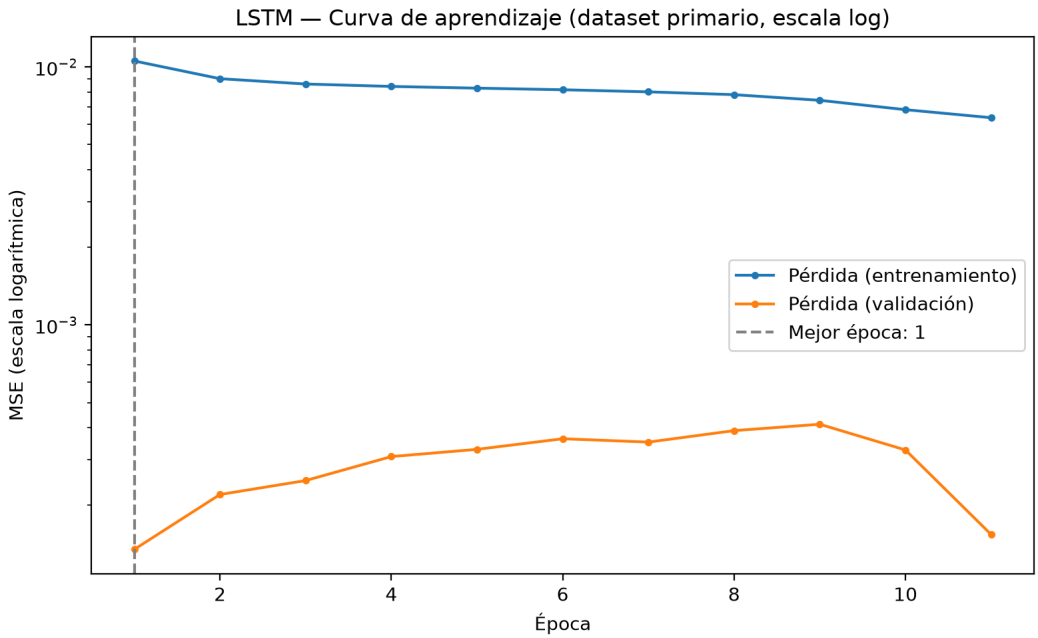


Figura 6.20: Curva de aprendizaje de la red LSTM en escala logarítmica, *dataset* primario.

Fase piloto: regresión

La Tabla 6.16 presenta las métricas sobre MAWI, en escala normalizada.

Conjunto	**RMSE**	**MAE**	**MAPE**
Validación	0.0860	0.0664	11.53 %
Prueba	0.0891	0.0690	11.00 %

Cuadro 6.16: Métricas de regresión de LSTM (escala normalizada), *dataset* piloto MAWI.

El error es estable entre validación y prueba: el RMSE aumenta un 3.6 % y el MAPE incluso baja de 11.53 % a 11.00 %. A diferencia de *Random Forest*, cuyo MAPE pasó de 12.83 % a 16.26 % (Sección 6.2), la red no se degrada en la partición de prueba. Respecto al ensamble, el RMSE de prueba baja un 32 % (de 0.1311 a 0.0891, ambos normalizados). Frente a ARIMA, que opera en escala original, la comparación exige reescalar: $0{,}0891 \times 0{,}306 \approx 0{,}027$, un valor del mismo orden que el 0.0282 de ARIMA, mientras que el MAPE de ARIMA (10.80 %) es ligeramente menor que el de LSTM (11.00 %). En regresión sobre el piloto, la red es comparable a la línea base estadística y mejor que el ensamble de árboles.

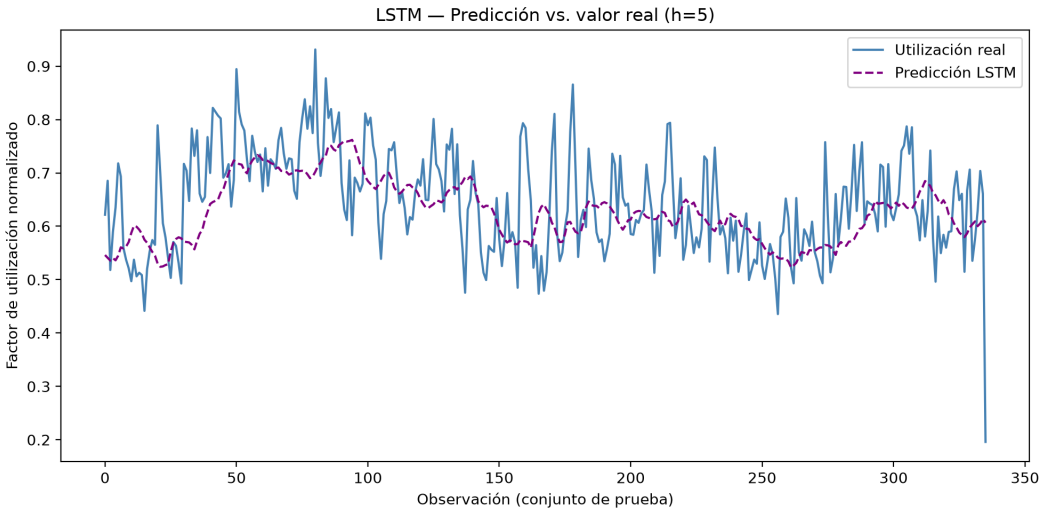


Figura 6.21: LSTM: predicción frente al valor real, partición de prueba del piloto MAWI (escala normalizada).

La señal real de la Figura 6.21 es la misma que la evaluada con *Random Forest*, en escala normalizada; su aspecto distinto de las gráficas de ARIMA se debe a que este operó sobre la métrica sin escalar. La predicción de la red (línea púrpura discontinua) suaviza las oscilaciones de corto plazo y traza la tendencia, como ARIMA. Además, acompaña la caída profunda del final de la serie (cerca de la observación 330), donde el ensamble de árboles se quedaba en un piso. Esa caída está dentro del rango de entrenamiento, por lo que no se trata de extrapolación propiamente dicha, sino de una mejor respuesta a valores poco frecuentes.

Fase piloto: clasificación

Con el umbral oficial $U_{th} = 0{,}80$ la clasificación no es evaluable en MAWI (Sección 5.1), por lo que se usó el umbral relativo de prueba $U_{test} \geq 0{,}25$. La Tabla 6.17 presenta el desempeño sobre la partición de prueba.

Métrica ($U_{test} \geq 0{,}25$)	**Prueba**
Precisión	81.2 %
Recall	79.4 %
F1-score	80.3 %
Tiempo de Anticipación Efectivo (TAE) promedio	**3.9 minutos**

Cuadro 6.17: Clasificación y alertamiento (TAE) de LSTM, fase piloto.

Cuatro de cada cinco alertas corresponden a un evento real (precisión de 81.2 %) y el modelo detecta también cuatro de cada cinco eventos (*recall* de 79.4 %). Respecto a ARIMA, el *recall* sube 37.9 puntos porcentuales (de 41.5 % a 79.4 %) y, respecto a *Random Forest*, 11.1 (de 68.3 %). El TAE de 3.9 minutos equivale al 78 % del horizonte objetivo de 5 minutos, frente a 2.8 minutos del ensamble y 1.2 de ARIMA. En el piloto, y con un umbral bajo, la

red es el mejor detector de los tres. Estas cifras validan el *pipeline*, pero no son comparables con las del *dataset* primario, que usan otro umbral y otros datos.

Dataset primario: regresión

La Tabla 6.18 presenta las métricas sobre las particiones reales, en escala normalizada.

Conjunto	RMSE	MAE
Validación (real)	0.000131	0.000019
Prueba (real)	0.001036	0.000125

Cuadro 6.18: Métricas de regresión de LSTM, *dataset* primario (escala normalizada).

El RMSE de prueba (0.001036) es unas 44 veces menor que el de ARIMA (0.0459) y unas 48 veces menor que el de *Random Forest* (0.0496) sobre las mismas particiones, y el MAE (0.000125) es unas 120 veces menor que el de ARIMA (0.0151). Como el rango de entrenamiento es cercano a $[0, 1]$, la escala normalizada es aproximadamente equivalente a la original, de modo que la comparación es razonable. El error crece entre validación y prueba (el RMSE se multiplica por 7.9 y el MAE por 6.6), y la relación RMSE/MAE (6.9 en validación y 8.3 en prueba) indica que el error se concentra en pocas desviaciones grandes, no en un desajuste generalizado.

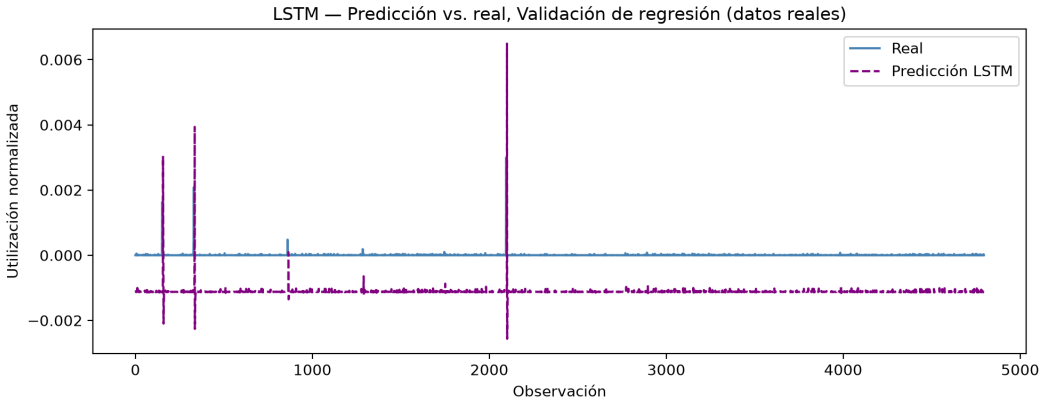


Figura 6.22: LSTM: predicción frente al valor real, validación de regresión (*dataset* primario, datos reales).

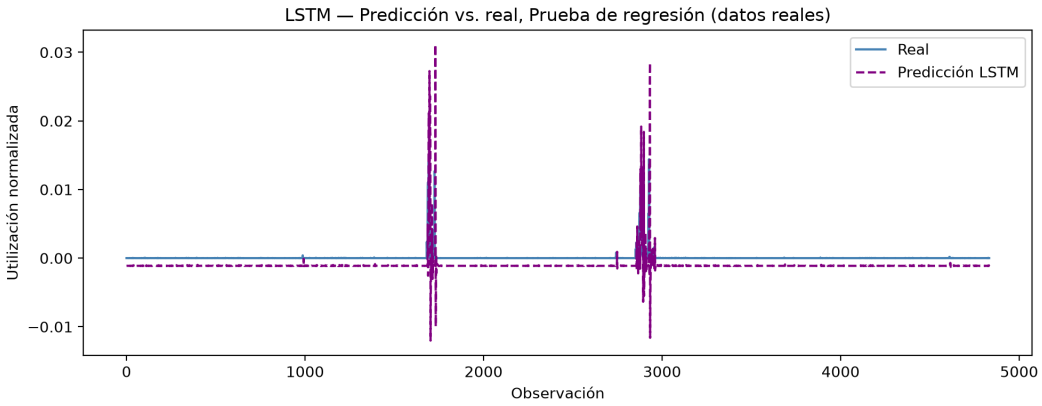


Figura 6.23: LSTM: predicción frente al valor real, prueba de regresión (*dataset* primario, datos reales).

En validación (Figura 6.22), la predicción se mantiene anclada al cero real, con perturbaciones que no superan 0.006. En prueba (Figura 6.23) la red también reacciona ante algunas micro-fluctuaciones y genera picos transitorios, pero su magnitud máxima es de aproximadamente 0.03, unas 23 veces menor que los picos de 0.7 de *Random Forest* sobre el mismo conjunto. Esos picos siguen siendo el doble de la utilización real máxima observada (0.0144) y explican el aumento del error en prueba. El sesgo de sobreestimación que afectó a ARIMA no aparece: la predicción base se adhiere a la señal real, con una oscilación leve hacia valores negativos, esperable en una capa de salida lineal sin restricción de rango.

La Figura 6.24 muestra el RMSE de regresión de la red en los cuatro contextos de evaluación, en escala logarítmica, porque los errores del *dataset* primario (del orden de 10^{-3} y 10^{-4}) quedarían indistinguibles del cero en escala lineal junto a los del piloto (del orden de 10^{-1}). La diferencia entre ambos conjuntos no debe leerse como una mejora de la red, porque son series distintas: el tráfico real del primario es casi nulo (máximo de 1.44 %), y cualquier predicción cercana a cero produce errores absolutos pequeños.

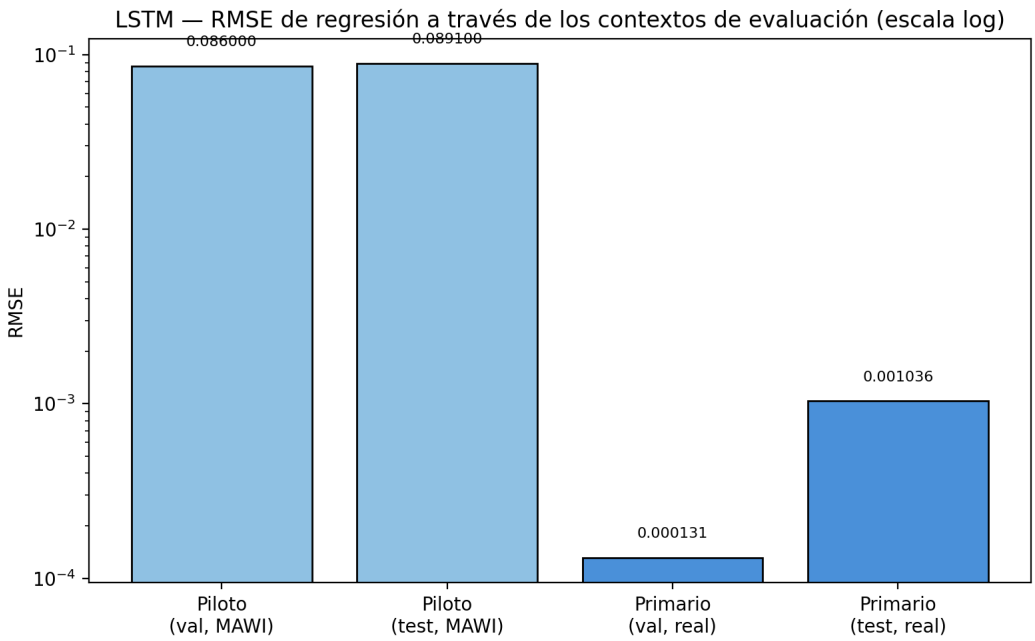


Figura 6.24: RMSE de regresión de LSTM en los contextos de evaluación (escala logarítmica).

Dataset primario: clasificación

La Tabla 6.19 presenta las métricas de clasificación sobre las particiones sintéticas aisladas, con $U_{th} = 0{,}80$ (en escala normalizada).

Métrica	**Validación**	**Prueba**
Exactitud	0.9742	0.9750
Precisión	0.7619	0.7778
Recall	0.0563	0.0509
F1-score	0.1049	0.0956
AUC-ROC	0.8783	0.9000

Cuadro 6.19: Métricas de clasificación de LSTM, $U_{th} = 0{,}80$, datos sintéticos aislados.

El patrón es estable entre ambas particiones. Como en los otros modelos, la exactitud (0.975) tiene poco valor informativo por el desbalance de clases, pues los casos de congestión son cerca del 2.6 % de las observaciones. La precisión es aceptable (0.78) y el *recall*, muy bajo (0.051), lo que da un *F1-score* de 0.096. Frente a ARIMA (0.0185), el *recall* es unas 2.8 veces mayor, aunque en términos absolutos sigue siendo muy bajo, y frente a *Random Forest* (0.2982) es unas seis veces menor. El AUC-ROC de 0.90 es alto y casi igual al de los otros dos modelos (0.89 en ARIMA y 0.92 en *Random Forest*).

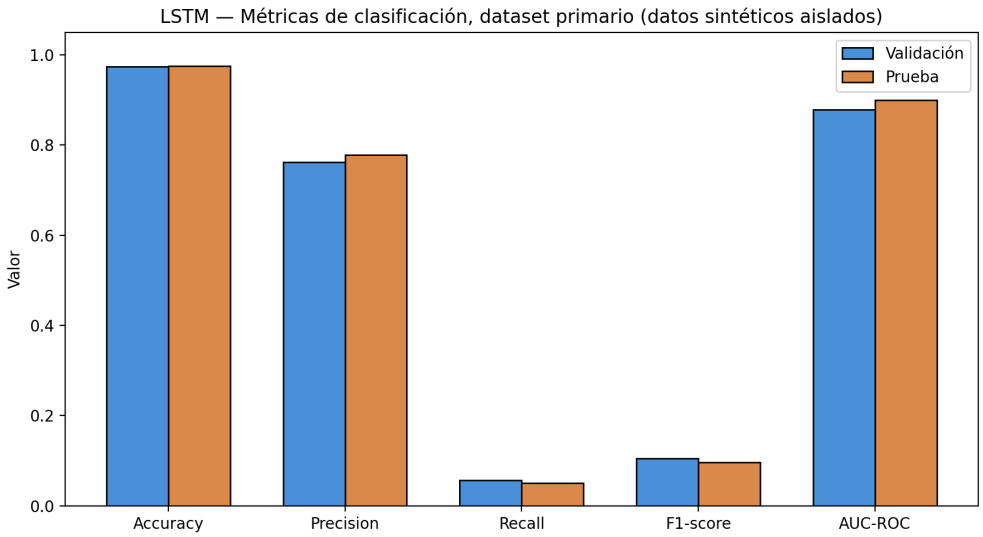


Figura 6.25: Comparación de métricas de clasificación, validación frente a prueba, LSTM sobre el *dataset* primario.

La Figura 6.25 confirma el patrón: exactitud y AUC-ROC altos, y *recall* y *F1-score* cercanos a cero en ambas particiones. La Ecuación 6.5 presenta la matriz de confusión sobre la partición de prueba.

$$\begin{pmatrix} \text{VN} = 10\,314 & \text{FP} = 4 \\ \text{FN} = 261 & \text{VP} = 14 \end{pmatrix} \tag{6.5}$$

Sus componentes se interpretan así:

- **Verdaderos negativos (VN=10 314):** el modelo reconoce correctamente el tráfico normal, coherente con su buen ajuste del régimen ocioso en regresión.
- **Falsos positivos (FP=4):** la tasa de falsas alarmas es de 0.04 % de las observaciones normales, y la precisión es de 77.78 % (14 de 18 alertas fueron correctas).
- **Verdaderos positivos (VP=14):** de 275 instantes de congestión, se detectaron 14. Es mucho menos que los 82 de *Random Forest*, aunque más que el único acierto de ARIMA.
- **Falsos negativos (FN=261):** el 94.9 % de los instantes de congestión pasa sin alerta.

En términos operativos, la red emitió solo 18 alertas en toda la partición. Es un detector casi silencioso: confiable cuando alerta, pero que no avisa en la gran mayoría de las congestiones.

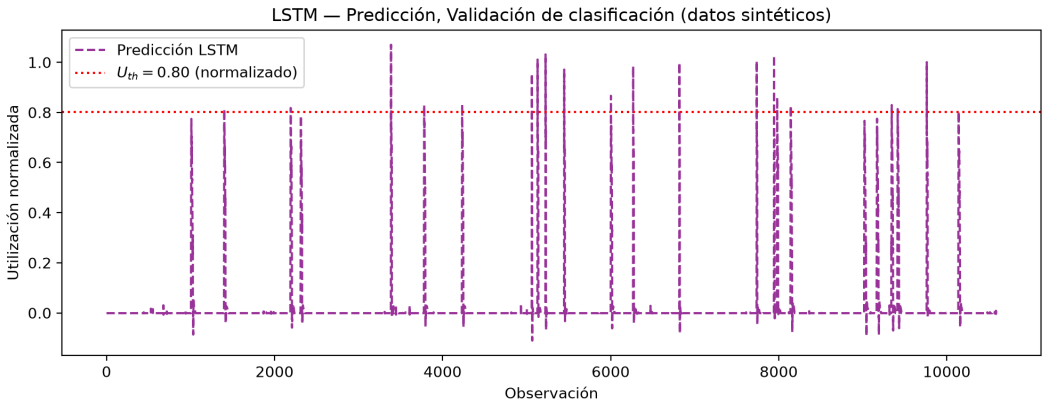


Figura 6.26: LSTM: predicción frente al valor real y umbral $U_{th} = 0{,}80$, validación de clasificación (*dataset* primario, datos sintéticos aislados).

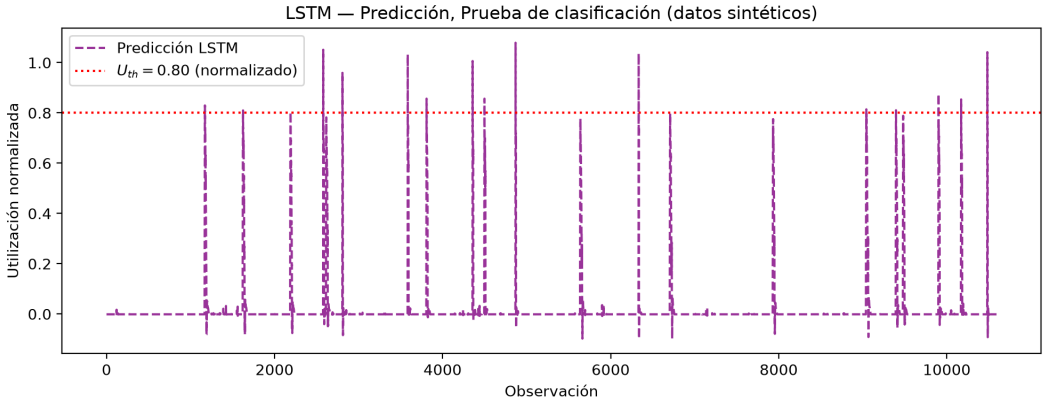


Figura 6.27: LSTM: predicción frente al valor real y umbral $U_{th} = 0{,}80$, prueba de clasificación (*dataset* primario, datos sintéticos aislados).

Las Figuras 6.26 y 6.27 muestran el origen de los falsos negativos. La predicción (línea púrpura discontinua) reacciona ante las anomalías inyectadas y asciende de forma sincronizada con las crisis, pero con magnitud amortiguada: muchos picos se quedan justo por debajo de la línea de alerta (0.80), y solo 14 la cruzan. El patrón es el mismo en validación y en prueba.

La Figura 6.28 sitúa este resultado frente a los otros dos modelos sobre la misma tarea, y la comparativa completa se desarrolla en la Sección 6.4. La comparación es indicativa, no estricta, porque las matrices de confusión de los modelos no se calcularon sobre el mismo número de observaciones.

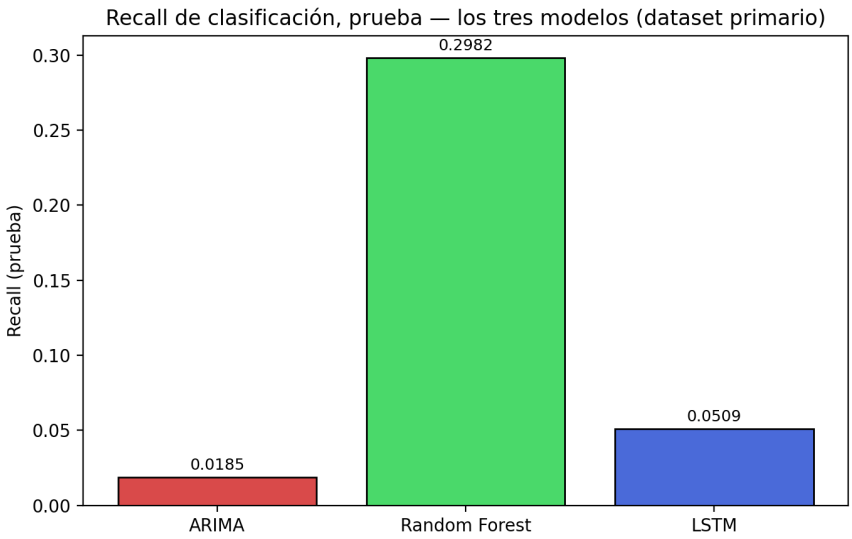


Figura 6.28: *Recall* de clasificación (prueba) de los tres modelos sobre el *dataset* primario.

Piloto frente a *dataset* primario

La Tabla 6.20 resume la comparación de la red entre ambas fases.

Métrica (prueba)	**Piloto (MAWI)**	**Primario**
Secuencias de entrenamiento	1 044	46 582
Épocas hasta la detención	12	16
RMSE (regresión, normalizado)	0.0891	0.001036
Recall (clasificación)	79.4 % ($U_{test} = 0{,}25$)	5.1 % ($U_{th} = 0{,}80$)
F1-score (clasificación)	80.3 % ($U_{test} = 0{,}25$)	9.6 % ($U_{th} = 0{,}80$)
AUC-ROC (clasificación)	N/A	0.9000

Cuadro 6.20: Comparación de LSTM entre la fase piloto y el *dataset* primario. Los RMSE corresponden a series distintas y las métricas de clasificación usan umbrales distintos, por lo que no son directamente comparables. N/A: no aplicable con el umbral oficial (Sección 5.1).

Validación de generalización no realizada

Como *Random Forest*, la red no pudo evaluarse sobre MAWI reagregado a la granularidad oficial de 1 minuto. Su ventana de entrada es de $w = 30$ pasos (Sección 4.5) y exige al menos 30 observaciones consecutivas para generar una predicción, mientras que cada traza de MAWI aporta solo 15 minutos continuos (Sección 5.1). ARIMA, en cambio, opera sobre la serie continua sin requerir una ventana fija, y por eso sí pudo evaluarse. Acortar la ventana alteraría la configuración validada y haría injusta cualquier comparación, por lo que esta validación se reporta como no realizada.

Síntesis de los resultados de LSTM

La Tabla 6.21 reúne las métricas de todos los contextos de evaluación.

Contexto (datos)	**RMSE**	***Recall***	**AUC-ROC**
Piloto, validación 100 % real (MAWI)	0.0860	N/A	N/A
Piloto, prueba 100 % real (MAWI)	0.0891	N/A	N/A
Primario, regresión (val.) 100 % real	0.000131	—	—
Primario, regresión (prueba) 100 % real	0.001036	—	—
Primario, clasificación (val.) 100 % sintético	—	0.0563	0.8783
Primario, clasificación (prueba) 100 % sintético	—	0.0509	0.9000
Generalización, MAWI (oficial) 100 % real (MAWI)	No evaluable (incompatibilidad estructural)		

Cuadro 6.21: Síntesis de los resultados de LSTM. N/A: no aplicable con el umbral oficial (Sección 5.1). Los RMSE están en escala normalizada.

Discusión de los resultados de LSTM

Regresión. Sobre el *dataset* primario, la red obtiene el menor error de los tres modelos, con una diferencia de dos órdenes de magnitud en el MAE y de más de uno en el RMSE. Esa ventaja debe leerse con cautela. El tráfico real es casi nulo (media de $2{,}2 \times 10^{-5}$ y desviación estándar de $3{,}5 \times 10^{-4}$ en fracción, Tabla 5.2), de modo que cualquier predicción pegada a cero tiene un error pequeño. ARIMA y *Random Forest* quedan muy por detrás sobre todo porque presentan sesgo de sobreestimación y picos espurios, respectivamente, no porque la red capte una dinámica que los otros ignoran. Para sustentar una ventaja predictiva real habría que compararla con una línea base ingenua (por ejemplo, repetir el último valor o predecir la media), que no se evaluó en este trabajo. De hecho, el RMSE de prueba (0.001036) es aproximadamente tres veces la desviación estándar de toda la serie real, lo que muestra el peso de los pocos picos espurios, aunque la red los limita mucho más que los otros modelos. Una explicación plausible es que el estado interno de la red atenúa las fluctuaciones aisladas, pero con estos datos no puede demostrarse.

Piloto. En el piloto la red es comparable a ARIMA en regresión (RMSE reescalado de 0.027 frente a 0.0282, y MAPE de 11.00 % frente a 10.80 %) y mejor que *Random Forest*. Su mayor fortaleza está en la clasificación con umbral bajo: *recall* de 79.4 % y TAE de 3.9 minutos, el mejor de los tres modelos. Estas cifras, sin embargo, se obtienen con $U_{test} = 0{,}25$ sobre otra serie, y no anticipan el comportamiento con el umbral oficial.

Clasificación en el primario. El resultado contrasta con el piloto: el *recall* cae a 5.1 %, menor que el del ensamble de árboles. Varias causas, no excluyentes, podrían explicarlo. La

primera es la función de pérdida: al minimizar el MSE sobre un entrenamiento con el 97.9 % de las observaciones por debajo del umbral (Sección 5.1), el error global se reduce sobre todo prediciendo bien el régimen mayoritario, y la curva de entrenamiento, que se estabiliza en 0.006, sugiere que los eventos quedan ajustados solo en parte. La segunda es el criterio de detención temprana, que se guía por la validación de regresión (100 % tráfico real ocioso): conserva los pesos que mejor predicen el tráfico ocioso, no los que mejor detectan congestión, y la curva sugiere que esos pesos son los de una etapa muy temprana del entrenamiento. La tercera es la distancia entre la magnitud predicha y el umbral fijo de 0.80, evidente en las Figuras 6.26 y 6.27. Con los experimentos realizados no se puede aislar cuánto aporta cada una.

Umbral frente a señal. El AUC-ROC de la red (0.90) es casi idéntico al de ARIMA (0.89) y al de *Random Forest* (0.92), a pesar de que sus *recall* difieren en más de un orden de magnitud. Esto indica que las tres predicciones continuas ordenan de forma parecida los instantes de congestión por encima de los normales, y que la diferencia en *recall* proviene sobre todo de la amplitud de las predicciones frente al umbral fijo. Por tanto, la calibración del umbral sobre la partición de validación, o una función de pérdida que pondere los eventos, son vías para mejorar la detección que este trabajo no exploró. Con un desbalance tan fuerte, convendría además complementar el AUC-ROC con una curva precisión-*recall*.

Generalización. La imposibilidad de evaluar la red sobre MAWI, igual que en *Random Forest*, tiene origen en la ventana de entrada de 30 pasos frente a trazas de 15 minutos. Es una consecuencia de la configuración elegida y no una propiedad de los algoritmos, pero limita la comparación con ARIMA, que fue el único modelo evaluable en este escenario.

Alcance. En conjunto, la red ajusta mejor que los otros modelos el tráfico real ocioso, con picos espurios muy acotados, pero con el umbral oficial detecta menos congestión que *Random Forest*. Estas conclusiones se integran con las de los otros modelos en las Secciones 6.4 y 6.5.

6.4. Comparativa de modelos

Las Secciones 6.1 a 6.3 presentaron los resultados de cada modelo por separado. Esta sección los coloca lado a lado para responder una pregunta: ¿existe un modelo que domine de manera uniforme, o el desempeño relativo depende de la tarea, del conjunto de datos y de las condiciones de evaluación (Sección 4.6)? Aquí se describe la comparación y su lectura inmediata; la interpretación causal y sus implicaciones se desarrollan en la Sección 6.5.

Alcance de la comparación

Los tres modelos se evaluaron con el mismo umbral operativo, las mismas particiones de datos y las mismas métricas, pero no en condiciones idénticas. La Tabla 6.22 resume las diferencias que conviene tener presentes al leer las cifras.

Aspecto	**ARIMA**	***Random Forest***	**LSTM**
Entrada	Serie continua (sesión sintética de 31 934 obs.)	5 rezagos y media móvil de 30 minutos	Ventana de 30 pasos
Escala de las métricas	Original	Normalizada (Min-Max)	Normalizada (Min-Max)
Observaciones de prueba en clasificación (positivos)	2 124 (54)	10 622 (275)	10 593 (275)

Cuadro 6.22: Condiciones de evaluación de cada modelo en el *dataset* primario.

Como el rango de entrenamiento está cerca de $[0, 1]$, la escala normalizada es aproximadamente equivalente a la original en el *dataset* primario, por lo que sus RMSE pueden compararse de forma razonable. En el piloto, los RMSE de *Random Forest* y LSTM se reescalan multiplicándolos por el rango de la variable (0.306). Además, ARIMA se evaluó con pronóstico progresivo por bloques, un esquema que introduce oscilaciones propias en cada reinicio (Sección 6.1). Cada modelo se entrenó una sola vez, sin repeticiones ni intervalos de confianza, de modo que las diferencias pequeñas deben tomarse como indicativas.

Tabla maestra de resultados

La Tabla 6.23 reúne las métricas principales de los tres modelos en todos los contextos de evaluación.

Contexto (datos)	**Modelo**	**RMSE**	***Recall***	**AUC-ROC**
Piloto, prueba MAWI, 100 % real	ARIMA	0.0282	N/A	N/A
	Random Forest	≈ 0.040	N/A	N/A
	LSTM	≈ 0.027	N/A	N/A
Primario, regresión, prueba 100 % real	ARIMA	0.0459	—	—
	Random Forest	0.0496	—	—
	LSTM	**0.0010**	—	—
Primario, clasificación, prueba 100 % sintético	ARIMA	—	0.0185	0.8939
	Random Forest	—	**0.2982**	0.9187
	LSTM	—	0.0509	0.9000
Generalización, 1 min MAWI, 100 % real	ARIMA	0.3233	N/A	N/A
	Random Forest	No evaluable		
	LSTM	No evaluable		

Cuadro 6.23: Tabla maestra de resultados de los tres modelos. Los RMSE de *Random Forest* y LSTM en el piloto están reescalados a la escala original. N/A: no aplicable con el umbral oficial (Sección 5.1).

La tabla muestra que ningún modelo domina en todos los contextos. En regresión sobre el piloto, ARIMA y LSTM son equivalentes y *Random Forest* queda detrás. En regresión sobre el *dataset* primario, LSTM obtiene el menor error. En clasificación sobre el primario, *Random Forest* es el que más detecta. En generalización, solo ARIMA pudo evaluarse. Las subsecciones siguientes detallan cada contexto.

Fase piloto

La Tabla 6.24 compara los tres modelos en MAWI. La clasificación usa el umbral relativo de prueba $U_{test} = 0{,}25$, y el Tiempo de Anticipación Efectivo (TAE) solo se calculó en esta fase.

Modelo	**RMSE**	**MAPE**	**Precisión**	***Recall***	***F1-score***	**TAE**
ARIMA(1,1,1)	0.0282	10.80 %	64.2 %	41.5 %	50.4 %	1.2 min
Random Forest	≈ 0.040	16.26 %	76.5 %	68.3 %	72.1 %	2.8 min
LSTM	≈ 0.027	11.00 %	81.2 %	79.4 %	80.3 %	3.9 min

Cuadro 6.24: Desempeño de los tres modelos en la fase piloto (partición de prueba, MAWI). Clasificación con $U_{test} = 0{,}25$.

En regresión, ARIMA y LSTM son equivalentes (RMSE de 0.0282 y de aproximadamente 0.027, MAPE de 10.80 % y 11.00 %), mientras que *Random Forest* presenta un RMSE un 42 % mayor que el de ARIMA y el MAPE más alto. En clasificación, el orden es el mismo en todas las métricas: LSTM supera a *Random Forest* y este a ARIMA. El TAE de LSTM (3.9 minutos) equivale al 78 % del horizonte objetivo de 5 minutos, frente al 56 % de *Random Forest* y el 24 % de ARIMA. Este orden no se reproduce con el umbral oficial en el *dataset* primario, como se ve a continuación, y proviene de una sola partición con un umbral bajo.

Dataset primario: regresión

Sobre las particiones reales de prueba, los RMSE son 0.0459 (ARIMA), 0.0496 (*Random Forest*) y 0.0010 (LSTM), y los MAE, 0.0151, 0.0103 y 0.000125 respectivamente. LSTM presenta un RMSE unas 44 veces menor que el de ARIMA y unas 48 veces menor que el de *Random Forest*, y un MAE unas 120 y 82 veces menor. Entre ARIMA y *Random Forest*, el orden depende de la métrica: ARIMA tiene menor RMSE, mientras que *Random Forest* tiene menor MAE. El RMSE penaliza más los errores grandes, y los de cada modelo tienen un origen distinto: los picos de hasta 0.7 de *Random Forest* y las oscilaciones por bloques de ARIMA (Secciones 6.2 y 6.1).

La magnitud de esta ventaja debe leerse con cautela. El tráfico real es casi nulo, con una desviación estándar de aproximadamente $3{,}5 \times 10^{-4}$ en fracción (Tabla 5.2), y una predicción constante igual a la media tendría un RMSE del orden de esa desviación. Con esa referencia, el RMSE de LSTM es unas tres veces mayor y los de ARIMA y *Random Forest*, más de cien veces mayores. Es una referencia aproximada, porque proviene de la serie completa y no de la partición de prueba. El control que falta para valorar la ventaja en términos absolutos es calcular líneas base ingenuas (media y persistencia) sobre esa partición. Por ahora, el resultado respalda que LSTM es el modelo que mejor evita los errores grandes sobre tráfico ocioso, no que capte una dinámica que los otros ignoran.

Dataset primario: clasificación

La Tabla 6.25 detalla la clasificación sobre las particiones sintéticas de prueba con el umbral oficial $U_{th} = 0{,}80$.

Modelo	VP	FP	FN	Exactitud	Precisión	*Recall*	*F1-score*	AUC-ROC
ARIMA	1	0	53	0.9750	1.0000	0.0185	0.0364	0.8939
Random Forest	82	6	193	0.9813	0.9318	**0.2982**	**0.4518**	0.9187
LSTM	14	4	261	0.9750	0.7778	0.0509	0.0956	0.9000

Cuadro 6.25: Clasificación con $U_{th} = 0{,}80$ en la partición sintética de prueba. VP, FP y FN provienen de las matrices de confusión de las Secciones 6.1, 6.2 y 6.3.

Random Forest emitió 88 alertas, con 82 aciertos. LSTM emitió 18, con 14 aciertos, y ARIMA, una. Los tres casi no generan falsas alarmas (entre 0 y 6), pero difieren en cuántos eventos detectan. El *recall* de *Random Forest* es unas 16 veces el de ARIMA y unas 6 veces el de LSTM, y aun así deja sin alerta el 70 % de los instantes de congestión. ARIMA y LSTM prácticamente no detectan. La diferencia entre ambos (1 acierto de 54 frente a 14 de 275) es pequeña frente a la que ambos tienen con *Random Forest*, y la precisión perfecta de ARIMA se apoya en una única alerta.

El AUC-ROC es parecido en los tres modelos (de 0.89 a 0.92), aunque sus *recall* difieren en más de un orden de magnitud. Esto indica que las tres predicciones continuas ordenan de forma semejante los instantes de congestión por encima de los normales, y que la diferencia se produce en cuántas predicciones cruzan el umbral fijo de 0.80. Con un desbalance tan marcado, conviene complementar el AUC-ROC con una curva de precisión-*recall*, que no se evaluó en este trabajo.

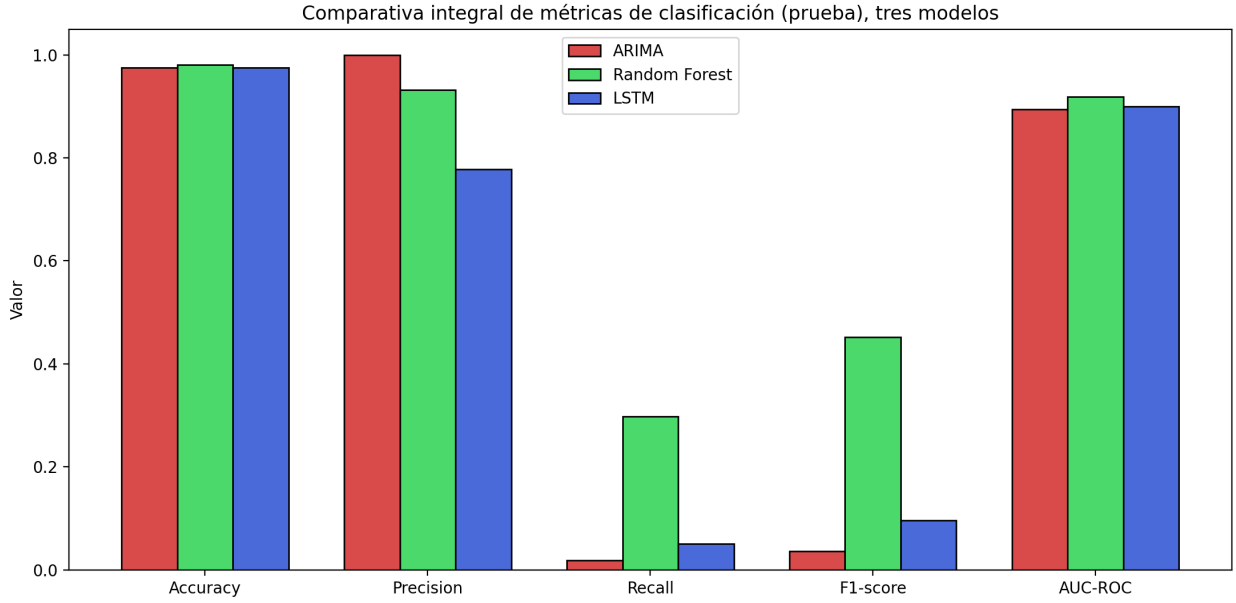


Figura 6.29: Métricas de clasificación de los tres modelos en el *dataset* primario (partición de prueba).

La Figura 6.29 muestra que la exactitud es casi la misma en los tres modelos (de 0.975 a 0.981), mientras que el *recall* y el *F1-score* varían en órdenes de magnitud. Con una proporción de casos de congestión cercana al 2.6 % (Sección 5.1), un clasificador que nunca alertara

obtendría una exactitud de aproximadamente 0.974. La exactitud de los modelos supera esa referencia por entre 0.05 puntos porcentuales (ARIMA) y 0.72 (*Random Forest*), por lo que no distingue entre un detector útil y uno que no detecta. Esto confirma la advertencia de la Sección 4.7: en este problema la evaluación debe basarse en la precisión, el *recall* y el *F1-score*.

Disociación entre error continuo y sensibilidad de alertamiento

La Figura 6.30 ubica a cada modelo según su RMSE de regresión (eje horizontal, escala logarítmica, menor es mejor) y su *recall* de clasificación (eje vertical, mayor es mejor), ambos sobre el *dataset* primario. Los dos ejes provienen de particiones distintas: la regresión, de tráfico real y la clasificación, de tráfico sintético.

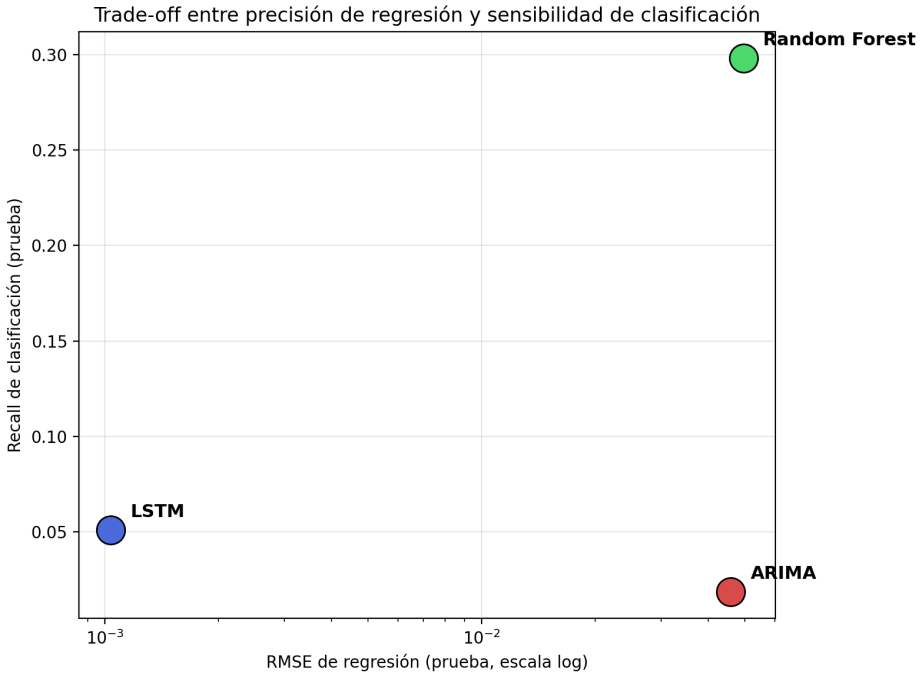


Figura 6.30: RMSE de regresión (eje X) frente a *recall* de clasificación (eje Y), *dataset* primario, partición de prueba.

LSTM tiene el menor RMSE y *Random Forest*, el mayor *recall*, y ninguno supera al otro en ambos ejes, por lo que forman la frontera de mejores compromisos. ARIMA queda dominado por LSTM, que tiene a la vez menor error y mayor *recall*. Frente a *Random Forest*, ARIMA tiene un RMSE ligeramente menor (0.0459 frente a 0.0496), pero un *recall* unas 16 veces inferior. En el *dataset* primario, entonces, el modelo que mejor ajusta el tráfico ocioso no es el que más detecta congestión. Esta disociación no es general: en el piloto (Tabla 6.24) LSTM fue el mejor detector con el umbral de prueba. Por qué ocurre en el primario se discute en la Sección 6.5.

Comparación multidimensional

La Figura 6.31 reúne cinco dimensiones del *dataset* primario: capacidad de regresión (invertida y normalizada, de modo que un valor periférico indica un RMSE cercano a cero), precisión, *recall*, *F1-score* y AUC-ROC.

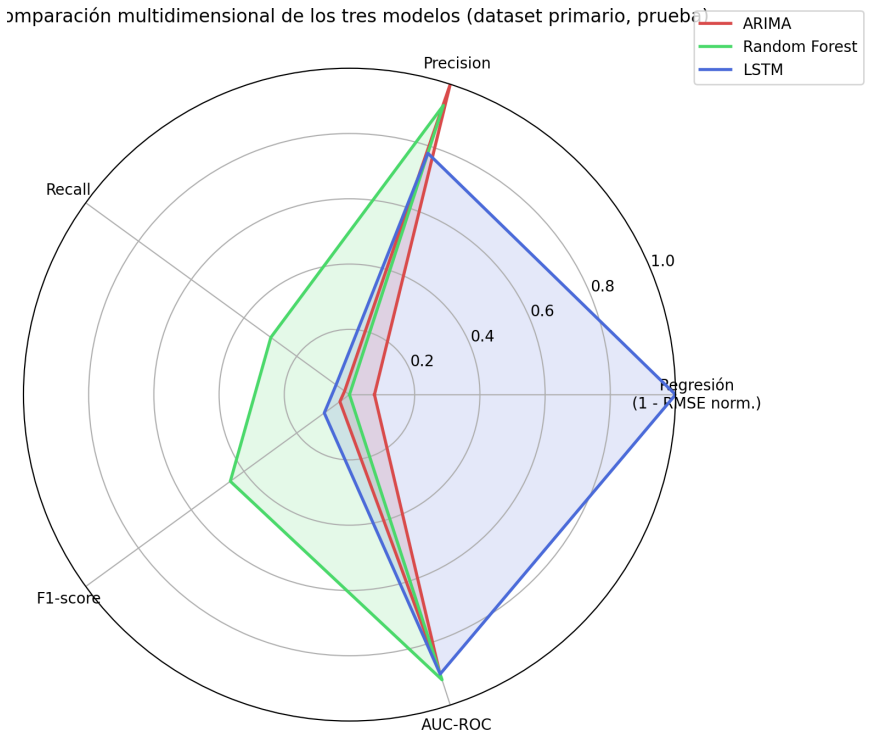


Figura 6.31: Comparación multidimensional de los tres modelos (*dataset* primario, partición de prueba).

Los perfiles son distintos. LSTM se extiende hacia el eje de regresión y mantiene un AUC-ROC alto, pero se contrae en *recall* y *F1-score*. *Random Forest* presenta la geometría más equilibrada: no domina la regresión, pero obtiene el mayor *recall* y el mayor *F1-score* (0.4518), y es el único con desempeño aceptable en ambas tareas. ARIMA casi no tiene área útil, salvo un máximo en precisión que se debe a su única alerta (Tabla 6.25).

Generalización entre escalas de red

Solo ARIMA pudo evaluarse sobre MAWI reagregado a granularidad de 1 minuto. Su RMSE (0.3233) fue el peor de toda la investigación, y superó al de su propio desempeño en el *dataset* primario (0.0459) y en el piloto (0.0282); además, se calculó sobre seis observaciones, por lo que es indicativo y no concluyente. *Random Forest* y LSTM no pudieron evaluarse porque exigen entradas que una traza de MAWI no puede proporcionar: la media móvil de 30

minutos en el primero y la ventana de 30 pasos en el segundo, frente a trazas de 15 minutos continuos (Secciones 6.2 y 6.3). Por tanto, no hay evidencia en este trabajo que respalde trasladar los modelos a infraestructuras de escala distinta sin reentrenarlos y rediseñar sus características. Como solo un modelo pudo evaluarse, esta dimensión no permite comparar a los tres.

Síntesis de la comparativa

De la comparación se desprenden cuatro puntos. Primero, ningún modelo domina todos los contextos: LSTM es el mejor en regresión sobre el primario y en clasificación con umbral bajo sobre el piloto, *Random Forest* es el mejor detector con el umbral oficial, y ARIMA es competitivo solo en regresión sobre el piloto. Segundo, la exactitud no distingue entre modelos en este problema. Tercero, el AUC-ROC similar de los tres sugiere que la diferencia en *recall* proviene sobre todo de cómo cada predicción se sitúa respecto al umbral. Cuarto, la generalización solo pudo evaluarse en ARIMA, con resultado desfavorable.

Esta comparación atiende la brecha metodológica de la Sección 3.5, que señalaba la ausencia de estudios que contrasten los tres paradigmas sobre los mismos datos. Con matices: las condiciones son comparables, pero no idénticas (Tabla 6.22), la clasificación se evaluó sobre eventos sintéticos y cada modelo se entrenó una sola vez. Estos hallazgos se interpretan, a la luz del problema de investigación, en la Sección 6.5.

6.5. Discusión

Esta sección interpreta los resultados de las Secciones 6.1 a 6.4 a la luz del problema de investigación (Capítulo 1), el marco teórico (Capítulo 2) y la literatura revisada (Capítulo 3), y discute las limitaciones del diseño experimental como base para las conclusiones del Capítulo 7. Las cifras ya presentadas no se repiten.

Retorno al problema de investigación

El Capítulo 1 formuló el problema como una pregunta: *¿Es posible, a partir del análisis de datos históricos de operación de una red, construir un modelo capaz de predecir con suficiente antelación que un enlace alcanzará un estado de congestión, de tal forma que dicha predicción resulte útil para la toma de decisiones antes de que la degradación del servicio se materialice?* La evidencia permite una respuesta matizada, en tres partes:

- **Predicción continua.** LSTM obtuvo un RMSE de 0.0010 sobre tráfico real no visto en el entrenamiento (Sección 6.3). Sin embargo, como el tráfico real es casi nulo, esa cifra no demuestra por sí sola una capacidad predictiva superior a la de una predicción trivial (Sección 6.4).
- **Detección del umbral.** Con $U_{th} = 0{,}80$, el mejor *recall* fue 29.82 % (*Random Forest*, Tabla 6.25): siete de cada diez instantes de congestión quedaron sin alerta, y esto sobre eventos sintéticos.

- **Antelación.** El Tiempo de Anticipación Efectivo (TAE) solo se calculó en la fase piloto, con el umbral de prueba $U_{test} = 0{,}25$: 1.2, 2.8 y 3.9 minutos para ARIMA, *Random Forest* y LSTM (Tabla 6.24). No se midió con el umbral oficial ni en el *dataset* primario, por lo que la parte de la pregunta sobre la antelación queda respaldada solo en el piloto.

Como se argumentó en la Sección 4.7, la investigación priorizó el *recall* por la asimetría de costos: un falso negativo (una congestión no anticipada) reproduce la limitación reactiva descrita en el Planteamiento del Problema, mientras que un falso positivo es una alerta injustificada. Los tres modelos produjeron muy pocas falsas alarmas (entre 0 y 6) con precisiones de 0.78 a 1.00 y un *recall* máximo de 0.30. Esto sugiere que el punto de operación es muy conservador y que existe margen para ganar *recall* a costa de precisión, aunque cuánto no se evaluó. El resultado no es un fracaso del diseño, pero señala con precisión cuánto falta para un despliegue autónomo con confianza operativa.

Interpretación de la disociación entre regresión y clasificación

En el *dataset* primario, el modelo con menor error continuo (LSTM) no es el que más detecta (*Random Forest*), como mostró la Figura 6.30. En la literatura revisada no se encontró un estudio que contrastara explícitamente ambos desempeños, ya que la mayoría reporta solo error continuo (Sección 3.5). La Tabla 6.26 relaciona este resultado con lo anticipado en capítulos anteriores.

Fuente	**Lo que anticipaba**	**Lo que esta tesis encontró**
Sección 4.6 (justificación de modelos)	LSTM, al capturar dependencias de largo plazo, tendría ventaja sobre ARIMA y *Random Forest*.	LSTM es el mejor en regresión sobre el primario y en clasificación con umbral bajo en el piloto, pero con el umbral oficial detecta menos que *Random Forest*.
Oliveira et al. [20]	Mayor complejidad arquitectónica no garantiza mejor desempeño.	Consistente: LSTM, el modelo más complejo, no es el mejor detector con el umbral oficial.
Santos et al. [22]	La superioridad de un modelo depende del tráfico analizado.	Consistente: el orden entre modelos cambia entre el piloto y el primario.
Sección 4.7 (advertencia sobre la exactitud)	Con desbalance de clases, la exactitud oculta el desempeño real.	La exactitud (0.975 a 0.981) supera a la de un clasificador que nunca alerta en menos de 0.8 puntos porcentuales, mientras el *recall* difiere hasta 16 veces (Figura 6.29).

Cuadro 6.26: Relación de los hallazgos con el marco teórico, la literatura y la metodología.

Los experimentos realizados no permiten establecer la causa de la disociación, pero sí identificar varios factores plausibles y qué haría falta para confirmarlos (Tabla 6.27).

Factor	Evidencia en los resultados	Cómo confirmarlo
Pérdida MSE con 97.9 % de observaciones bajo el umbral	Los picos predichos por LSTM y *Random Forest* quedan justo por debajo de 0.80 (Secciones 6.2 y 6.3); la pérdida de entrenamiento de LSTM se estabiliza en torno a 0.006.	Entrenar con pérdida ponderada o con pesos por clase.
Detención temprana guiada por la validación de regresión (100 % tráfico real ocioso)	La pérdida de validación alcanza su mínimo en las primeras épocas (Figura 6.19); se conservan pesos que ajustan el tráfico ocioso.	Guiar la detención con una métrica de clasificación.
Umbral fijo sobre predicciones amortiguadas	AUC-ROC similar en los tres modelos (0.89 a 0.92) con *recall* muy distinto.	Calibrar el umbral sobre la validación y evaluar curvas de precisión-*recall*.
Composición asimétrica entre entrenamiento (real y sintético) y evaluación	Sesgo de sobreestimación de ARIMA ($\mu = 0{,}0324$ frente a una media real del orden de 10^{-5}) y picos de hasta 0.7 de *Random Forest* sobre tráfico real ocioso.	Inspeccionar las entradas que generan los picos; entrenar solo con datos reales o solo sintéticos.
Esquema de evaluación por bloques (ARIMA)	Oscilaciones regulares al inicio de cada bloque (Figura 6.2).	Evaluar sin reinicios de estado.

Cuadro 6.27: Factores plausibles de la disociación entre regresión y clasificación. Ninguno se verificó experimentalmente.

Además, algunos resultados específicos admiten más de una lectura. En el piloto, el piso de la predicción de *Random Forest* frente a las caídas profundas es coherente con el promediado de las hojas, pero esas caídas están dentro del rango de entrenamiento, por lo que no es extrapolación en sentido estricto y podría deberse a su baja representación en la partición de entrenamiento. La inversión de la importancia de variables (de la media móvil al rezago inmediato) puede deberse a la granularidad de muestreo o a la composición del entrenamiento, que incluye eventos sintéticos de perfil trapezoidal, y estos experimentos no permiten separar ambos efectos. Además, la importancia por impureza favorece a las variables con muchos valores distintos, por lo que sus cifras son orientativas.

La congestión como fenómeno raro

La utilización real máxima de 1.44 % durante los 21 días (Sección 5.1) no contradice el fundamento teórico: según la teoría de colas [6], el retraso y la pérdida crecen de forma no lineal, proporcional a $\frac{1}{1-\rho}$, solo cerca de la saturación ($\rho \to 1$). Con utilización baja, la red no manifiesta síntomas, como confirman el RTT estable y la ausencia de pérdidas. Esa misma rareza justificó inyectar eventos sintéticos, sin los cuales ningún modelo habría tenido positivos de los cuales aprender. Conviene precisar que el 2.6 % de casos positivos no es una medida de la frecuencia natural de la congestión en esta red, que fue cero en el periodo

observado, sino una proporción fijada por el diseño de la inyección. Por eso la precisión y el *F1-score*, que dependen de la prevalencia, no son directamente trasladables a la red real.

Implicaciones operativas

A modo ilustrativo, un detector con *recall* adecuado y un TAE del orden de 3 minutos, como el observado en el piloto, abriría una ventana para actuar antes de la degradación. Con $U(t+h) \geq 0{,}80$ como condición de alerta, el administrador, o un *script* integrado al *router* MikroTik, podría ejecutar acciones escalonadas:

1. **QoS dinámico:** aplicar *Simple Queues* o *Queue Trees* en RouterOS para priorizar el tráfico crítico (por ejemplo, telefonía IP o bases de datos) y limitar el de fondo (por ejemplo, la sincronización de Nextcloud).

2. **Enrutamiento de respaldo:** si la alerta persiste, desviar el tráfico no esencial a una interfaz WAN secundaria.

3. **Degradación controlada:** reducir temporalmente el *bitrate* de las cámaras de videovigilancia, la acción más costosa ante una falsa alarma y, por tanto, reservada a alertas persistentes.

Con los resultados actuales, este protocolo es una propuesta y no una consecuencia demostrada: el *recall* con el umbral oficial es bajo y el TAE solo se midió en el piloto.

Líneas de mejora y diseño híbrido

La Tabla 6.27 apunta a mejoras concretas que este trabajo no exploró: calibrar el umbral sobre la validación, usar una pérdida ponderada por clase, guiar la detención temprana con una métrica de clasificación, incorporar líneas base ingenuas y curvas de precisión-*recall*, repetir los entrenamientos con distintas semillas y obtener eventos reales con una recolección más larga.

Una idea adicional es un esquema híbrido (Figura 6.32) en el que *Random Forest*, el modelo de mayor *recall*, dispare la alerta y LSTM aporte una estimación de la magnitud esperada. La primera parte cuenta con respaldo en los resultados, pero la segunda no: los picos que LSTM predice sobre los eventos son de menor magnitud que los reales, y su bajo error continuo corresponde a tráfico ocioso y no a la magnitud de los eventos. Por ello, la propuesta es una hipótesis para el trabajo futuro (Capítulo 7), que debería comprobar primero si algún modelo estima bien la magnitud de los eventos.

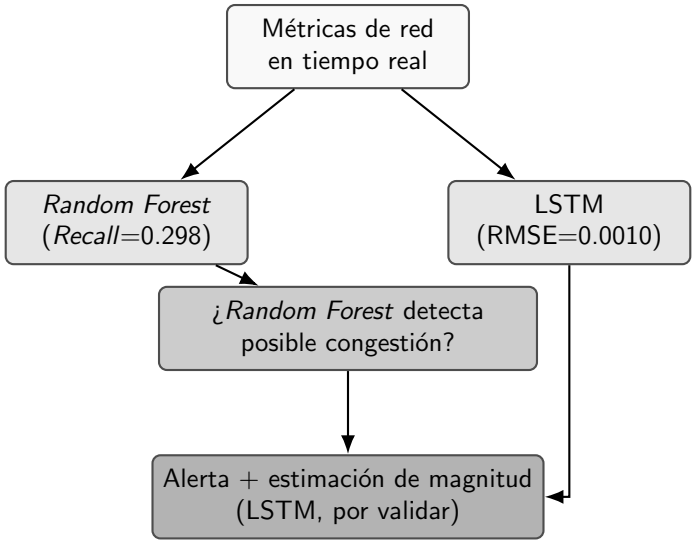


Figura 6.32: Esquema conceptual de un sistema de alertamiento híbrido. No fue implementado ni evaluado en este trabajo.

Cierre de las brechas identificadas en el Estado del Arte

La Tabla 6.28 retoma las tres brechas de la Sección 3.5 y las contrasta con la evidencia generada.

Brecha (Sección 3.5)	**Cómo se abordó**	**Evidencia generada**
Conceptual: predicción de volumen frente a congestión como evento discreto.	Se definió una variable objetivo de clasificación ($U \geq U_{th}$) evaluada junto con la regresión.	Tablas 6.6, 6.12, 6.19 y 6.25.
Metodológica: ausencia de comparación simultánea entre paradigmas.	Se compararon los tres paradigmas con las mismas particiones, el mismo umbral y las mismas métricas, en condiciones comparables pero no idénticas.	Tablas 6.23 y 6.22.
De validación: dependencia de una única fuente de datos.	Se adoptó un *dataset* híbrido con validación externa frente a MAWI.	Sección 5.1; solo ARIMA pudo evaluarse en MAWI (Sección 6.4).

Cuadro 6.28: Cierre de las brechas del Estado del Arte a la luz de los resultados del Capítulo 6.

Las tres brechas se abordaron, la metodológica con los matices indicados y la de validación solo parcialmente. Este último punto delimita dónde debería concentrarse una extensión de este trabajo.

Limitaciones reconocidas

1. **Periodo y cobertura de la recolección.** Se dispuso de 21 días efectivos, con una cobertura del 82.9 % (Tabla 5.1). Un periodo más largo habría permitido observar ciclos estacionales completos y, posiblemente, episodios de congestión reales.

2. **Dependencia de datos sintéticos para la clasificación.** Toda la evidencia de clasificación proviene de eventos inyectados, con perfil trapezoidal y una prevalencia fijada por el diseño. Los eventos reales pueden ser menos regulares, y las métricas que dependen de la prevalencia (precisión y *F1-score*) no son directamente trasladables.

3. **Fuente única de telemetría.** El *dataset* primario proviene de un solo *router* en un entorno PYME, con tráfico de baja utilización característico de ese despliegue.

4. **Diseño de la evaluación.** Cada modelo se entrenó una sola vez, sin repeticiones ni intervalos de confianza, y la evaluación usó una partición cronológica 60/20/20 sin validación cruzada. No se calcularon líneas base ingenuas ni se calibró el umbral de decisión. Las condiciones de ARIMA, *Random Forest* y LSTM no son idénticas en escala, entrada y tamaño de la partición de clasificación (Tabla 6.22).

5. **Generalización.** Solo ARIMA pudo evaluarse frente a MAWI, con seis observaciones; *Random Forest* y LSTM no pudieron, por la incompatibilidad entre sus ventanas de entrada y la duración de las trazas. Los hallazgos son válidos para la infraestructura estudiada y no deben extrapolarse sin reentrenar.

6. **Alcance de los modelos.** Se evaluaron ARIMA, *Random Forest* y LSTM. Otros modelos considerados en la revisión, como GRU o SVR, no se evaluaron.

Capítulo 7

Conclusiones

El presente trabajo de tesis se propuso, desde su planteamiento original (Sección 1.1), resolver una problemática de ingeniería fundamental: evaluar la viabilidad de construir un sistema predictivo capaz de anticipar la congestión en una red de datos, habilitando con ello la transición de una administración de infraestructura empírica y reactiva hacia una gestión algorítmica y preventiva.

A lo largo de los capítulos precedentes, este desafío fue delimitado conceptualmente mediante la formalización operativa de la congestión (Sección 2.1.3) y la identificación de brechas críticas en la literatura (Sección 3.5). Posteriormente, fue abordado mediante un diseño metodológico riguroso que combinó la recolección de telemetría real con la inyección calibrada de anomalías sintéticas para subsanar el desbalance de clases (Sección 5.1). Finalmente, el problema fue sometido a escrutinio empírico a través del diseño, entrenamiento y evaluación comparativa exhaustiva de tres paradigmas de modelado: la estadística clásica autorregresiva (ARIMA), los ensambles de árboles de decisión (*Random Forest*) y las redes neuronales recurrentes de memoria a corto y largo plazo (LSTM) (Capítulos 5 y 6).

Corresponde a este capítulo final desempeñar una función de síntesis analítica: no generar nueva evidencia empírica, sino articular, con rigor y honestidad intelectual, las conclusiones definitivas que dicha evidencia permite sostener. Para lograr este cometido, el capítulo se estructura en dos ejes fundamentales.

En primera instancia, la Sección 7.1 detalla las contribuciones concretas que esta investigación aporta tanto a la comunidad académica como a la práctica profesional de la ingeniería de redes, diferenciando sistemáticamente entre hallazgos metodológicos, empíricos y operativos. Finalmente, la Sección 7.2 expone con total transparencia las limitaciones inherentes al diseño experimental implementado —muchas de ellas anticipadas en los alcances originales de la tesis (Sección 1.4) y confirmadas algorítmicamente en la fase de generalización—. A partir de estas restricciones, se formulan líneas directas de trabajo futuro, garantizando que los límites topológicos de este estudio no representen un cierre definitivo, sino el cimiento legítimo para la evolución continua de los sistemas predictivos de red.

7.1. Contribuciones

Más allá de la demostración de los paradigmas predictivos, esta investigación aporta valor empírico, metodológico y operativo —atribuible directamente a las decisiones de diseño del autor— a la comunidad académica y a la práctica profesional de la administración de redes. Dichas contribuciones se estructuran en las siguientes tres vertientes:

Contribuciones metodológicas

- **Estrategia reproducible de aumento de datos sin contaminación (*Data Leakage*):** Ante la infrecuencia empírica de la congestión severa, se diseñó e implementó un esquema de inyección sintética calibrada estadísticamente. Esta inyección incluye aserciones matemáticas en tiempo de ejecución que garantizan el aislamiento absoluto entre las particiones de entrenamiento y evaluación, subsanando una vulnerabilidad metodológica recurrente en la literatura del campo.

- **Fundamentación de la métrica objetivo en la Teoría de Colas:** A diferencia de enfoques predictivos agnósticos al contexto, la variable objetivo de congestión se operacionalizó basándose en el comportamiento asintótico no lineal del retraso postulado por Kleinrock. Esta definición unificó los criterios de evaluación y guio matemáticamente la inyección sintética, cerrando el ciclo entre el marco teórico y la experimentación empírica.

Contribuciones empíricas

- **Descubrimiento del *Trade-off* predictivo entre arquitecturas:** Al someter los tres paradigmas algorítmicos (Estadístico, Ensambles y *Deep Learning*) a condiciones experimentales idénticas, se demostró empíricamente la disociación matemática entre regresión continua y clasificación binaria. Se evidenció que la LSTM dominó la precisión del flujo basal, mientras que *Random Forest* dominó la sensibilidad de alertamiento. Este hallazgo refuta la superioridad infalible frecuentemente atribuida a las redes neuronales en la literatura general.

- **Estudio de caso en topologías asimétricas de pequeña escala:** Se documenta de forma transparente el comportamiento de una red PYME real (telemetría, videovigilancia y sincronización en la nube), caracterizada por un régimen de letargo extremo (utilización ≤ 1.44 %). Este esfuerzo enriquece un campo académico históricamente sesgado hacia infraestructuras troncales masivas o entornos de laboratorio hiper-controlados.

- **Identificación de límites de generalización en repositorios públicos:** La evaluación cruzada reveló que arquitecturas como MAWI (cuyas trazas están limitadas a 15 minutos continuos) resultan estructuralmente incompatibles para validar modelos de *Machine Learning* que dependen de ventanas temporales (w) extensas, estableciendo un precedente crítico para futuras investigaciones sobre dichos repositorios.

Contribuciones de ingeniería de *software*

- **Decodificador *libpcap* de bajo perfil de memoria (*Streaming*):** Se desarrolló un *script* original en Python capaz de interpretar la estructura binaria de paquetes a nivel de *byte* sin saturar la memoria RAM. Esta herramienta viabiliza el procesamiento de decenas de millones de paquetes en estaciones de trabajo convencionales, democratizando el análisis de grandes volúmenes de telemetría de red.
- **Instrumentación de telemetría asíncrona mediante RouterOS:** Ante la imposibilidad técnica de emplear SNMP por restricciones de infraestructura (ausencia de IP pública), se instrumentó una solución *ad hoc* explotando los *scripts* nativos y el *Scheduler* del equipo MikroTik. Este desarrollo aporta una plantilla práctica invaluable para el monitoreo de redes remotas sujetas a políticas de seguridad estrictas.

7.2. Limitaciones del estudio

La rigurosidad de toda investigación en ingeniería exige el reconocimiento explícito de sus fronteras metodológicas y empíricas. Las limitaciones documentadas en esta sección no invalidan los hallazgos expuestos en los capítulos precedentes, sino que delimitan con precisión su contexto de aplicabilidad, garantizando la honestidad intelectual del estudio y estableciendo el punto de partida exacto para la continuidad del desarrollo tecnológico.

La Tabla 7.1 sintetiza el escrutinio de estas restricciones, categorizándolas según su origen estructural: (a) limitaciones de alcance declaradas por diseño; (b) restricciones logísticas y computacionales derivadas del entorno de ejecución; y (c) limitantes topológicas y algorítmicas descubiertas empíricamente durante la fase de evaluación. Para cada restricción se identifica su efecto directo sobre las conclusiones y se vincula con la línea de trabajo futuro destinada a solventarla.

Limitación	**Origen**	**Efecto sobre las conclusiones**	**Continuación**
Ausencia de interfaz gráfica, empaquetado de *software* u operación en producción.	(a)	Ninguno sobre la viabilidad predictiva; restringe la utilidad operativa inmediata para administradores.	Trabajo futuro 1
Carencia de alertamiento en tiempo real y evaluación del Tiempo de Antelación (TAE) operativo.	(a)/(c)	La utilidad para la toma de decisiones se argumenta a nivel analítico, sin demostración en flujo continuo.	Trabajo futuro 2
Recolección acotada a 21 días (82.9 % de cobertura) frente a los 60 días proyectados.	(b)	Reducción de ciclos estacionales evaluables; ausencia empírica de eventos de congestión masiva nativa.	Trabajo futuro 3
Evaluación sobre una topología única con régimen de letargo extremo (utilización máxima 1.44 %).	(b)/(c)	Impide la extrapolación directa de los pesos y umbrales algorítmicos hacia infraestructuras corporativas densas.	Trabajo futuro 3
Desempeño de clasificación evaluado exclusivamente sobre inyecciones sintéticas aisladas.	(c)	El *Recall* documentado representa una cota estadística de sensibilidad, no una métrica operativa sobre crisis reales.	Trabajo futuro 4
Diseño de características (*features*) restringido a la utilización del canal.	(c)	Omite el contraste empírico de vectores latentes como pérdida de paquetes, RTT y periodicidad de calendario.	Trabajo futuro 5
Arquitecturas SVR y GRU descartadas; búsqueda de hiperparámetros acotada y ejecución exclusiva en CPU.	(a)/(b)	Conclusiones circunscritas estrictamente a los tres algoritmos base bajo configuraciones estructurales conservadoras.	Trabajo futuro 5
Etiqueta de estado y umbral de congestión (U_{th}) fijados analíticamente, sin contraste dinámico.	(c)	Simplifica la detección, ignorando la adaptabilidad requerida para líneas base de tráfico mutables.	Trabajo futuro 4
Ejecución única sin cálculo de intervalos de confianza; evaluación de MAWI reducida a 6 tensores.	(b)/(c)	Las brechas de magnitud superior son categóricas, pero las variaciones porcentuales finas carecen de significancia estadística plena.	Trabajo futuro 6
Validación de generalización (MAWI) ejecutable de manera exclusiva para el modelo ARIMA.	(c)	Ausencia total de evidencia sobre la portabilidad matemática de *Random Forest* y LSTM ante restricciones de ventana temporal.	Trabajo futuro 6

Cuadro 7.1: Síntesis de limitaciones operativas, logísticas y empíricas del estudio, su impacto en las afirmaciones de la tesis y su directriz metodológica de resolución.

Alcance del prototipo y métricas operativas

El alcance operativo del prototipo se circunscribió estrictamente a la demostración matemática de viabilidad predictiva en un entorno fuera de línea (*offline*), prescindiendo del desarrollo de interfaces gráficas, ingestión continua de telemetría o arquitecturas de reentrenamiento autónomo (*concept drift*). Esta decisión priorizó el rigor del diseño experimental y la evaluación algorítmica sobre el empaquetado de un producto comercial.

Como consecuencia directa de este enfoque y de la escasez de anomalías reales, los mecanismos automatizados de alertamiento y la métrica de Tiempo de Anticipación Efectivo (TAE) no fueron evaluados operativamente. Dado que la congestión sostenida en el *dataset* primario provino de inyecciones sintéticas con rampas de ascenso predefinidas, el cálculo del TAE habría medido la morfología estática del generador algorítmico y no la agudeza orgánica del modelo, careciendo además de significancia estadística ante el reducido volumen de detecciones efectivas en la tarea de clasificación.

Restricciones topológicas y dimensionalidad de variables

La recolección de telemetría se acotó a 21 días efectivos sobre una única infraestructura PYME, caracterizada por un régimen de tráfico crónicamente ocioso (utilización ≤ 1.44 %). Esta limitación topológica impidió el modelado sobre anomalías orgánicas, por lo que los pesos sinápticos resultantes están fuertemente condicionados a este ecosistema particular y exigen un proceso de *fine-tuning* previo a cualquier extrapolación.

Asimismo, el espacio de características (*features*) se contrajo de forma exclusiva al factor de utilización y sus rezagos. El RTT y la pérdida de paquetes reales carecían de entropía operativa (con asintonía a cero); intentar inyectarlos de forma sintética habría provocado una fuga de datos (*target leakage*), al derivarse determinísticamente de la misma ecuación de congestión (Ecuación 4.1). Las restricciones computacionales también limitaron la evaluación a tres arquitecturas representativas (excluyendo alternativas como SVR y GRU), evaluadas en un único horizonte predictivo ($h = 5$ minutos) y procesadas íntegramente en CPU, acotando la validez estricta de los resultados a dichas configuraciones.

Sesgos en la inyección sintética y evaluación

La validación de la tarea de clasificación descansa sobre datos sintéticos, lo que conlleva tres sesgos estadísticos explícitos: (1) la ausencia de inercia estocástica temporal en el ruido de fondo (producto del remuestreo independiente); (2) el uso de perfiles de congestión geométricamente estilizados (trapezoidales) que podrían inducir al modelo a detectar patrones matemáticos artificiales en lugar de saturaciones orgánicas; y (3) la ejecución de la evaluación sobre la misma distribución generativa del entrenamiento. Estos sesgos garantizan una comparación transversal justa entre modelos, pero prohíben interpretar numéricamente el *Recall* documentado como una garantía de rendimiento infalible en un entorno de producción inexplorado.

Por su parte, el protocolo de evaluación careció de repeticiones masivas estocásticas (estilo Montecarlo) para el cálculo de intervalos de confianza, y la validación de generalización contra MAWI se restringió a 6 observaciones exclusivas para ARIMA. Esto ocurrió debido a que

las ventanas estructurales de los modelos de *Machine Learning* ($w = 30$ minutos) resultaron algorítmicamente incompatibles con las capturas fragmentadas de 15 minutos del repositorio japonés, impidiendo evaluar empíricamente su portabilidad inter-dominio.

Alcance absoluto de las conclusiones

Como epílogo de este ejercicio crítico, la Tabla 7.2 traza una frontera diáfana entre las afirmaciones que cuentan con el respaldo irrestricto de la evidencia recabada y aquellas presunciones que el rigor metodológico desautoriza categóricamente.

Afirmaciones sostenidas por la evidencia	**Presunciones no sostenidas por la evidencia**
LSTM predice con el menor error probabilístico el flujo de tráfico orgánico de la red privada estudiada (Sección 6.3).	Que cualquiera de estos modelos logre detectar una crisis de congestión puramente humana con la misma sensibilidad reportada sobre los datos sintéticos.
Bajo las rigurosas condiciones evaluadas, ningún modelo algorítmico logra dominar de manera simultánea las tareas de regresión continua y clasificación discreta (Sección 6.4).	Que el orden jerárquico de eficacia (LSTM > *Random Forest* > ARIMA en regresión) se mantenga inmutable ante nuevas *features* o arquitecturas.
En escenarios de red asimétricos (masivo desbalance de clases), la Exactitud (*Accuracy*) resulta ser una métrica matemáticamente engañosa e insuficiente (Figura 6.29).	Que la arquitectura resultante esté lista para un despliegue operativo en producción sin requerir fases de prueba beta y evaluación de latencia.
Un algoritmo ARIMA calibrado en la dinámica ociosa de una red menor fracasa estrepitosamente al intentar generalizar sobre un enlace troncal académico de uso intenso (Sección 6.1).	Que los algoritmos de *Machine Learning* sean infalibles a la hora de generalizar, dado que la disparidad en las longitudes de captura impidió comprobarlo.

Cuadro 7.2: Delimitación rigurosa del alcance científico de las conclusiones dictaminadas en esta investigación.

La exposición de estas limitaciones metodológicas no invalida el peso científico del dictamen central de la investigación: el diseño experimental implementado logró aislar el impacto de estas variables condicionantes, garantizando que la comparativa transversal definitiva entre los tres modelos matemáticos se mantuviera inquebrantablemente justa, equitativa y controlada.

7.3. Trabajo futuro

Las limitaciones y descubrimientos empíricos documentados en esta tesis no representan un punto final, sino que trazan una hoja de ruta estructurada para la evolución de los sistemas predictivos de red. Se proponen las siguientes seis líneas estratégicas de continuación:

1. **Operacionalización y despliegue en tiempo real.** Evolucionar los *scripts* algorítmicos validados *offline* hacia una arquitectura de producción. Esto requiere desarrollar conductos de ingestión continua de telemetría (vía API o gRPC nativo de *routers*), interfaces gráficas de usuario (*dashboards*) y rutinas de reentrenamiento autónomo que mitiguen el cambio de concepto (*concept drift*).

2. **Alertamiento híbrido y evaluación operativa del TAE.** Instrumentar en un entorno vivo el orquestador conceptualizado en la Figura 6.32, empleando *Random Forest* como percutor rápido de alarmas y LSTM como proyector de impacto continuo. Esto permitirá medir empíricamente el Tiempo de Anticipación Efectivo (TAE) ante crisis orgánicas y validar la integración de mitigaciones automatizadas en arquitecturas *Software-Defined Networking* (SDN).

3. **Expansión longitudinal y escalabilidad topológica.** Extender la captura de telemetría a ciclos multimensuales para internalizar patrones estacionales (e.g., respaldos masivos y mantenimientos). Paralelamente, evaluar el despliegue del sistema en infraestructuras corporativas densas, analizando cómo debe mutar la dimensionalidad de las características (*features*) al transitar entre distintas escalas de red.

4. **Validación orgánica del umbral y rediseño de la variable objetivo.** Sustituir la inyección de tráfico sintético mediante la correlación con bitácoras reales de incidentes (*tickets* de soporte). Esto permitirá validar empíricamente si el umbral heurístico del 80 % ($U_{th} = 0{,}80$) detona inflexiones comprobables en el RTT y el *Jitter* a nivel de usuario, y facilitará la reprogramación del tensor para evaluar congestión sostenida en lugar del cruce probabilístico instantáneo.

5. **Enriquecimiento dimensional y arquitecturas de frontera.** Integrar al tensor predictivo variables de Capa 4 (e.g., retransmisiones TCP) y codificaciones temporales, evaluándolas sobre una red con mayor entropía dinámica. Bajo estas nuevas condiciones, escalar la comparativa algorítmica hacia topologías SVR, GRU y Mecanismos de Atención (*Transformers*), apalancando clústeres GPU para búsquedas hiperparamétricas exhaustivas.

6. **Robustez estocástica y portabilidad inter-dominio.** Automatizar pruebas de reentrenamiento estocástico masivo (métodos Montecarlo) para establecer intervalos de confianza estadísticamente irrefutables. Asimismo, desarrollar estrategias de adaptación de dimensionalidad (*padding*) que compatibilicen ventanas tensoriales amplias (e.g., $w = 30$ minutos) con capturas históricas breves (e.g., 15 minutos en repositorios MAWI), destrabando la evaluación externa de algoritmos de *Deep Learning*.

El generador sintético se calibró con la distribución horaria de los 21 días reales, incluidos los periodos posteriormente empleados como validación y prueba; en consecuencia, el entrenamiento incorpora información distribucional marginal (no secuencial) de dichos periodos, y los días sintéticos de entrenamiento poseen fechas posteriores al conjunto de prueba real. Este efecto se estima moderado, pues solo se remuestrearon valores aislados por hora, sin conservar su orden temporal, pero impide afirmar una separación temporal estricta entre entrenamiento y evaluación.

Bibliografía

[1] Cisco Systems, "Cisco annual internet report (2018–2023) white paper," Cisco, Tech. Rep., 2020, consultado el 25 de mayo de 2026. [Online]. Available: https://www.cisco.com/c/en/us/solutions/collateral/executive-perspectives/annual-internet-report/white-paper-c11-741490.html

[2] R. Boutaba, M. A. Salahuddin, N. Limam, S. Ayoubi, N. Shahriar, F. Estrada-Solano, and O. M. Caicedo, "A comprehensive survey on machine learning for networking: evolution, applications and research opportunities," *Journal of Internet Services and Applications*, vol. 9, no. 1, p. 16, 2018.

[3] J. Zhang, Y. Liu, Z. Li, and Y. Lu, "Network traffic prediction using deep learning: An experimental study," in *2020 IEEE 6th International Conference on Computer and Communications (ICCC)*, 2020, pp. 1320–1325.

[4] A. S. Tanenbaum and D. J. Wetherall, *Computer Networks*, 5th ed. Boston: Pearson Prentice Hall, 2011.

[5] J. F. Kurose and K. W. Ross, *Computer Networking: A Top-Down Approach*, 7th ed. Boston: Pearson, 2017.

[6] L. Kleinrock, *Queueing Systems, Volume II: Computer Applications*. New York: John Wiley & Sons, 1976.

[7] V. Jacobson, "Congestion avoidance and control," in *Symposium proceedings on Communications architectures and protocols (SIGCOMM '88)*, vol. 18, no. 4. Stanford, CA, USA: ACM, 1988, pp. 314–329.

[8] S. Floyd, "Congestion control principles," RFC 2914, Internet Engineering Task Force (IETF), BCP 41, September 2000. [Online]. Available: https://www.rfc-editor.org/rfc/rfc2914

[9] H. Schulzrinne, S. Casner, R. Frederick, and V. Jacobson, "Rtp: A transport protocol for real-time applications," RFC 3550, Internet Engineering Task Force (IETF), July 2003. [Online]. Available: https://www.rfc-editor.org/rfc/rfc3550

[10] G. E. P. Box, G. M. Jenkins, G. C. Reinsel, and G. M. Ljung, *Time Series Analysis: Forecasting and Control*, 5th ed. Hoboken, NJ: John Wiley & Sons, 2015.

[11] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[12] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[13] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, "Learning phrase representations using rnn encoder-decoder for statistical machine translation," in *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, Doha, Qatar, 2014, pp. 1724–1734.

[14] H. Zare Moayedi and M. A. Masnadi-Shirazi, "Arima model for network traffic prediction and anomaly detection," in *2008 International Symposium on Information Technology (ITSim)*, vol. 4. IEEE, 2008, pp. 1–6.

[15] F. Tian, "Network traffic prediction method based on autoregressive integrated moving average and adaptive volterra filter," *International Journal of Communication Systems*, vol. 34, no. 12, p. e4891, 2021.

[16] I. Kochetkova, A. Kushchazli, S. Burtseva, and A. Gorshenin, "Short-term mobile network traffic forecasting using seasonal arima and holt-winters models," *Future Internet*, vol. 15, no. 9, p. 290, 2023.

[17] Y. V. Sneha, Vimitha, Vishwasini, S. Boloor, and N. D. Adesh, "Prediction of network congestion at router using machine learning technique," in *2020 IEEE International Conference on Distributed Computing, VLSI, Electrical Circuits and Robotics (DISCOVER)*. IEEE, 2020, pp. 188–193.

[18] Z. L. Qian, D. C. Juan, P. Bogdan, C. Y. Tsui, D. Marculescu, and R. Marculescu, "A support vector regression (svr)-based latency model for network-on-chip (noc) architectures," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 35, no. 3, pp. 471–484, 2016.

[19] M. Labonne, C. Chatzinakis, and A. Olivereau, "Predicting bandwidth utilization on network links using machine learning," arXiv:2112.02417, 2021. [Online]. Available: https://arxiv.org/abs/2112.02417

[20] T. P. Oliveira, J. S. Barbar, and A. S. Soares, "Computer network traffic prediction: A comparison between traditional and deep learning neural networks," *International Journal of Big Data Intelligence*, vol. 3, no. 1, pp. 28–37, 2016.

[21] S. Saha, A. Haque, and G. Sidebottom, "Overcoming data limitations in internet traffic forecasting: Lstm models with transfer learning and wavelet augmentation," arXiv:2409.13181, 2024. [Online]. Available: https://arxiv.org/abs/2409.13181

[22] G. L. Santos, P. Rosati, T. Lynn, J. Kelner, D. Sadok, and P. T. Endo, "Predicting short-term mobile internet traffic from internet activity using recurrent neural networks," *International Journal of Network Management*, vol. 32, no. 3, p. e2191, 2022.

[23] MAWI Working Group, "Mawi working group traffic archive," WIDE Project, consultado en 2026. [Online]. Available: https://mawi.wide.ad.jp/mawi/

[24] R. Hernández-Sampieri, *Metodología de la Investigación: Las Rutas Cuantitativa, Cualitativa y Mixta*, 2nd ed. Ciudad de México: McGraw-Hill, 2023.

[25] A. Boukerche, Y. Sun, and M. Z. A. Bhuiyan, "Performance evaluation of machine learning-based network traffic prediction approaches: A survey," *IEEE Communications Surveys & Tutorials*, vol. 23, no. 1, pp. 405–428, 2021.

Apéndice A

Código fuente de ejemplos críticos

Este apéndice reproduce el código fuente de las piezas de software consideradas más relevantes para la comprensión y reproducibilidad del prototipo desarrollado en esta tesis. Se excluyen deliberadamente los scripts de configuración de entorno, corrección de errores y depuración que no aportan al entendimiento metodológico del trabajo. El repositorio completo del proyecto contiene la totalidad de los scripts empleados.

A.1. Decodificador de trazas libpcap

El siguiente fragmento corresponde al decodificador propio empleado para extraer telemetría de las trazas de MAWI (Sección 5.2), evitando la sobrecarga de memoria de herramientas de disección de protocolos de alto nivel. A diferencia del fragmento presentado en el Capítulo 5 (Código 5.1), aquí se incluye el manejo completo de las cuatro variantes del encabezado global de *libpcap* (orden de bytes y precisión temporal).

Listing A.1: Decodificador completo de trazas libpcap.

```
import struct
from collections import defaultdict

def parsear_pcap(pcap_path, ventana_seg=60):
    """
␣␣␣␣Lee␣un␣archivo␣.pcap␣directamente␣(formato␣clasico␣libpcap),␣extrayendo
␣␣␣␣timestamp␣y␣tamano␣original␣de␣cada␣paquete␣SIN␣decodificar␣protocolos.
␣␣␣␣Agrega␣el␣volumen␣de␣bytes␣transmitidos␣en␣ventanas␣temporales␣discretas.
␣␣␣␣"""
    acumulado = defaultdict(int)
    contador = 0

    with open(pcap_path, "rb") as f:
        # --- Encabezado global (24 bytes) ---
        cabecera_global = f.read(24)
        magic = struct.unpack("I", cabecera_global[0:4])[0]

```

```
        if magic == 0xa1b2c3d4:
            endian = "<"   # little-endian, microsegundos
            nano = False
        elif magic == 0xd4c3b2a1:
            endian = ">"   # big-endian, microsegundos
            nano = False
        elif magic == 0xa1b23c4d:
            endian = "<"   # little-endian, nanosegundos
            nano = True
        elif magic == 0x4d3cb2a1:
            endian = ">"   # big-endian, nanosegundos
            nano = True
        else:
            raise ValueError(f"Magic number no reconocido: {hex(magic)}")

        fmt_registro = endian + "IIII"  # ts_sec, ts_usec, incl_len,
            orig_len

        while True:
            encabezado_paquete = f.read(16)
            if len(encabezado_paquete) < 16:
                break  # fin del archivo

            ts_sec, ts_usec, incl_len, orig_len = struct.unpack(
                fmt_registro, encabezado_paquete
            )

            if nano:
                timestamp = ts_sec + (ts_usec / 1_000_000_000)
            else:
                timestamp = ts_sec + (ts_usec / 1_000_000)

            ventana = int(timestamp // ventana_seg)
            acumulado[ventana] += orig_len
            contador += 1

            f.seek(incl_len, 1)  # salta los bytes del paquete sin leerlos

    filas = [{"timestamp": v * ventana_seg, "throughput_bps": (b * 8) /
        ventana_seg}
             for v, b in sorted(acumulado.items())]
    return filas, contador
```

A.2. Script de recolección en RouterOS

El siguiente script, escrito en RouterOS Script y ejecutado periódicamente mediante el Programador de Tareas (*Scheduler*), constituye el mecanismo de recolección del dataset primario (Sección 4.4), adoptado como alternativa funcionalmente equivalente a SNMP ante las restricciones de acceso remoto descritas en la Sección 5.1.

Listing A.2: Script de recolección de telemetría en RouterOS.

```
:local wanInterface "ether1";
:local pingTarget "1.1.1.1";

:local traf [/interface monitor-traffic interface=$wanInterface once as-value];
:local rxbps ($traf->"rx-bits-per-second");
:local txbps ($traf->"tx-bits-per-second");

:local pingResult [/ping $pingTarget count=5 as-value];
:local recibidos [:len $pingResult];
:local perdida ((5 - $recibidos) * 100 / 5);

:local sumaUs 0;
:foreach r in=$pingResult do={
    :local timeStr [:tostr ($r->"time")];
    :local hh [:tonum [:pick $timeStr 0 2]];
    :local mm [:tonum [:pick $timeStr 3 5]];
    :local ss [:tonum [:pick $timeStr 6 8]];
    :local frac [:tonum [:pick $timeStr 9 15]];
    :local totalUs (((($hh*3600)+($mm*60)+$ss)*1000000 + $frac);
    :set sumaUs ($sumaUs + $totalUs);
};
:local rttUsProm 0;
:if ($recibidos > 0) do={ :set rttUsProm ($sumaUs / $recibidos) };

:local fecha [:tostr [/system clock get date]];
:local hora [:tostr [/system clock get time]];
:local linea ($fecha . "," . $hora . "," . $rxbps . "," . $txbps . "," . $rttUsProm . "," . $perdida);
:local nombreArchivo ("log_" . $fecha . ".csv");

:if ([:len [/file find name=$nombreArchivo]] = 0) do={
    /file add name=$nombreArchivo contents=("fecha,hora,rx_bps,tx_bps,rtt_us,perdida_pct\n" . $linea . "\n")
} else={
    :local actual [/file get [find name=$nombreArchivo] contents];
    /file set [find name=$nombreArchivo] contents=($actual . $linea . "\n")
}
```

Este script se programó mediante `/system scheduler add name=run-recolectar interval=1m on-event=recolectar-metrica`, ejecutándose de manera autónoma cada minuto durante todo el periodo de recolección, con respaldo local resiliente a interrupciones de la conectividad VPN (Tabla 5.1).

A.3. Pipeline de preprocesamiento

El siguiente script implementa las etapas de limpieza, detección de sesiones, generación de variables de rezago y definición de las variables objetivo de regresión y clasificación (Sección 5.2), aplicado de manera uniforme a los cinco conjuntos derivados del dataset primario.

Listing A.3: Pipeline de limpieza, detección de sesiones y generación de variables.

```
import pandas as pd
import numpy as np

BANDA_NOMINAL = 1_000_000_000
U_TH = 0.80
H = 5
UMBRAL_SESION_MIN = 5
N_REZAGOS = 5
VENTANA_MEDIA_MOVIL = 30

def procesar(path_entrada, nombre_salida, es_train=False):
    df = pd.read_csv(path_entrada)
    df["timestamp"] = pd.to_datetime(df["timestamp"])
    df = df.sort_values("timestamp").reset_index(drop=True)
    df["utilizacion"] = df["rx_bps"] / BANDA_NOMINAL

    # --- Deteccion de sesiones (huecos grandes = frontera) ---
    diffs = df["timestamp"].diff().dt.total_seconds().fillna(0) / 60.0
    df["sesion_id"] = (diffs > UMBRAL_SESION_MIN).cumsum()

    # --- Limpieza por sesion: reindexar, interpolar huecos cortos ---
    sesiones_limpias = []
    for sid, grupo in df.groupby("sesion_id"):
        grupo = grupo.sort_values("timestamp")
        rango = pd.date_range(grupo["timestamp"].min(), grupo["timestamp"
            ].max(), freq="1min")
        g_completo = pd.DataFrame({"timestamp": rango})
        g_completo = g_completo.merge(
            grupo[["timestamp", "utilizacion", "rtt_us", "perdida_pct", "
                es_sintetico"]],
            on="timestamp", how="left"
        )
        g_completo["utilizacion"] = g_completo["utilizacion"].interpolate(
            method="linear", limit=3)
        g_completo["rtt_us"] = g_completo["rtt_us"].interpolate(method="
            linear", limit=3)
        g_completo["perdida_pct"] = g_completo["perdida_pct"].interpolate(
            method="linear", limit=3)
        g_completo["es_sintetico"] = g_completo["es_sintetico"].ffill().
            bfill()
        g_completo["sesion_id"] = sid
        sesiones_limpias.append(g_completo)

    df_limpio = pd.concat(sesiones_limpias, ignore_index=True)
    df_limpio = df_limpio.dropna(subset=["utilizacion"]).reset_index(drop=
        True)
```

```
40
41      # --- Rezagos y media movil, calculados POR SESION ---
42      for i in range(1, N_REZAGOS + 1):
43          df_limpio[f"U_lag{i}"] = df_limpio.groupby("sesion_id")["
                utilizacion"].shift(i)
44
45      df_limpio["U_media_movil"] = (
46          df_limpio.groupby("sesion_id")["utilizacion"]
47          .rolling(window=VENTANA_MEDIA_MOVIL, min_periods=
                VENTANA_MEDIA_MOVIL)
48          .mean()
49          .reset_index(level=0, drop=True)
50      )
51
52      # --- Variables objetivo: regresion U(t+h) y clasificacion U(t+h) >=
            U_th ---
53      df_limpio["target_regresion"] = df_limpio.groupby("sesion_id")["
            utilizacion"].shift(-H)
54      df_limpio["target_clasificacion"] = (df_limpio["target_regresion"] >=
            U_TH).astype(float)
55      df_limpio.loc[df_limpio["target_regresion"].isna(), "
            target_clasificacion"] = np.nan
56
57      columnas_clave = [f"U_lag{i}" for i in range(1, N_REZAGOS+1)] + ["
            U_media_movil", "target_regresion"]
58      df_final = df_limpio.dropna(subset=columnas_clave).reset_index(drop=
            True)
59
60      df_final.to_csv(nombre_salida, index=False)
61      return df_final
```

Listing A.4: Normalización Min-Max sin fuga de datos (ajustada exclusivamente con el conjunto de entrenamiento).

```
1  import pandas as pd
2
3  COLS_NORM = ["utilizacion", "U_lag1", "U_lag2", "U_lag3", "U_lag4", "
       U_lag5",
4               "U_media_movil", "target_regresion"]
5
6  def normalizar(train_df, path_entrada, nombre_salida, min_vals, max_vals):
7      df = pd.read_csv(path_entrada)
8      df_norm = df.copy()
9      for col in COLS_NORM:
10         rango = max_vals[col] - min_vals[col]
11         df_norm[col] = (df[col] - min_vals[col]) / rango if rango != 0
               else 0.0
12     df_norm.to_csv(nombre_salida, index=False)
13     return df_norm
14
15 train = pd.read_csv("train_procesado.csv")
16 min_vals = train[COLS_NORM].min()
17 max_vals = train[COLS_NORM].max()
```

Listing A.5: Construcción de secuencias mediante ventana deslizante, respetando fronteras de sesión.

```python
import pandas as pd
import numpy as np

def construir_ventanas(path_csv, w=30):
    df = pd.read_csv(path_csv)
    X, y_reg, y_clas = [], [], []
    for sid, grupo in df.groupby("sesion_id"):
        grupo = grupo.reset_index(drop=True)
        serie = grupo["utilizacion"].values
        target_reg = grupo["target_regresion"].values
        target_clas = grupo["target_clasificacion"].values
        n = len(serie)
        for i in range(n - w + 1):
            ventana = serie[i: i + w]
            idx_target = i + w - 1
            X.append(ventana)
            y_reg.append(target_reg[idx_target])
            y_clas.append(target_clas[idx_target])
    return np.array(X), np.array(y_reg), np.array(y_clas)
```

A.4. Generador de datos sintéticos

El siguiente script implementa la estrategia de aumento de datos descrita en la Sección 5.1: generación de tráfico normal mediante *bootstrap* horario de datos reales, e inyección de eventos de congestión con perfil trapezoidal cuyo RTT y pérdida de paquetes se derivan mediante el modelo de teoría de colas de la Sección 2.1.3.

Listing A.6: Generador de tráfico sintético calibrado con datos reales.

```python
import pandas as pd
import numpy as np

np.random.seed(42)
BANDA_NOMINAL = 1_000_000_000

def generar_dia_normal(fecha, pools_rx_por_hora, pools_tx_por_hora,
    pools_rtt_por_hora):
    """Genera 1440 minutos de trafico 'normal' por bootstrap horario de
        datos reales."""
    filas = []
    for minuto in range(1440):
        hora = minuto // 60
        rx = np.random.choice(pools_rx_por_hora[hora]) * np.random.normal
            (1.0, 0.05)
        tx = np.random.choice(pools_tx_por_hora[hora]) * np.random.normal
            (1.0, 0.05)
        rtt = np.random.choice(pools_rtt_por_hora[hora])
        ts = fecha + pd.Timedelta(minutes=minuto)
        filas.append({"timestamp": ts, "rx_bps": max(0, rx), "tx_bps": max
            (0, tx),
```

```
                          "rtt_us": rtt, "perdida_pct": 0})
    df_dia = pd.DataFrame(filas)
    return df_dia.astype({"rx_bps": "float64", "tx_bps": "float64",
                          "rtt_us": "float64", "perdida_pct": "float64"})

def perfil_rampa(n_puntos, u_pico):
    """Perfil trapezoidal: sube (20%) - se mantiene (60%) - baja (20%)."""
    n_subida = max(1, int(n_puntos * 0.2))
    n_bajada = max(1, int(n_puntos * 0.2))
    n_meseta = n_puntos - n_subida - n_bajada
    subida = np.linspace(0.10, u_pico, n_subida)
    meseta = np.full(n_meseta, u_pico) * np.random.normal(1.0, 0.03,
        n_meseta)
    bajada = np.linspace(u_pico, 0.10, n_bajada)
    return np.clip(np.concatenate([subida, meseta, bajada]), 0, 0.999)

def rtt_por_teoria_colas(utilizacion, rtt_base_us):
    """RTT(rho) = rtt_base / (1 - rho), consistente con Kleinrock (Seccion
        2.1.2)."""
    rho = np.clip(utilizacion, 0, 0.98)
    return rtt_base_us / (1 - rho)

def perdida_por_utilizacion(utilizacion):
    """Perdida de paquetes: 0% hasta ~0.85, luego crece cuadraticamente.""
        "
    return np.where(
        utilizacion < 0.85, 0,
        np.clip((utilizacion - 0.85) / 0.15, 0, 1) ** 2 * 15
    )

def inyectar_eventos_congestion(df_dia, n_eventos, umbral=0.80):
    """Inyecta eventos de congestion con perfil de rampa, duracion e
        intensidad variadas."""
    df_dia = df_dia.copy()
    n = len(df_dia)
    eventos_generados = []
    for _ in range(n_eventos):
        duracion = np.random.randint(5, 30)
        inicio = np.random.randint(0, n - duracion)
        u_pico = np.random.uniform(umbral + 0.01, min(0.98, umbral + 0.35)
            )
        perfil = perfil_rampa(duracion, u_pico)

        idx = df_dia.index[inicio: inicio + duracion]
        df_dia.loc[idx, "rx_bps"] = perfil * BANDA_NOMINAL
        df_dia.loc[idx, "tx_bps"] = perfil * BANDA_NOMINAL * np.random.
            uniform(0.85, 1.0, len(idx))
        df_dia.loc[idx, "rtt_us"] = rtt_por_teoria_colas(perfil,
            RTT_BASE_US)
        df_dia.loc[idx, "perdida_pct"] = perdida_por_utilizacion(perfil)

        eventos_generados.append({
            "inicio": df_dia.loc[idx[0], "timestamp"],
            "duracion_min": duracion,
```

```
            "u_pico": round(u_pico, 3)
        })
    return df_dia, eventos_generados
```

Listing A.7: Verificación automática de la ausencia de fuga de datos entre entrenamiento y los conjuntos de evaluación.

```
# Verificaciones de seguridad, ejecutadas tras ensamblar los conjuntos finales
ts_train = set(train_final["timestamp"])

assert len(set(val_regresion["timestamp"]) & ts_train) == 0, \
    "FUGA: val_regresion se solapa con train"
assert len(set(test_regresion["timestamp"]) & ts_train) == 0, \
    "FUGA: test_regresion se solapa con train"
assert len(set(val_clasificacion["timestamp"]) & ts_train) == 0, \
    "FUGA: val_clasificacion se solapa con train"
assert len(set(test_clasificacion["timestamp"]) & ts_train) == 0, \
    "FUGA: test_clasificacion se solapa con train"

assert val_regresion["es_sintetico"].sum() == 0, "ERROR: val_regresion contiene sintetico"
assert test_regresion["es_sintetico"].sum() == 0, "ERROR: test_regresion contiene sintetico"
assert val_clasificacion["es_sintetico"].sum() == len(val_clasificacion), \
    "ERROR: val_clasificacion contiene datos reales"
assert test_clasificacion["es_sintetico"].sum() == len(test_clasificacion), \
    "ERROR: test_clasificacion contiene datos reales"

assert (val_clasificacion["utilizacion"] >= U_TH).sum() > 0, \
    "ERROR: val_clasificacion no tiene casos positivos"
assert (test_clasificacion["utilizacion"] >= U_TH).sum() > 0, \
    "ERROR: test_clasificacion no tiene casos positivos"
```

A.5. Entrenamiento y evaluación de modelos

A continuación se presenta la implementación de cada uno de los tres modelos evaluados, incluyendo la función común de métricas empleada de manera consistente en los Capítulos 5 y 6.

Listing A.8: Función común de cálculo de métricas de clasificación (Accuracy, Precision, Recall, F1, AUC-ROC).

```
from sklearn.metrics import (roc_auc_score, precision_score, recall_score,
                             f1_score, accuracy_score, confusion_matrix)

def evaluar_clasificacion(y_real_cont, y_pred_cont, umbral):
    y_real_bin = (y_real_cont >= umbral).astype(int)
    y_pred_bin = (y_pred_cont >= umbral).astype(int)
    if y_real_bin.sum() == 0:
        return None
    return {
        "accuracy": accuracy_score(y_real_bin, y_pred_bin),
        "precision": precision_score(y_real_bin, y_pred_bin, zero_division
           =0),
        "recall": recall_score(y_real_bin, y_pred_bin, zero_division=0),
        "f1": f1_score(y_real_bin, y_pred_bin, zero_division=0),
        "auc": roc_auc_score(y_real_bin, y_pred_cont),
        "matriz_confusion": confusion_matrix(y_real_bin, y_pred_bin)
    }
```

Listing A.9: ARIMA: identificación de orden y pronóstico progresivo por bloques.

```
import numpy as np
from statsmodels.tsa.stattools import adfuller
from statsmodels.tsa.arima.model import ARIMA

def identificar_orden(serie_train, d_sugerido, p_max=3, q_max=3):
    mejor_aic, mejor_orden = np.inf, None
    for p in range(p_max + 1):
        for q in range(q_max + 1):
            try:
                m = ARIMA(serie_train, order=(p, d_sugerido, q)).fit()
                if m.aic < mejor_aic:
                    mejor_aic, mejor_orden = m.aic, (p, d_sugerido, q)
            except Exception:
                continue
    return mejor_orden, mejor_aic

def pronostico_bloques(modelo_ajustado, serie, h=5):
    """Walk-forward por bloques de tamano h, revelando h valores reales
       por paso."""
    predicciones, reales = [], []
    modelo_actual = modelo_ajustado
    i = 0
    while i + h <= len(serie):
        pron = modelo_actual.get_forecast(steps=h).predicted_mean[-1]
        predicciones.append(pron)
```

```
        reales.append(serie[i + h - 1])
        modelo_actual = modelo_actual.append(serie[i:i+h], refit=False)
        i += h
    return np.array(predicciones), np.array(reales)

# Uso:
resultado_adf = adfuller(serie_train)
d = 0 if resultado_adf[1] <= 0.05 else 1
orden, aic = identificar_orden(serie_train, d)
modelo_final = ARIMA(serie_train, order=orden).fit()
```

Listing A.10: Random Forest: entrenamiento y análisis de importancia de variables.

```
import pandas as pd
from sklearn.ensemble import RandomForestRegressor

FEATURES = ["U_lag1", "U_lag2", "U_lag3", "U_lag4", "U_lag5", "
    U_media_movil"]

train = pd.read_csv("train_norm.csv")
X_train, y_train = train[FEATURES], train["target_regresion"]

modelo = RandomForestRegressor(n_estimators=200, max_depth=10,
    random_state=42, n_jobs=-1)
modelo.fit(X_train, y_train)

importancias = pd.Series(modelo.feature_importances_, index=FEATURES) \
                 .sort_values(ascending=False)

# Prediccion sobre cualquier conjunto de evaluacion:
def predecir(df_eval):
    return modelo.predict(df_eval[FEATURES])
```

Listing A.11: LSTM: arquitectura y entrenamiento con early stopping.

```
import numpy as np
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

np.random.seed(42)
tf.random.set_seed(42)

datos = np.load("ventanas_primario.npz")
X_train = datos["X_train"].reshape((-1, datos["X_train"].shape[1], 1))
y_train = datos["y_train_reg"]
X_val = datos["X_val_reg"].reshape((-1, datos["X_val_reg"].shape[1], 1))
y_val = datos["y_val_reg"]

modelo = keras.Sequential([
    layers.Input(shape=(X_train.shape[1], 1)),
    layers.LSTM(32, activation="tanh"),
    layers.Dense(16, activation="relu"),
    layers.Dense(1)
```

```
])
modelo.compile(optimizer="adam", loss="mse", metrics=["mae"])

early_stop = keras.callbacks.EarlyStopping(
    monitor="val_loss", patience=10, restore_best_weights=True
)

historial = modelo.fit(
    X_train, y_train,
    validation_data=(X_val, y_val),
    epochs=100, batch_size=64,
    callbacks=[early_stop], verbose=1
)
```